

—— // THE STRYKE ECOSYSTEM // ——

The Stryke Ecosystem

First Edition

THE STRYKE ECOSYSTEM

the connectors · the batteries · the package fleet

2026-07-18

MenkeTechnologies

—— MENKETECHNOLOGIES ——

Contents

Before You Begin	3
The Shape of the Fleet — the package model	5
The FFI Boundary in Depth — the JSON wire, the cache, the panic seam	7
stryke-postgres — the PostgreSQL client	10
stryke-mysql — the MySQL/MariaDB client	12
stryke-mssql — the SQL Server / Azure SQL client	14
stryke-mongo — the MongoDB client	16
stryke-redis — the Redis / Valkey client	19
stryke-neo4j — the Neo4j graph client	21
stryke-scylla — the ScyllaDB / Cassandra client	24
stryke-clickhouse — the ClickHouse client	27
stryke-search — the Elasticsearch / OpenSearch client	30
stryke-arrow — the Apache Arrow / Parquet columnar-data package	32
stryke-parquet — the Parquet file inspector	34
stryke-polars — the pandas + numpy surface	36
stryke-duckdb — the embedded analytical SQL engine	38
stryke-spark — the Apache Spark client	40
stryke-kafka — the Apache Kafka client	42
stryke-zmq — the ZeroMQ client	44
stryke-grpc — the reflection-based gRPC client	46
stryke-aws — the AWS client	48
stryke-gcp — the Google Cloud client	51
stryke-azure — the Azure cloud client	54
stryke-k8s — the Kubernetes client	56
stryke-docker — the Docker daemon client	58

stryke-scrape — the web scraping / crawling client	60
stryke-selenium — the browser automation client	63
stryke-mcpd — the MCP server discipline layer	66
stryke-email — the SMTP + campaign client	69
stryke-office — Excel, Word, PowerPoint, ODF, and PDF I/O	72
stryke-utils — the pure-stryke utility belt	74
stryke-gui — the desktop automation cdylib	77
Writing Your Own Battery — how a connector is built	79
Testing and Packaging a Connector	86
Versioning, the Store, and Compatibility	89
Embedding the Fleet in zshrs	92
Patterns Across the Fleet	94
Appendix A — The Catalogue of the Fleet	98
Appendix B — Connection & Configuration Reference	101
Appendix C — The Pure-Helper Layer	104
Reading the Source Alongside This Book	106

Before You Begin

This is the book about the batteries. The Stryke language ships a small, fast core — the interpreter, the bytecode VM it shares with `zshrs`, the one-liner surface that replaces `awk`, `sed`, and a shell pipeline. What the core deliberately does *not* ship is a connection to the outside world: a Postgres driver, a Kafka producer, an AWS SDK, a headless browser. Those live in a fleet of opt-in packages, one per external system, kept out of the core binary so the daily-driver install stays slim. This book is the map to that fleet and the full chapter for each package in it.

It is the companion to *the Stryke language reference*, not a replacement for it. That book documents the language — the grammar, the sigils, the `->>` threading, the bytecode model. This one starts where the language stops: at the edge where a stryke script reaches a database, a message broker, a cloud API, a spreadsheet, a web page. The reference tells you how to write the loop; this book tells you what the loop can talk to.

The structure follows from that split. After this preface comes one shared chapter — *the shape of the fleet* — that pulls out the package model every connector uses, so the per-package chapters can stay focused on their own system instead of re-explaining the FFI plumbing each time. Everything after that is a full chapter for one package, grounded in that package's own source: its `README`, its `Cargo.toml`, its `stryke.toml` manifest, and a real script from its `examples/` directory. No package is documented from memory, and no stryke syntax appears in these pages that was not copied or adapted from code that ships in the repo.

The book is honest about what each package is. Almost every connector wraps an existing, well-known Rust crate — `postgres`, `mysql`, `rdkafka`, `mongodb`, the `aws-sdk-*` family, `polars`, `arrow`, `duckdb`. The chapter names that crate and its version plainly. The original engineering is the *binding*: the stryke package that turns a crate's async, typed, builder-pattern API into a handful of verbs a one-liner can call, the in-process `cdylib` that carries the calls, the connection cache that makes repeated calls cheap. Where a package is a thin stub rather than a finished connector, the chapter says so and spends its pages on intent rather than pretending to a surface that is not there.

Where a number is provisional — a package version, an export count, a wrapped crate's version — the text takes it from the source at the time of writing and marks it as such. Versions move and export tables grow; a book that rounds those into permanent claims goes stale the day it prints.

THE STRYKE ECOSYSTEM – what this book contains

Front matter

The shape of the fleet the package model shared by every connector
The FFI boundary in depth the JSON wire, the connection cache, the panic seam

Part I – Databases

postgres · mysql · mssql · mongo · redis · neo4j · scylla · clickhouse · search

Part II – Data & analytics

arrow · parquet · polars · duckdb · spark

Part III – Messaging & RPC

kafka · zmq · grpc

Part IV – Cloud & infrastructure

aws · gcp · azure · k8s · docker

Part V – Web & automation

scrape · selenium · mcpd · email

Part VI – Documents & office

office

Part VII – Foundation

utils · gui

Part VIII – Building on the fleet

writing your own battery · testing & packaging a connector ·
versioning, the store & compatibility · embedding the fleet in zshrs ·
patterns across the fleet

Appendices

A the catalogue of the fleet · B connection & configuration reference ·
C the pure-helper layer

The [strykelang](#) language itself, and the packages that already carry their own book, are not re-documented here — they are cross-referenced to their own volumes. This book covers the connector fleet, and only the connector fleet.

The Shape of the Fleet — the package model

Every connector in this book is built the same way, and it is worth spending one chapter on that shared shape so the rest can be short. A stryke connector is a **package**: a thin stryke-language library plus a compiled Rust dynamic library, distributed and loaded as a unit, that adds one namespace of verbs to the language.

Why a package and not a builtin

The rationale is the same in every package's README, and it is a size argument, not a philosophical one. A real client for an external system pulls in a large dependency tree — a TLS stack, an async runtime, type encoders, the vendor SDK itself — often a hundred or more transitive crates. Most stryke one-liners never touch any of that; they read a file, reshape some fields, print a result. Paying for a Postgres driver or an AWS SDK on every invocation of a text-processing language would be the wrong trade. So the core binary ships without them, and the scripts that do need a database, a broker, or a cloud API opt in to exactly the one they need.

How a package is built

A connector repository produces two artifacts. The first is the stryke-side library — the file that defines the namespace and its wrapper functions, written in stryke. The second is a Rust `cdylib` (`crate-type = ["cdylib"]`), compiled to `libstryke_<name>.{dylib,so}`, that holds the actual client code and the `extern "C"` entry points the wrappers call.

The `stryke.toml` manifest is where the two halves are wired together. Its `[ffi]` table names the library (`lib-name`), the namespace the package exports (`namespace = "Postgres", "Kafka", "S3", ...`), and the full list of `extern "C"` symbols the cdylib exposes (`exports = [...]`). The manifest's `[scripts]` table gives the standard `build` / `test` / `install` commands, all driven through `s`, the stryke CLI.

```
[ffi]
lib-name = "stryke_postgres"
namespace = "Postgres"
exports = ["pg__query", "pg__execute", "pg__insert_many", "pg__listen", ...]

[scripts]
build = "cargo build --release"
test = "s test t/"
install = "s pkg install -g ."
```

How a call crosses the boundary

The cdylib is `dlopened` in-process the first time a script runs `use <Namespace>` — for example `use Postgres`. From then on, a call written in stryke as `Postgres::query(...)` resolves to the matching `pg__query` export in the loaded library. The convention across the fleet is a JSON boundary: the wrapper packs its arguments into a JSON dict, the export decodes it, does the work against the external system, and returns a JSON reply

the wrapper decodes back into stryke values. That keeps the FFI surface uniform — every export has the same C signature regardless of which system it talks to — and it means the type mapping (a Postgres `timestampz` to an RFC-3339 string, a Mongo document to a stryke hash) is a JSON encoding decision, made once per package.

Connections are not reopened per call. Each package caches its client — a `postgres::Client`, a `mysql::Pool`, a broker producer — in a process-lifetime `OnceCell`, keyed by the connection URL, guarded by `parking_lot`. A script that runs a hundred statements against the same URL pays for one TCP + TLS + auth handshake, not a hundred. This is the same anti-fork discipline the rest of the stack is built on: state that survives across calls, held in-process, rather than a fresh subprocess for every operation.

How a package is installed and configured

Installation goes through the stryke package manager. From a release, no Rust toolchain is required on the consumer machine:

```
s pkg install -g github.com/MenkeTechnologies/stryke-postgres
```

From a local checkout, the `cdylib` is built first and then installed into the per-user store at `~/.stryke/store/<name>@<version>/:`

```
cargo build --release
s pkg install -g .
```

Configuration lives at the call site, not in the manifest. A connector takes its connection information as a named argument — `url => $dsn` — or falls back to a package-specific environment variable (`DATABASE_URL`, `POSTGRES_DSN`, `AWS_REGION`, and so on) when no explicit argument is given. The `stryke.toml` carries the export table, not credentials.

With that shared shape established, the chapters that follow can do the one thing each connector actually differs in: which system it reaches, which crate it wraps, and which verbs it puts in a stryke script's hands.

The FFI Boundary in Depth – the JSON wire, the cache, the panic seam

The previous chapter described the package model from the outside: a stryke library, a cdylib, a manifest that wires them together. This chapter goes one layer down, into the boundary itself — the exact shape of a call as it leaves stryke, crosses into compiled Rust, does its work, and comes back. Almost every per-package chapter in this book cites the same four mechanisms — a JSON wire, a process-lifetime connection cache, a panic seam, and an error envelope — and rather than re-derive them thirty times, it is worth pinning them down once, in one place, so the connector chapters can name them and move on.

One C signature for the whole fleet

The design decision that makes the fleet uniform is that every export has the same C signature, regardless of which system it talks to. A Postgres query, a Kafka produce, an S3 put, a screenshot capture — all of them are, at the ABI level, the identical function shape:

```
#[no_mangle]
pub extern "C" fn pkg__verb(args_json: *const c_char) -> *mut c_char {
    // decode args_json -> serde_json::Value
    // do the work against the external system
    // encode the reply -> a freshly allocated C string
}
```

A `*const c_char` in, a `*mut c_char` out. This is the wire shape the packages refer to as `FfiSig::StrToStr` (the name appears in the `stryke-k8s` and `stryke-docker` chapters, where three streaming verbs are documented as *not live* precisely because a single string-in/string-out call cannot model a callback that fires many times). Everything richer than a scalar — a result set, a nested document, an options hash, an error — is carried *inside* the JSON, not in the C types. That is the trade the fleet makes on purpose: a narrow, boring ABI, with all the variety pushed up into a payload format that both sides already know how to speak.

The consequence shows up everywhere. Because the boundary is JSON, the type mapping for each system is a *serialization* decision made once per package, not a set of bespoke C structs. A Postgres `timestampz` becomes an RFC-3339 string; a Mongo `ObjectId` becomes `{"$oid": "..."};` a DuckDB `HUGEINT` becomes a decimal string to preserve precision; a ScyllaDB exotic CQL type falls back to its debug string. None of those choices touch the ABI. They are all just “how this package encodes a value into the JSON reply,” and each connector chapter has a *type encoding* section that is really documenting nothing more than its own answer to that one question.

The load: use becomes dlopen

A stryke script names a package with `use Postgres`, `use Kafka`, `use GUI`. That statement does two things. First, stryke’s package runtime finds the installed package in the per-user store, reads its `stryke.toml`, and locates the `[ffi]` table — `lib-name`, `namespace`, and the `exports` array. Second, it hands that to the resolver hook the connector chapters name repeatedly: `pkg::commands::try_load_ffi_for`. That hook `dlopens` the cdylib (`Libstryke_<name>.{dylib,so}`) *in-process*, once, and registers each symbol in the `exports` list as a callable stryke function under the declared namespace.

In-process is the load-bearing word. There is no helper binary, no subprocess, no socket to a driver daemon, no network hop. The compiled client code is mapped into the same address space as the running stryke program, and a `Namespace::verb(...)` call is a direct call into it. Several chapters ([stryke-arrow](#), [stryke-parquet](#), [stryke-aws](#), [stryke-gui](#), [stryke-kafka](#), [stryke-mysql](#)) explicitly frame this as the replacement for a *v1 helper-binary* design that did `fork(2) + exec(2)` per call. The [stryke-kafka](#) chapter is the sharpest example of why that mattered: forking a fresh producer per call defeated Kafka's own batching and compression, so the whole point of the cdylib model was to hold one producer alive across calls. The FFI boundary is not just a convenience; for several of these systems it is what makes the client *correct*, not merely fast.

The dispatch: a wrapper per verb

On the stryke side, the namespace is not magic — it is a real `.stk` file (`lib/Postgres.stk`, `lib/Kafka.stk`, `lib/GUI.stk`) full of thin wrapper functions, one per export. Every wrapper has the same three-line body, and the connector chapters describe it in almost identical words each time:

```
fn Pkg::verb ($positional, %opts) {
  val $reply = pkg__verb(to_json({ ... $positional ..., %opts }))
  Pkg::_decode($reply) # from_json, die on { error }
}
```

Build a JSON args dict from the call's positional arguments and its trailing `%opts` hash; call the sibling `pkg__verb` symbol resolved out of the cdylib; decode the JSON reply. The decode step is centralized — `Neo4j::_decode`, `Search::_decode`, `Kafka::_decode`, `Parquet::_decode` all name the same helper — so that error handling and option merging live in one place and the dozens of public wrappers stay one-liners around the raw FFI call. The [stryke-neo4j](#) chapter points at `lib/Neo4j.stk:17-36` for exactly this: a `_decode` plus a `_req` that merges connection params (including the environment-variable fallback) so the ~35 public functions above them carry no boilerplate.

This two-layer split — a dumb JSON-in/JSON-out FFI floor, and a real stryke package on top that owns error handling, option merging, and the friendly verb names — is what the [stryke-scylla](#) chapter calls “the original work here; the driver underneath is ScyllaDB’s.” It is the same everywhere. The engineering the fleet claims credit for is not the wrapped crate; it is this binding.

The cache: state that survives the call

A JSON-in/JSON-out function is stateless on its face — nothing in the signature carries a connection. The connection lives beside the export, in process-lifetime state, and this is the second mechanism every data connector shares. The pattern, stated almost verbatim across the chapters, is a `OnceCell` holding a map from a connection key to a live client, guarded by a mutex:

```
static CLIENTS: OnceCell<Mutex<HashMap<String, Client>>> = OnceCell::new();
```

What differs between packages is only the key and the client type. The [stryke-postgres](#) cache keys a `postgres::Client` by connection URL; [stryke-mysql](#) keys a `mysql::Pool`; [stryke-redis](#) keys a `redis::Connection` by `(url, db, auth)`; [stryke-duckdb](#) keys a `duckdb::Connection` by `(db, session, read_only)`; [stryke-mssql](#) keys a `tiberius::Client` by `(host, port, database, auth, encrypt)`; [stryke-k8s](#) keys a `kube::Client` by kubeconfig context; [stryke-docker](#) keys a `bollard::Docker` by `$DOCKER_HOST`. The shape is identical; the tuple is what the system needs to distinguish two distinct connections.

The payoff is the anti-fork discipline the rest of the stack is built on. A script that runs a hundred statements against one URL pays for one TCP + TLS + auth handshake, not a hundred. A `send_bulk` email campaign of a thousand recipients reuses one pooled `SmtplibTransport`. A gRPC caller reuses one HTTP/2 connection and one reflection `DescriptorPool` instead of re-reflecting per call. A Selenium script opens the browser

once and drives it across many calls because the `WebElement` handles persist in a registry keyed by integer id. The cache is not an optimization bolted on afterward — it is why the in-process model exists at all.

Two failure-handling details recur. First, a client that *errors* is evicted and reopened on the next call, so a dropped connection heals rather than poisoning the cache — the `stryke-mssql` and `stryke-neo4j` chapters both state this. Second, an embedded async runtime is part of the cached state for every package whose underlying crate is *async*: the chapters for `stryke-mssql`, `stryke-scylla`, `stryke-mongo`, `stryke-neo4j`, `stryke-grpc`, `stryke-k8s`, `stryke-docker`, `stryke-selenium`, `stryke-scrape`, `stryke-gcp`, and `stryke-azure` all describe the *cdylib* owning *one* embedded `tokio` runtime and *block_on*-ing every call, so the *stryke* caller never sees a future. One runtime, built once, is the *async* sibling of the one-client cache.

The panic seam and the error envelope

The third and fourth mechanisms handle failure. Because the boundary is a C ABI, a Rust `panic!` that unwound across it would be undefined behavior — at best a process abort, and this *cdylib* is loaded *into the host shell's address space*, so an abort takes the shell with it. Every handler therefore runs inside `catch_unwind`. The `stryke-clickhouse` chapter states the rule plainly: “handlers run inside `catch_unwind` — a panic in the Rust side becomes a *stryke*-side error, not a process crash.” The `stryke-search`, `stryke-zmq`, and `stryke-selenium` chapters say the same. The panic is caught at the seam, converted into an ordinary error result, and returned as JSON. Nothing unwinds across the boundary.

That error result is the fourth shared mechanism: the `{ "error": "<msg>" }` envelope. A handler that fails — whether from a caught panic, a driver error, or a validation failure — returns a JSON object with an `error` key, and the *stryke*-side `_decode` helper turns that shape into a *die* with a package-scoped message. The format is consistent across the fleet: `Kafka::<op>: <reason>`, `Mongo::<op>: <reason>`, `Mssql::$name: $error`, `Search::$name: $error`. The `stryke-zmq` chapter documents two refinements on top of the base envelope: a receive timeout comes back as `{"timeout": true}` and maps to `undef` (or an empty list for the drain variants) rather than an error, and the same catch-at-the-seam rule keeps a *libzmq*-level fault from aborting the host shell.

Memory: who frees the reply

One detail the ABI forces into the open: the export returns a `*mut c_char` it allocated, and *stryke*, on the other side of the boundary, must not free foreign memory with its own allocator. So the packages that spell this out (`stryke-polars`, `stryke-zmq`, `stryke-gui`, `stryke-selenium`) export a companion symbol, `stryke_free_cstring`, that frees a reply using the *cdylib*'s own allocator. *stryke*'s `rust_ffi::load_cdylib` wires it automatically: the runtime copies the returned JSON into a native *stryke* string and then calls `stryke_free_cstring` on the original pointer. The caller never sees this; it is the invisible bookkeeping that lets a JSON string cross an allocator boundary safely.

Why this uniformity is worth it

Put together, the four mechanisms — one C signature, a JSON payload, a process-lifetime cache, and a caught-panic error envelope — are what let this book be short. A connector author does not re-invent a calling convention, a threading model, or an error protocol; those are fixed by the boundary. What is left to design, per package, is the interesting part: which verbs to expose, how to encode that system's types into JSON, and what the connection key should be. The next several chapters are, in a sense, thirty answers to those three questions, all sitting on the one boundary described here. And the *Building on the fleet* part at the end of the book returns to this boundary from the author's side — how to stand up a new *cdylib* that speaks it, test it, version it, and ship it into the same store every chapter here installs into.

stryke-postgres — the PostgreSQL client

`stryke-postgres` is the opt-in PostgreSQL connector for `stryke`. It is not part of the `stryke` core binary: per the README's [0x00] rationale, "a real database client pulls in 100+ transitive crates (TLS, async runtime, type encoders)," and most `stryke` one-liners never touch Postgres. The package exists so the ones that do can opt in without taxing the daily-driver install — the same rationale the README cites for `stryke-arrow` and `stryke-mysql`.

What it wraps

The crate (`Cargo.toml`, version `0.20.0`) is a `cdylib`, not a binary. Its dependency is the `postgres` crate at `0.19`, built with `with-serde_json-1`, `with-uuid-1`, and `with-chrono-0_4` features — the synchronous Rust Postgres client, not `tokio-postgres` directly (though `postgres` is built on it). Supporting crates: `serde_json` (with `preserve_order`, since the FFI boundary is JSON), `chrono`, `uuid`, `once_cell` and `parking_lot` for the connection cache, and `bytes`.

The binding mechanism is explicit in the README: `stryke-postgres` ships as a thin `stryke` library (`lib/Postgres.stk`) plus the `cdylib` (`libstryke_postgres.{dylib,so}`). The `cdylib` is `dlopened` in-process the first time a script does `use Postgres`; each `Postgres::*` call in `stryke` invokes a `pg__*` extern "C" export with a JSON args dict and decodes a JSON reply. A `postgres::Client` is cached in a `OnceCell`, keyed by connection URL — no fork-per-call, no fresh TCP/TLS/auth handshake per query.

The exported surface

`stryke.toml`'s `[ffi]` table lists 68 `pg__*` exports under `namespace = "Postgres"`. Grouped by what the README documents:

- **Read:** `query`, `query_stream`, `query_one`, `query_col`, `query_scalar`, `dump`
- **Write:** `execute`, `exec_file`, `insert_many`, `upsert`, `update`, `delete`, `truncate`
- **Bulk transfer:** `copy_in`, `copy_out` (Postgres's `COPY ... FROM/TO STDIN`)
- **LISTEN/NOTIFY:** `notify`, `listen`
- **Transactions:** `begin`, `commit`, `rollback`, `transaction`, `savepoint`, `release_savepoint`, `rollback_to`
- **Metadata/admin:** `ping`, `tables`, `databases`, `schema`, `count`, `exists`, `table_exists`, `views`, `functions`, `indexes`, `roles`, `explain`, `db_size`, `table_size`, `activity`, `locks`, `sequences`, `extensions`, `triggers`, `cancel_backend`, `describe`, `current`, `schemas`, `matviews`, `server_settings`, `server_encoding`, `constraints`, `foreign_keys`, `primary_key`, `column_defaults`, `table_stats`, `uptime`, `replication`, `current_setting`, `set_config`, `analyze`, `vacuum`, `reindex`
- **Pure DSN/quoting helpers** (open no socket): `parse_dsn`, `build_dsn`, `parse_keyword_dsn`, `build_keyword_dsn`, `quote_ident`, `unquote_ident`, `quote_qualified_ident`, `parse_qualified_ident`, `quote_literal`, `unquote_literal`, `quote_nullable`, `escape_like`, `unescape_like`, `dollar_quote`, `unquote_dollar`, `format_array`, `parse_array`, `format_in_list`, `parse_in_list`, `parse_range`, `split_statements`

Statements against the same `%opts` ride the same cached backend connection, which is what makes `begin/commit/savepoint/rollback_to` coherent without extra FFI plumbing.

A real usage example

From `examples/crud.stk`:

```
use Postgres

val $tbl = "stryke_crud_$$"
Postgres::execute("CREATE TABLE $tbl (id INT PRIMARY KEY, name TEXT, hits INT)", url =>
  ~ $url)

val $n = Postgres::insert_many($tbl,
  [{ id => 1, name => "alice", hits => 1 },
    { id => 2, name => "bob", hits => 2 },
    { id => 3, name => "carol", hits => 3 }], url => $url)
p "insert_many -> $n rows"

p "exists id=2 -> " . Postgres::exists($tbl, "id = 2", url => $url)

Postgres::upsert($tbl, { id => 4, name => "dave", hits => 1 }, conflict => ["id"], url =>
  ~ $url)
Postgres::upsert($tbl, { id => 4, name => "dave", hits => 9 },
  conflict => ["id"], update => ["hits"], url => $url)

val $u = Postgres::update($tbl, { hits => 0 }, "name = 'alice'", url => $url)
val $d = Postgres::delete($tbl, "hits = 0", url => $url)
```

Table and SET column names go through identifier validation; `WHERE` clauses in `update/delete/count/exists` are interpolated (trusted-value territory) while bind values use positional `$1`, `$2`, ... placeholders decoded server-side.

Config surface

Connection resolution, in priority order: an explicit `url =>` named arg; individual `host/port/user/password/database` named args; or `$ENV{DATABASE_URL}` / `$ENV{POSTGRES_URL}` when neither is given. No `stryke.toml` connection config beyond the `[ffi]` export table — connection state lives at the call site, not the manifest.

Type encoding

Postgres types decode to JSON on the wire back to stryke: `bool`→`bool`, `int2/int4/int8`→`number`, `date`→`"YYYY-MM-DD"`, `timestamp/tz`→`RFC 3339`, `json/jsonb`→`preserved JSON`, everything else falls back to `text` (or `null` if no text conversion exists). `json_to_param` does the reverse mapping for binds: `null`→`NULL`, `bool`→`bool`, `int`→`i64`, `float`→`f64`, `string`→`text`, `array/object`→`jsonb`.

Install is `s pkg install -g github.com/MenkeTechnologies/stryke-postgres` from a release, or `cargo build --release && s pkg install -g .` from a checkout — the `cdylib` lands in `~/.stryke/store/postgres@<version>/.` Tests split into `cargo test` (unit/contract, no live calls) and `s test t/` gated on `$DATABASE_URL/$POSTGRES_URL/$POSTGRES_DSN` being reachable.

stryke-mysql — the MySQL/MariaDB client

`stryke-mysql` connects stryke to MySQL and MariaDB. It is not part of the stryke core binary — the README states the rationale directly: “MySQL clients pull in a big native dependency (`mysql` + TLS stack + Tokio for async variants). Most stryke one-liners don’t touch a database; for the ones that do, opt in with this package” (README.md:46-49). The same opt-in-package model applies across the connector fleet (stryke-postgres, stryke-mongo, stryke-redis) — keep the daily-driver `stryke` install slim, load database drivers only when a script actually needs one.

What it wraps

The crate is `mysql = "28"` (Cargo.toml:18), built `default-features = false` with `minimal-rust`, `buffer-pool`, and `client_ed25519` features enabled. Supporting crates: `serde/serde_json` (JSON marshaling across the FFI boundary), `chrono` for date/time encoding, `once_cell` and `parking_lot` for the connection pool cache. The package itself is versioned 0.21.0 (Cargo.toml:3, stryke.toml:3).

The stryke binding is a cdylib (`libstryke_mysql.{dylib,so}`, `crate-type = ["cdylib"]`, Cargo.toml:14) built from `src/lib.rs`, dlopened in-process the first time a script runs `use MySQL`. A `mysql::Pool` keyed by connection URL is cached in a `OnceCell` (README.md:77-79) — no fork-per-call, no fresh TCP+auth handshake per query. This replaced an earlier subprocess-based helper binary (`stryke-mysql-helper`), which the README keeps documented under a collapsed “v1 wire shape (historical)” section (README.md:334-359) but is no longer the live call path.

How stryke reaches it

Each `MySQL::*` stryke-side wrapper builds a JSON args dict and calls a sibling `mysql__*` symbol resolved out of the cdylib via stryke’s `pkg::commands::try_load_ffi_for` resolver hook (README.md:316-320). The authoritative export list lives in `[ffi].exports` of `stryke.toml`, which currently enumerates 70 symbols under `namespace = "MySQL"` — everything from `mysql__query` and `mysql__execute` to pure helpers like `mysql__quote_ident` and `mysql__parse_enum`. Errors come back as `{error}` JSON; the stryke wrapper dies with `MySQL::<op>: <reason>` (README.md:331-332).

Exported verbs

The API surface (README.md:154-312) groups into:

- **Read paths:** `query`, `query_stream` (callback per row — the cdylib still returns all rows in one FFI call), `query_one`, `query_col`, `query_scalar`, `query_multi`, `call` (stored procedures), `dump`.
- **Write paths:** `execute`, `exec_file`, `insert_many`, `upsert` (MySQL `ON DUPLICATE KEY UPDATE` — no `RETURNING`, fires on any unique/PK collision, no per-target conflict clause), `update`, `delete`, `truncate`.
- **Transactions:** `MySQL::transaction(\@statements, %opts)` — runs a batch on one checked-out connection inside a real `START TRANSACTION`, because the pool cache means back-to-back calls may

otherwise land on different pooled connections (README.md:219-225).

- **Metadata:** `tables`, `databases`, `schema`, `columns`, `indexes`, `foreign_keys`, `create_table`, `views`, `procedures`, `triggers`, `events`, `users`, `grants`, `charsets`, `collations`, `explain`, `db_size`, `table_size`, `processlist`, `status`, `variables`, `engines`, `kill`, `current_database`, `create_database`, `drop_database`.
- **Pure helpers with no connection:** DSN parsing/building, identifier and literal quoting (MySQL backtick style), `LIKE` escaping, ENUM/SET encoding (`parse_enum`, `enum_index`, `set_mask`, ...), and WHERE-clause builders — all connection-free string transforms usable without a live server.

Example (from `examples/crud.stk`)

```
use MySQL

val $url = $ENV{MYSQL_URL};
unless (defined $url) {
  p "set \${MYSQL_URL} to a mysql connection URL to exercise the demo"
  exit 0
}

val $tbl = "stryke_crud_$$"
MySQL::execute("CREATE TABLE $tbl (id INT PRIMARY KEY, name VARCHAR(32), hits INT)", url
  => $url)

val $n = MySQL::insert_many($tbl,
  [{ id => 1, name => "alice", hits => 1 },
   { id => 2, name => "bob", hits => 2 },
   { id => 3, name => "carol", hits => 3 }], url => $url)
p "insert_many -> $n rows"

MySQL::upsert($tbl, { id => 4, name => "dave", hits => 1 }, url => $url)
val $u = MySQL::update($tbl, { hits => 0 }, "name = 'alice'", url => $url)
p "update alice -> $u row(s)"
```

Config surface

Connection resolution follows a fixed priority order (README.md:116-120): a `url =>` named arg, then individual named args (`host`, `port`, `user`, `password`, `database`), then `$ENV{MYSQL_URL}`. Table and SET column names in `update/delete/upsert` are identifier-validated; `$where` clauses are interpolated, not bound — callers pass trusted values there or drop to `execute` for a fully parameterized statement (README.md:186-190, 270-272). The package also ships a standalone `mysql` CLI (query/execute/exec/dump/tables/schema/ping) accepting the same connection flags via `--dsn/--host/etc.` or their `$MYSQL_*` env equivalents (README.md:122-150).

Install is `s pkg install -g github.com/MenkeTechnologies/stryke-mysql` from a release, or `cargo build --release && s pkg install -g .` from a local checkout (README.md:57-69).

stryke-mssql — the SQL Server / Azure SQL client

`stryke-mssql` connects stryke to Microsoft SQL Server (2012+) and Azure SQL. Like `stryke-postgres` and `stryke-mysql`, it ships as a separate repository and a separate `s add` install rather than living in the stryke core binary — a language runtime has no business statically linking a TDS driver, rustls, and a tokio runtime for users who will never touch SQL Server. The core binary stays slim; you opt in per project with:

```
s add github.com/MenkeTechnologies/stryke-mssql
```

On first `use Mssql`, stryke dlopens the package's cdylib in-process and registers its exports. That in-process dlopen — not a subprocess, not a network hop to a driver daemon — is the actual mechanism, per `stryke.toml`'s `[ffi]` block (`lib-name = "stryke_mssql"`, `namespace = "Mssql"`).

What it wraps

The driver is `tiberius` 0.12, the pure-Rust TDS implementation (`Cargo.toml`), built with `default-features = false` and the `tds73`, `rustls`, `chrono`, `rust_decimal` features. No FreeTDS, no unixODBC, no system driver dependency — the whole TDS protocol stack is vendored Rust. Because `tiberius`'s client API is async, the cdylib owns one tokio runtime internally and `block_ons` every call (`tokio` with `rt-multi-thread`), presenting a synchronous facade that matches the connection-cache model the other stryke data packages (`stryke-postgres`, `stryke-mysql`) already use.

Row values are converted to JSON cell-by-cell, trying TDS types in order (int/float/bool/string/decimal/uuid/date-time/binary) — decimals stay as strings to preserve precision, binary is base64, datetimes are ISO-8601. That's what lets an arbitrary result set round-trip through the FFI boundary without a compile-time schema. A `tiberius::Client` is cached per (`host`, `port`, `database`, `auth`, `encrypt`); a connection that errors is evicted and reopened on the next call.

The stryke binding

The original engineering here isn't `tiberius` — it's the package's FFI surface and the `Mssql::*` package that sits on top of it. `src/lib.rs` (1,368 lines) exports 34 `#[no_mangle] extern "C" fn mssql__*` functions, each taking a JSON args string and returning a JSON result string. `stryke.toml`'s `[ffi].exports` lists them all: liveness (`version`, `ping`, `server_version`), query (`query`, `query_one`, `scalar`, `exists`, `simple_query`), write (`execute`, `insert`, `batch`), DDL (`create_table`, `drop_table`, `truncate`), introspection (`databases`, `tables`, `columns`, `views`, `schemas`, `primary_keys`, `foreign_keys`, `indexes`, `table_exists`, `current_database`, `count`, `row_count`), and pure SQL/URL helpers (`quote_ident`, `quote_literal`, `valid_identifier`, `escape_like`, `format_value`, `format_in_list`, `split_batch`, `parse_url`, `redact_url`, `build_url`).

`lib/Mssql.stk` is the thin stryke-side wrapper: each `Mssql::*` fn builds a JSON args hash, calls the matching `mssql__*` FFI symbol, decodes the JSON reply, and `dies` with `"Mssql:$name: $o->{error}"` on a reported error. Connection params are threaded per call as `%opts` (or fall back to `$ENV{MSSQL_URL}`, an ADO connection string), never held as global session state.

Usage

From the README's quick start — real, checked-in stryke syntax, not paraphrased:

```
use Mssql

val %conn = (
  host      => "localhost",
  database  => "app",
  username  => "sa",
  password  => $ENV{MSSQL_PASS},
  trust_cert => 1,           # for a dev cert
)

# parametrized query - @P1, @P2, ...
val @users = Mssql::query("SELECT id, name FROM users WHERE active = @P1", params => [1],
  ~ %conn)
p len(@users)

# scalar + exists
p Mssql::scalar("SELECT COUNT(*) FROM users", %conn)
p Mssql::exists("SELECT 1 FROM users WHERE id = @P1", params => [42], %conn)

# write
Mssql::execute("UPDATE users SET name = @P1 WHERE id = @P2", params => ["Ada", 42], %conn)

# transaction batch (commit on success, rollback on any failure)
Mssql::batch(
  ["INSERT INTO audit (msg) VALUES ('x')", "UPDATE users SET seen = 1 WHERE id = 42"],
  %conn,
)
```

Parametrized statements bind `@P1`, `@P2`, ... from a `params` array (null / bool / integer / float / string) — SQL Server's native positional-parameter syntax, not `$1`-style Postgres placeholders.

Config surface

`%conn` accepts `url` (a full ADO connection string, e.g. `Server=db;Database=app;UserId=sa;Password=...`), or discrete `host` (default `127.0.0.1`), `port` (default `1433`), `database`, `username`, `password`, `encrypt` (`required` / `off` / `not_supported`, default `required`), and `trust_cert` (default `false`, set `true` to accept a self-signed dev cert). With no `url` or `host` given, the wrapper falls back to `$ENV{MSSQL_URL}`.

Tests split the same way as the other data packages: `cargo test` runs in-crate unit tests (ADO string parsing/redaction, defaults, param extraction) with no database required; `t/test_mssql.stk` runs connection-free asserts unconditionally and gates the live-database asserts (create table, parametrized insert, batch transaction, `primary_keys`, `indexes`, `schemas`) behind `$MSSQL_HOST` being set, so CI stays green without a SQL Server instance and exercises the full surface against a throwaway container when one is available.

stryke-mongo — the MongoDB client

`stryke-mongo` wraps MongoDB (5.0+ standalone, replica set, or sharded cluster) as an opt-in stryke package — CRUD, aggregation, and index/collection administration reachable from a stryke one-liner via `Mongo::*`. It ships separately from the stryke core binary (`stryke.toml`, `Cargo.toml`) rather than baked in, for the same reason `stryke-postgres`, `stryke-mysql`, and `stryke-redis` are separate repos: the daily-driver stryke install doesn't need a MongoDB driver linked in unless the script actually touches Mongo. Install it with `spkg install -g github.com/MenkeTechnologies/stryke-mongo` or from a local checkout via `cargo build --release && spkg install -g ..`

What it wraps

The Rust side (`src/lib.rs`, per `Cargo.toml`) is a single-file `cdylib` around the official `mongodb` crate, version 3, with `rustls-tls`, `dns-resolver`, and `compat-3-0-0` features, plus `bson = "2"` for the wire types. Package version is 0.21.0 (`Cargo.toml`, `stryke.toml`). `tokio` (multi-thread runtime), `futures-util`, `serde/serde_json` (with `preserve_order`), `once_cell`, and `parking_lot` round out the dependency set — `once_cell` backs the shared `tokio` runtime + `mongodb::Client` cache keyed by connection URI, so repeated calls don't pay a fresh TCP+TLS+auth handshake per call.

How a stryke call reaches Mongo

`use Mongo` dlopens `libstryke_mongo.{dylib,so}` in-process on first use, via stryke's `pkg::commands::try_load_ffi_for` resolver hook (README §0x05). Each `Mongo::*` stryke-side function builds a JSON args dict and calls a sibling `mongo__*` symbol in the `cdylib` — e.g. `Mongo::find_one` → `mongo__find_one`. The authoritative export list lives in `[ffi].exports` in `stryke.toml`, currently 59 symbols spanning read/write/admin/pure-helper categories. Errors return as a `{error}` JSON payload and the stryke wrapper dies with `Mongo::<op>: <reason>`.

BSON round-trips through JSON using MongoDB's relaxed extended JSON — `ObjectId` becomes `{"_id": "..."}>`, `DateTime` becomes `{"$date": "..."}>`, `Decimal128` becomes `{"$numberDecimal": "..."}>`, and extended-JSON wrappers passed back into a filter (e.g. `{"_id": {"$oid": "65f9..."}}`) get re-parsed to real BSON types before hitting the wire (README §0x04).

Exported verbs

The `Mongo::` namespace groups into read paths (`find`, `find_one`, `find_stream`, `count`, `aggregate`), write paths (`insert_one/insert_many`, `update_one/update_many`, `replace_one`, `delete_one/delete_many`), atomic `find_one_and_{update,replace,delete}`, admin (`create_index`, `drop_index`, `indexes`, `create_collection`, `drop_collection`, `drop_database`, `rename_collection`, `run_command`, `coll_stats`, `db_stats`, `explain`, `server_status`, `ping`), and a large tier of connection-free pure helpers with no client involved: connection-string parse/build/validate/redact, namespace/collection/field-name validators, `ObjectId` construction and decomposition (`new_objectid`, `parse_objectid`, `objectid_timestamp`, `objectid_range`), and query builders (`merge_filters`, `build_update`, `build_sort`, `build_projection`, `in_filter`, `between_filter`, `or_filter`, `text_filter`, etc.) that assemble filter/update/sort documents without touching the wire. Every write/read fn accepts a URI override in `%opts`

(`Mongo::find "logs/errors", filter => {...}, uri => "mongodb+srv://..."`), and target collections are addressed as `"db/collection"` or `"db.collection"` strings.

A real example (`examples/crud.stk`)

```
use Mongo

val $alive = eval { Mongo::ping() }
unless ($alive) {
  p "mongod not reachable - set MONGODB_URI or start mongod to exercise the demo"
  exit 0
}

val $t = "demo.users"

val $r = Mongo::insert_one $t, { name => "alice", age => 30, tags => ["admin","analyst"] }
p "inserted: " . to_json($r->{inserted_id})

Mongo::insert_many $t, [
  { name => "bob",    age => 25 },
  { name => "charlie", age => 35 },
  { name => "dana",   age => 28 },
]

p "count: " . Mongo::count($t)

var @over28 = Mongo::find($t, filter => { age => { '$gt' => 28 } }, sort => { age => -1 })
p "over-28:"
for val $u (@over28) {
  p "  $u->{name} (age $u->{age})"
}

Mongo::update_one $t, { name => "alice" }, { '$inc' => { age => 1 } }

p "after birthday:"
p to_json Mongo::find_one($t, filter => { name => "alice" })

Mongo::delete_many $t, {}
p "cleaned up"
```

Run with `MONGODB_URI=mongodb://localhost s examples/crud.stk`. Four other examples ship alongside it — `aggregate.stk`, `index_admin.stk`, `counts.stk`, `discover.stk` — each exercising one verb group.

Config surface

The only required connection input is a Mongo URI: `$ENV{MONGODB_URI}`, the CLI's `-u/--uri` flag on the standalone `mongo` binary (`mongo find app/users --filter=... -u mongodb://localhost:27017`), or a per-call `uri => "..."` in `%opts`, which takes precedence and is cached separately in the client pool. Both `mongodb://` (host list, replica-set aware) and `mongodb+srv://` (single host, SRV-resolved by the driver, structure-parsed only by `parse_connection_string` — it never resolves DNS itself) are accepted. There is no `stryke.toml`-level connection config; auth/TLS/replica-set topology all live in the URI, matching the underlying `mongodb` crate's own configuration model.

What's not there yet

Per README §0x09: change streams (needs a replica set), multi-document transactions (needs a replica set), GridFS, `bulk_write` (mixed ordered/unordered), canonical extended JSON for `$numberLong` precision, and a persistent connection-pool/serve daemon are open roadmap items, not implemented.

stryke-redis — the Redis / Valkey client

`stryke-redis` (v0.19.0) is stryke's binding to Redis and Redis-compatible stores (Valkey, KeyDB, Dragonfly): key-value, lists, sets, hashes, sorted sets, streams, geo, HyperLogLog, Lua scripting, pub/sub publish, SCAN, and server admin. It ships as an opt-in package, not part of the core `stryke` binary — a daily install doesn't need to link a Redis client, so it stays out of the base binary and is pulled in only by `use Redis`.

What it wraps

`Cargo.toml` pins the Rust `redis` crate at `1.x`, with `default-features = false` and an explicit feature list: `acl`, `streams`, `geospatial`, `script`, `num-bigint`, `tls-rustls` (`Cargo.toml:14-16`). The crate compiles to a `cdylib` (`name = "stryke_redis"`) rather than a normal `rlib` — it's dlopened in-process by stryke's FFI resolver, not linked at compile time.

The binding shape

`stryke.toml`'s `[ffi]` table is the contract: `lib-name = "stryke_redis"`, `namespace = "Redis"`, and an `exports` array listing every `redis__*` symbol the `cdylib` exposes — well over 150 entries covering all the Redis command families plus a set of pure helper functions with no connection (`redis__parse_info`, `redis__glob_match`, `redis__cluster_keyslot`, `redis__parse_stream_id`, and similar). `lib/Redis.stk` is the thin `.stk` wrapper that gives each exported symbol its `Redis::foo(...)` name inside stryke.

Per the README's FFI section (`README.md`, "[0x05] FFI layer"): each `Redis::*` call builds a JSON args dict and invokes the matching `redis__*` symbol resolved out of `libstryke_redis.{dylib,so}`. The library is loaded on first `use Redis` via stryke's `pkg::commands::try_load_ffi_for` hook, and it caches one `redis::Connection` per `(url, db, auth)` tuple in an `OnceCell<Mutex<HashMap>>` for the process lifetime — repeated calls against the same connection options reuse the existing TCP socket instead of re-handshaking per call.

Return values cross the FFI boundary as one of a fixed set of JSON shapes: scalars as `{"value": ...}`, lists/sets as `{"values": [...]}` or `{"members": [...]}`, hashes as `{"hash": {...}}`, sorted-set rows as `{"pairs": [[member, score], ...]}`, and errors as `{"error": "<msg>"}` (the `.stk` wrapper `dies` on that shape).

Usage — from `examples/kv.stk`

```
#!/usr/bin/env stryke
# examples/kv.stk — simple KV demo.
# Run:  REDIS_URL=redis://localhost s examples/kv.stk

use Redis

val $alive = eval { Redis::ping() }
unless ($alive) {
  p "redis not reachable — set REDIS_URL or start redis to exercise the demo"
  exit 0
}

Redis::set("greeting", "hello from stryke")
p "stored: " . Redis::get("greeting")

Redis::mset({ a => 1, b => 2, c => 3 })
val @v = Redis::mget(["a", "b", "c"])
p "mget: " . join(", ", @v)

p "incr: " . Redis::incr("counter")
p "incr: " . Redis::incr("counter", by => 10)
p "ttl(counter): " . Redis::ttl("counter") . " (no expiry → -1)"

Redis::del(["greeting", "a", "b", "c", "counter"])
```

This is the actual contents of `examples/kv.stk` in the repo, unmodified. `examples/pipeline.stk` shows the same `Redis::ping()`-guarded pattern extended to bulk `mset/mget` and a TTL'd counter (rate-limit idiom); `examples/structures.stk`, `examples/publish.stk`, and `examples/discover.stk` cover the remaining hash/set/zset ops, pub/sub publish, and the `parse_*/glob_*` pure helpers respectively.

Config surface

Every `Redis::*` call takes an optional trailing `%opts` hash. Connection is resolved from, in order: an explicit `url` (or `host/port/db/username/password/tls` parts) in that `%opts` hash, falling back to `$ENV{REDIS_URL}`, falling back to `127.0.0.1:6379`. `tls => 1` forces a `redis://` → `rediss://` rewrite without editing the URL string. There is no separate `stryke.toml` connection block — connection state is passed per-call or via the environment variable, matching the “v1 helper-binary flags” the README documents.

What's not here yet

Per the README's roadmap table: `SUBSCRIBE` (streaming) is deferred to v2, pending a stryke Unix-socket builtin; pipelining is per-call round trips today, not batched `MULTI/EXEC` transactions (`Redis::pipeline` exists in the export list but the roadmap still lists “Pipelined ops + transactions” as v2+); and it's on the `redis` crate's 1.x sync API, not RESP3 or the Redis 8 vector/Bloom extensions.

stryke-neo4j – the Neo4j graph client

`stryke-neo4j` is an opt-in stryke package: a Cypher query/run client for Neo4j 4.x/5.x over the Bolt protocol, plus schema introspection (labels, relationship types, property keys, indexes, constraints). It is not part of the stryke core binary — graph-database access is a niche need, and shipping a Bolt driver in every stryke install would bloat the daily-driver binary for the vast majority of users who never touch Neo4j. Like the other stryke data connectors, it ships as a separate cdylib pulled in explicitly with `s add github.com/MenkeTechnologies/stryke-neo4j` (`README.md`).

What it wraps

The crate is a thin Rust cdylib (`Cargo.toml`):

```
[lib]
name = "stryke_neo4j"
crate-type = ["cdylib"]

[dependencies]
neo4rs = "0.8"
tokio = { version = "1", features = ["rt", "rt-multi-thread", "net", "time", "sync"] }
serde = { version = "1", features = ["derive"] }
serde_json = { version = "1", features = ["preserve_order"] }
anyhow = "1"
once_cell = "1"
```

`neo4rs` 0.8 is the pure-Rust Bolt driver — no JVM, no official-driver FFI dependency. Because `neo4rs` is async, the cdylib owns a single embedded tokio runtime and `block_ons` every call, presenting the same blocking facade the other stryke data packages use so the stryke caller never deals with futures. Bolt rows deserialize straight into JSON, so a returned node or relationship becomes its property map with no per-query schema declaration required on the stryke side. Connections are cached as a `neo4rs::Graph` pool keyed by `(uri, user, database)`; a pool that errors is evicted and reopened on the next call (`README.md`, “Architecture”).

Package version: `0.5.0` (`Cargo.toml`, `stryke.toml`).

The binding, not the driver

The original engineering here is the stryke binding, not the Bolt protocol implementation. `lib/Neo4j.stk` is a pure-stryke wrapper layer over the cdylib’s `neo4j__*` FFI exports (`stryke.toml`’s `[ffi] exports` lists 40 symbols). Every wrapper function follows the same shape: build a JSON args hash from `%opts`, call the matching `neo4j__*` symbol, decode the JSON return, `die` on an `{error}` key. `Neo4j::_decode` and `Neo4j::_req` (`lib/Neo4j.stk:17-36`) centralize that dispatch and connection-param merging (including a `$NEO4J_URL` env fallback) so the ~35 public functions in the file stay one-liners around the FFI call.

The exported verb groups, as registered under the `Neo4j::` namespace (`README.md`, “API reference”; cross-checked against `lib/Neo4j.stk`):

Group	Functions
Liveness	<code>version</code> , <code>ping</code> , <code>server_info</code>
Query	<code>query</code> , <code>query_one</code> , <code>scalar</code> , <code>query_values</code> , <code>query_column</code>
Write	<code>run</code> , <code>batch</code>
Graph helpers	<code>create_node</code> , <code>merge_node</code> , <code>set_props</code> , <code>get_node</code> , <code>exists_node</code> , <code>create_rel</code> , <code>merge_rel</code> , <code>delete_rel</code> , <code>delete_node</code> , <code>delete_nodes</code> , <code>degree</code> , <code>node_count</code> , <code>relationship_count</code> , <code>count_property_values</code>
Schema	<code>create_index</code> , <code>create_constraint</code> , <code>drop_index</code> , <code>drop_constraint</code>
Introspection	<code>labels</code> , <code>relationship_types</code> , <code>property_keys</code> , <code>indexes</code> , <code>constraints</code> , <code>databases</code>
Planning	<code>explain</code> , <code>profile</code>
Cypher/URL helpers	<code>escape</code> , <code>quote_literal</code> , <code>quote_ident</code> , <code>valid_identifier</code> , <code>format_value</code> , <code>parse_url</code> , <code>redact_url</code> , <code>build_url</code>

Cypher is always parametrized — `$name` placeholders bound from a `params` hashref — so values never enter the query text ([README.md](#), “Architecture”). The graph/schema convenience helpers (`create_node`, `merge_rel`, etc.) take scalar properties and backtick-quote labels, relationship types, and property names server-side, since Cypher can’t parametrize identifiers; anything with list/map properties goes through `run` with hand-written Cypher instead.

Usage

From the test suite ([t/test_neo4j.stk:8-42](#)), which is the real exercised call path — connection-free asserts always run, the live-graph asserts short-circuit unless `$NEO4J_URL` is set:

```

use Neo4j

var %conn
$conn{uri}      = $ENV{NEO4J_URL}
$conn{user}     = $ENV{NEO4J_USER} // "neo4j"
$conn{password} = $ENV{NEO4J_PASS} if defined $ENV{NEO4J_PASS}

assert_eq Neo4j::scalar("RETURN 1 + 1 AS n", %conn), 2, "scalar arithmetic"

Neo4j::run(
  "CREATE (p:Person {id: \${id}, name: \${name}})",
  params => { id => 1, name => "Ada" },
  %conn,
)

val @rows = Neo4j::query(
  "MATCH (p:Person) WHERE p.id = \${id} RETURN p.name AS name",
  params => { id => 1 },
  %conn,
)
assert_eq $rows[0]{name}, "Ada", "node property round-trips"

```

Point-lookup helpers avoid hand-written Cypher for the common case: `Neo4j::get_node($tag, "id", 1, %conn)`, `Neo4j::exists_node($tag, "id", 1, %conn)`, and relationship creation via `Neo4j::create_rel(from_label => ..., to_label => ..., type => "KNOWS", %conn)` ([t/test_neo4j.stk:49-63](#)).

Config surface

Connection is a `%conn` hash passed to every call, or `$NEO4J_URL` as a fallback Bolt URI ([lib/Neo4j.stk:32-34](#)):

Key	Default	Notes
<code>uri</code>	<code>127.0.0.1:7687</code>	Bolt URI: <code>neo4j://</code> , <code>neo4j+s://</code> , <code>bolt://</code>
<code>user</code>	<code>neo4j</code>	
<code>password</code>	—	
<code>database</code>	server default	multi-database servers

There is no separate `stryke.toml` config block for connections — `[ffi]` only declares the exported symbol list and `[scripts]` the build/test/install targets (`stryke.toml`). Live tests need `$NEO4J_URL` (+ `$NEO4J_USER/$NEO4J_PASS`) pointed at a real or throwaway Neo4j instance; `cargo test` alone runs only the connection-free unit tests (URI parse/redact, defaults, param shaping) ([README.md](#), “Build & test”).

stryke-scylla — the ScyllaDB / Cassandra client

`stryke-scylla` is the `Scylla` package: CQL query, DDL, and schema introspection against ScyllaDB or Apache Cassandra clusters over the native CQL binary protocol (source: [README.md](#), [stryke.toml](#)). It ships as its own repository — `s add github.com/MenkeTechnologies/stryke-scylla` — and is not part of the stryke core binary. That split is the point of the “opt-in package tier”: a wide-column driver, its transitive dependency tree, and an embedded async runtime have no business bloating the install of anyone who never touches ScyllaDB. Pull it in on demand; leave it out otherwise.

What it wraps

The crate is a `cdylib` (`Cargo.toml`: `crate-type = ["cdylib"]`, `name = "stryke_scylla"`, version `0.3.0`) built on ScyllaDB’s official pure-Rust CQL driver:

```
scylla = "1"
tokio = { version = "1", features = ["rt-multi-thread"] }
```

The `scylla` crate also speaks Apache Cassandra, so this one binding covers both targets. Because that driver is async and stryke call sites are synchronous, the `cdylib` owns a single embedded tokio runtime and `block_ons` each FFI call — [README’s \[0x03\] Architecture](#) section is explicit about this: “the `cdylib` owns ONE embedded tokio runtime and `block_ons` each call, so the stryke-facing API stays synchronous.” A `Session` is cached per `(nodes, auth, keyspace)` for the life of the stryke process; connection pooling within that is the driver’s own responsibility.

Row decoding takes each `CqlValue` to its natural JSON type (ints, text, bool, lists/sets/maps) with a debug-string fallback for exotic CQL types, and rows come back keyed by column name — this is the shape every `query/query_row/query_value` call returns.

The binding, not just the driver

The Rust side (`src/lib.rs`, 934 lines) exports 33 `scylla__*` FFI symbols, enumerated verbatim in `stryke.toml`’s `[ffi] exports` array — `scylla__query`, `scylla__execute`, `scylla__create_keyspace`, `scylla__escape_string`, and so on. Those are not what stryke code calls directly. `lib/Scylla.stk` is the actual stryke-facing package: a thin wrapper that builds a JSON args dict per call, invokes the raw `scylla__*` export, and decodes the JSON return, dying on `{error: ...}`. Connection parameters (`nodes`, `node`, `host`, `port`, `username`, `password`, `keyspace`) are threaded through every wrapper call via a shared `Scylla:::req` helper, with a fallback to `$ENV{SCYLLA_NODES}` when no node is supplied in the call’s `%opts` hash. This two-layer design — a dumb, JSON-in/JSON-out FFI boundary plus a real stryke package on top that owns error handling and option merging — is the original work here; the driver underneath is ScyllaDB’s.

Exported verbs

Per [README.md](#)'s API reference table and `t/test_stryke_scylla_surface.stk` (which asserts every one of these resolves at runtime):

Group	Functions
Liveness	<code>version</code> , <code>ping</code> , <code>server_version</code> , <code>cluster_name</code> , <code>peers</code>
Query	<code>query</code> , <code>query_row</code> , <code>query_value</code> , <code>execute</code> , <code>batch</code> , <code>raw</code>
Introspection	<code>keyspaces</code> , <code>tables</code> , <code>columns</code> , <code>count</code> , <code>indexes</code> , <code>views</code> , <code>types</code> , <code>partition_keys</code> , <code>clustering_keys</code>
DDL	<code>create_keyspace</code> , <code>drop_keyspace</code> , <code>create_table</code> , <code>drop_table</code> , <code>create_index</code> , <code>drop_index</code> , <code>truncate</code>
Pure helpers	<code>escape</code> , <code>quote_literal</code> , <code>quote_ident</code> , <code>valid_identifier</code> , <code>format_value</code> , <code>format_in_list</code> , <code>contact_points</code>

The pure helpers (`escape`, `quote_ident`, `contact_points`, ...) do no I/O and are unit-tested in-crate, so CI validates CQL-escaping and identifier-quoting logic without a live cluster.

Usage

There is no `examples/` directory in this repo; the canonical usage is the README quick-start, which the test suite (`t/test_scylla.stk`) exercises end to end against a live cluster:

```
use Scylla

var %conn
$conn{node}      = "127.0.0.1:9042"
$conn{keyspace} = "app"

Scylla::create_keyspace("app", %conn)
Scylla::create_table(
  "app.users",
  columns      => "id uuid, name text, age int",
  primary_key => "id",
  %conn,
)

Scylla::execute("INSERT INTO app.users (id, name, age) VALUES (uuid(), 'ada', 36)", %conn)

p Scylla::count("app.users", %conn)
val @rows = Scylla::query("SELECT name, age FROM app.users", %conn)
```

Queries are unparameterized CQL — there is no prepared-statement binding in this API surface — so untrusted values go through `Scylla::escape` before interpolation:

```
val $name = Scylla::escape($user_input)
val @hits = Scylla::query("SELECT * FROM app.users WHERE name = '$name' ALLOW FILTERING",
    ~ %conn)
```

Configuration surface

There is no `stryke.toml` connection block; every connection parameter is passed per call through the `%opts` hash, or falls back to environment. From the README's connecting table:

Key	Default	Notes
<code>node</code>	<code>127.0.0.1:9042</code>	one contact point, <code>host</code> or <code>host:port</code>
<code>host</code>	—	alias for <code>node</code> ; combine with <code>port</code>
<code>port</code>	<code>9042</code>	default CQL port
<code>nodes</code>	—	array of contact points, overrides <code>node/host</code>
<code>username / password</code>	—	CQL <code>PasswordAuthenticator</code> credentials
<code>keyspace</code>	—	<code>USE</code> d on the session after connect

With no `node/host/nodes` given, the wrapper reads `$SCYLLA_NODES` as a comma-separated list (`lib/Scylla.stk`, `Scylla::_req`).

Build & test

`make debug` (`cargo build`), `make test` (`cargo test` for the pure-Rust unit tests, then `s test t/` for the stryke-level suite — `t/test_stryke_scylla_surface.stk` pins the wrapper surface with no cluster required; `t/test_scylla.stk` runs a full keyspace/table/insert/query/index lifecycle against `$SCYLLA_NODES` and short-circuits cleanly when no cluster answers), `make install` (`s pkg install -g .`).

stryke-clickhouse — the ClickHouse client

`stryke-clickhouse` is the ClickHouse binding for stryke: SELECTs, bulk inserts, DDL, and schema introspection against a ClickHouse server over its HTTP interface (default port 8123). Version `0.4.0` per `Cargo.toml` and `stryke.toml`. Like `stryke-postgres`, `stryke-duckdb`, and `stryke-search`, it ships as a separate opt-in package rather than living in the core binary — the core stryke install stays free of any particular database's transport dependencies, and you pull in ClickHouse only if you're actually running an OLAP workload against it.

Transport

The crate wraps `ureq "2"` — a synchronous, pure-Rust, rustls-backed HTTP client. There's no tokio and no OpenSSL in the dependency tree; `Cargo.toml` calls this out directly: "Same durable transport model as stryke-search." The other dependencies are unsurprising for a JSON-over-HTTP FFI shim: `serde/serde_json` (with `preserve_order`), `base64`, `anyhow`, `once_cell`, `parking_lot`.

This is a deliberate choice against wrapping `clickhouse-rs` or another native-protocol driver — ClickHouse's HTTP interface is well documented, supports `FORMAT JSON` for reads and `FORMAT JSONEachRow` for writes, and lets the binding stay dependency-light. A `ureq::Agent` (an HTTP keep-alive connection pool) is cached per `(base_url, auth, database)` for the life of the stryke process, so repeated calls against the same connection reuse sockets.

How it's built

Like the rest of the fleet, this is a `cdylib` (`src/lib.rs`, 1033 lines) exporting `clickhouse__*` C-ABI symbols declared in `stryke.toml`'s `[ffi]` block under `namespace = "Clickhouse"`. On `use Clickhouse`, stryke dlopens the shared library in-process and registers every export. Each `clickhouse__*` function takes a JSON args dict and returns JSON, and handlers run inside `catch_unwind` — a panic in the Rust side becomes a stryke-side error, not a process crash.

The actual stryke-facing API is the pure-stryke wrapper at `lib/Clickhouse.stk`, which builds the request hash, calls the raw `clickhouse__*` export via `to_json/from_json`, and decodes the result — including turning `{ error: ... }` responses into a `die`.

Exported verbs

From `stryke.toml`'s export list and `lib/Clickhouse.stk`, grouped as the README documents them:

Group	Functions
Liveness	<code>version</code> , <code>ping</code> , <code>server_version</code>

Group	Functions
Query	<code>query</code> , <code>query_rows</code> , <code>query_row</code> , <code>query_value</code> , <code>exec</code> , <code>insert</code> , <code>delete_where</code> , <code>raw</code>
Introspection	<code>databases</code> , <code>tables</code> , <code>describe</code> , <code>columns</code> , <code>count</code> , <code>table_exists</code> , <code>database_exists</code> , <code>settings</code> , <code>parts</code> , <code>disk_usage</code> , <code>mutations</code> , <code>processes</code>
DDL	<code>create_database</code> , <code>drop_database</code> , <code>create_table</code> , <code>drop_table</code> , <code>truncate_table</code> , <code>rename_table</code> , <code>add_column</code> , <code>drop_column</code> , <code>optimize</code>
Pure helpers	<code>escape</code> , <code>escape_like</code> , <code>quote_literal</code> , <code>quote_ident</code> , <code>valid_identifier</code> , <code>format_value</code> , <code>format_in_list</code> , <code>format_array</code> , <code>build_url</code> , <code>parse_url</code> , <code>redact_url</code>

`query` returns the full `{ meta, data, rows, statistics }` response ClickHouse's JSON format sends back; `query_rows`, `query_row`, and `query_value` peel off the array, first row, or first scalar respectively. `exec` is for statements with no row result. The “pure helpers” group — `escape`, `quote_ident`, `parse_url`, etc. — takes no connection at all and is unit-tested directly in the Rust crate, so it validates in CI without a live server.

Usage (from the README quick start)

```
use Clickhouse

var %conn
$conn{url} = "http://127.0.0.1:8123"

Clickhouse::create_table(
    "events",
    columns => "ts DateTime, user UInt64, kind String",
    order_by => "(ts, user)",
    %conn,
)

Clickhouse::insert("events", [
    { ts => "2026-06-20 12:00:00", user => 1, kind => "click" },
    { ts => "2026-06-20 12:00:01", user => 2, kind => "view" },
], %conn)

p Clickhouse::count("events", %conn) # 2
p Clickhouse::query_value("SELECT uniqExact(user) FROM events", %conn)
val @rows = Clickhouse::query_rows("SELECT kind, count() c FROM events GROUP BY kind",
    %conn)
```

`insert` sends the array of row hashes as `FORMAT JSONEachRow` and returns the row count actually sent. `create_table` accepts either a full `sql => CREATE` statement or a `$name` plus `columns =>`, `engine =>` (default `MergeTree`), and `order_by =>`.

Parameterizing a query safely goes through the pure `escape` helper rather than string interpolation of untrusted input:

```
val $name = Clickhouse::escape($user_input)
val @hits = Clickhouse::query_rows("SELECT * FROM events WHERE kind = '$name'", %conn)
```

Connection config

Every call takes connection params through the `%opts` hash (`%conn` in the examples above), read by the `_req` helper in `lib/Clickhouse.stk`:

Key	Default	Notes
<code>url</code>	—	Full base URL, e.g. <code>https://ch.example.com:8443</code>
<code>host</code>	<code>127.0.0.1</code>	Used with <code>port/tls</code> when <code>url</code> is absent
<code>port</code>	<code>8123</code>	HTTP interface port
<code>tls</code>	<code>false</code>	<code>true</code> selects the <code>https</code> scheme
<code>username</code>	<code>default</code>	HTTP Basic auth user
<code>password</code>	(empty)	HTTP Basic auth password
<code>database</code>	<code>default</code>	Sent as <code>?database=</code> on every request
<code>params</code>	—	Extra ClickHouse settings as URL query params

When neither `url` nor `host` is given, `_req` falls back to the `$CLICKHOUSE_URL` environment variable. There is no separate `stryke.toml` connection-string surface beyond the `[ffi]` export list — connection state is entirely per-call, keyed for pooling by `(base_url, auth, database)`.

Testing

`make test` runs `cargo test` (string/identifier escaping, URL parse/redact, connection-key, result-extraction — no server needed) followed by `s test t/. t/test_stryke_clickhouse_surface.stk` pins the wrapper surface; `t/test_clickhouse.stk` runs a full DDL-insert-query-mutate-drop cycle against a live server when `$CLICKHOUSE_URL` (or `127.0.0.1:8123`) answers, and short-circuits cleanly otherwise so CI without a ClickHouse instance still passes on the connection-free assertions.

stryke-search — the Elasticsearch / OpenSearch client

`stryke-search` (crate `stryke-search`, package `Search`, v0.3.0) is stryke's binding to the Elasticsearch/OpenSearch REST API — index administration, document CRUD, bulk indexing, the query DSL, scroll, aliases, snapshots, and cluster health, against any Elasticsearch 7+/8+ or OpenSearch 1+/2+ cluster ([README.md](#)). Both engines expose the same `_search/_bulk/_doc/_cat/_cluster` endpoints, so one client covers both ([README.md](#), “Architecture”).

It ships as an opt-in package, not part of the stryke core binary. `s add github.com/MenkeTechnologies/stryke-search` records the dependency in `stryke.toml` and pulls a prebuilt cdylib (SHA-256 verified) for the host triple from the GitHub release; nothing search-related is compiled into or loaded by a stryke process that never calls `use Search`. That is the point of the package tier: full-text search pulls in an HTTP client and a large query-DSL surface that most scripts never touch, so it stays out of the default install.

What it wraps

`Cargo.toml` builds a cdylib (`stryke_search`) against:

```
ureq = "2"
base64 = "0.22"
serde = { version = "1", features = ["derive"] }
serde_json = { version = "1", features = ["preserve_order"] }
anyhow = "1"
once_cell = "1"
parking_lot = "0.12"
```

`ureq` is the transport — synchronous, pure-Rust, rustls-backed, no tokio, no OpenSSL (`Cargo.toml` comment; [README.md](#) “Architecture”). A `ureq::Agent` (HTTP keep-alive pool) is cached per `(base_url, auth)` for the life of the stryke process, matching the sync connection-cache model `stryke-redis` uses. Every `search__*` export takes a JSON args dict and returns JSON; handlers run inside `catch_unwind` so a panic becomes an error result rather than an unwind across the FFI boundary.

Exported surface

`stryke.toml`'s `[ffi] exports` list names 137 `search__*` symbols, loaded under the `Search` namespace. `lib/Search.stk` is the stryke-side wrapper (137 `fn Search::*` definitions, `lib/Search.stk`) — it JSON-encodes each call's arguments, invokes the matching FFI export, and decodes the JSON result, dying with `"Search::$name: $error"` on a cluster-reported error (`lib/Search.stk::Search::_decode`).

The exports group into: cluster/nodes (`version`, `ping`, `health`, `cat`, `raw`, `cluster_stats`, `nodes_info`, ...), index admin (`index_create`, `index_delete`, `index_refresh`, `mapping_put`, `settings_update`, `forcemerge`, ...), aliases and templates, documents (`doc_index`, `doc_get`, `doc_update`, `mget`, `bulk`, `termvectors`, ...), search (`search`, `count`, `msearch`, `scroll_start/scroll_next/scroll_clear`, `delete_by_query`, `reindex`, `pit_open/pit_close`, ...), a full query-DSL builder set (`match`, `term`, `range`,

`bool`, `nested`, `function_score`, `geo_distance`, ...), aggregations (`agg_terms`, `agg_date_histogram`, `agg_percentiles`, ...), ingest pipelines, snapshot/repo management, tasks, stored scripts, and URL helpers (`build_url`, `parse_url`, `redact_url`, `valid_index_name`) (`README.md` "API reference"). Query builders return bare clauses (`{"match": {...}}`) that nest directly inside `bool/nested`; `Search::search` and `Search::count` auto-wrap a top-level clause as `{"query": clause}`.

Real usage

From `t/test_search.stk`, the package's own end-to-end assertion test:

```
use Search

var %conn
$conn{url} = $ENV{SEARCH_URL} if defined $ENV{SEARCH_URL}
val $alive = eval { Search::ping(%conn) }

if ($alive) {
    val $idx = "stryke-test-$$"

    Search::index_create(
        $idx,
        body => { mappings => { properties => { title => { type => "text" }, year => {
~ type => "integer" } } } },
        %conn,
    )
    assert_ok Search::index_exists($idx, %conn), "index created"

    Search::doc_index($idx, { title => "rust in action", year => 2019 }, id => "1", %conn)
    Search::index_refresh(index => $idx, %conn)
    assert_ok Search::doc_exists($idx, "1", %conn), "doc indexed"

    val $res = Search::search($idx, Search::match("title", "rust"), %conn)
    assert_eq $res->{hits}{total}{value}, 1, "match query finds the doc"
}
```

The test short-circuits on the connection-free asserts (DSL builders, `Search::version`) when no cluster answers `Search::ping`, so CI passes without a live Elasticsearch/OpenSearch instance; the CRUD/search asserts only run when `$alive` is true.

Configuration surface

There is no `[search]` table in `stryke.toml` — connection parameters are passed per-call through the `%conn` opts hash that every `Search::*` function accepts: `url`, `host/port`, `tls`, `username/password` (HTTP Basic), `api_key` (sent as `Authorization: ApiKey <key>`, takes precedence over Basic), and `params` (extra URL query params) (`README.md` "Connecting"). When neither `url` nor `host` is set, `Search::_req` falls back to the `$SEARCH_URL` environment variable (`lib/Search.stk::Search::_req`). `stryke.toml`'s `[scripts]` section defines `build`, `test` (`s test t/`), and `install` (`s pkg install -g .`, which drops the `cdylib` at `~/.stryke/store/search@<ver>/`).

stryke-arrow — the Apache Arrow / Parquet columnar-data package

`stryke-arrow` (v0.19.0) wires stryke to Apache Arrow: Parquet, Arrow IPC, Feather, arrow-CSV, and arrow-JSON, via the `arrow/arrow-csv/arrow-ipc/arrow-json/arrow-schema/parquet` crate family (all pinned to "58" in `Cargo.toml`). It is not part of the stryke core binary. Per the README's own framing: "stryke's core stays small on purpose — most one-liner / awk-replacement work doesn't need 200 transitive crates of columnar data infrastructure." Arrow and Parquet sit in a separate opt-in tier alongside "big-ML, cloud SDKs, niche formats" — heavy dependency, narrow use case — kept out of the ~40 MB daily-driver install.

Shape of the package

This is a cdylib, not a linked Rust dependency. `Cargo.toml` declares `crate-type = ["cdylib"]`, building `libstryke_arrow.{dylib,so}`. Core stryke is never linked against arrow-rs. The stryke-facing side is a set of thin `.stk` wrapper files under `lib/` (`Arrow.stk`, `Parquet.stk`, `IPC.stk`, `Feather.stk`, `CSV.stk`, `JSON.stk`, `DataFrame.stk`) that build a JSON args dict and call a sibling `arrow__*` symbol resolved out of the dylib. On `use Arrow`, stryke's FFI bridge (`pkg::commands::try_load_ffi_for`) reads the package's `stryke.toml`, finds the `[ffi]` table, and dlopens the cdylib once for the process lifetime — no helper binary, no fork per call. The README notes this replaces a "v1 helper-binary fork-per-call model."

`stryke.toml`'s `[ffi].exports` is the authoritative export list — 48 symbols, from `arrow__version` and `arrow__read` through the full compute/aggregation set (`arrow__filter`, `arrow__sort`, `arrow__corr`, `arrow__quantile`, `arrow__fold_case`, etc.).

What it does

Every op reads or writes one of five formats, detected from file extension: Parquet, Arrow IPC (`.arrow`), Feather (an IPC v2 alias), CSV, and NDJSON. The API groups into:

- **Read:** `Arrow::read`, `read_stream` (streamed, no buffering), `read_columnar`, `schema` (footer-only, no scan), `stats`, `row_count`, `column_names`, `column_count`, `is_empty`.
- **Write:** `Arrow::write`, `write_iter` (streaming iterator form).
- **Conversion:** `Arrow::convert(SRC, DST, %opts)` — server-side reader-to-writer pipeline that never round-trips data through the stryke process.
- **Compute (file → file):** `filter`, `filter_in`, `filter_not_in`, `filter_str`, `select`, `drop`, `distinct`, `drop_nulls`, `keep_nulls`, `fill_null`, `sort`, `reverse`, `gather`, `top_k`, `value_counts`, `head`, `tail`, `slice`, `with_row_index`, `concat`, `hstack`, `rename`, `cast`, `count`, `null_counts`, `shape` — Arrow's compute kernels applied source-to-destination without materializing the whole file in stryke.
- **Aggregation / numeric transforms:** `sum`, `mean`, `min_max`, `std`, `median`, `quantile`, `corr`, `describe`, `aggregate`, `unique`, `clip`, `scale`, `add_const`, `abs`, `round`, `add_column`, `sample`, `fold_case`, `metadata`.

Per-format sublibraries (`Arrow::Parquet`, `Arrow::IPC`, `Arrow::Feather`, `Arrow::CSV`, `Arrow::JSON`) pin the `format` option automatically. `Arrow::DataFrame::load` returns a stryke DataFrame when the builtin is available, otherwise a `{ col => [vals] }` columnar hash.

Real usage

`examples/csv_to_parquet.stk`, verbatim:

```
#!/usr/bin/env stryke
# examples/csv_to_parquet.stk - convert a CSV to compressed parquet.
#
# Run:    s examples/csv_to_parquet.stk in.csv out.parquet [snappy|zstd|gzip]

use Arrow

var ($src, $dst, $comp) = @ARGV
die "usage: csv_to_parquet.stk SRC.csv DST.parquet [compression]\n" unless defined $dst
$comp ||= "zstd"

Arrow::convert($src, $dst, compression => $comp)

val $stats = Arrow::stats($dst)
p sprintf("wrote %d rows × %d cols (%s, %s) → %s",
  $stats->{num_rows}, $stats->{num_columns}, $comp,
  human_bytes($stats->{file_size}), $dst)
```

And `examples/read_parquet.stk`, which shows the footer-only-metadata / streamed-preview pattern:

```
use Arrow

val $path = $ARGV[0]
die "usage: read_parquet.stk PATH.parquet\n" unless defined $path

val $sch = Arrow::schema($path)
p "fields: " . join(", ", map { $_->{name} . ":" . $_->{type} } @{$sch->{fields}})
p "rows:   " . Arrow::row_count($path)

Arrow::read_stream($path,
  limit    => 100,
  callback => sub ($row) {
    p to_json($row)
  },
)
```

`schema` and `row_count` read only the Parquet footer — no full-file scan — and `read_stream` never buffers the whole dataset.

Config surface

There is no connection string or credential config — this package operates on local files, not a network service. The configurable surface is per-call `%opts`: `format`, `columns`, `limit`, `skip`, `batch_size` on reads; `format`, `compression` (`snappy|gzip|zstd|lz4|brotli|none`, Parquet only, default `snappy`), `row_group` (default 65536) on writes. Compression codecs map to concrete crates: `snap`, `zstd`, `flate2` (gzip), `lz4_flex`, `brotli`.

Install/discovery is filesystem-based: `s pkg install -g .` (or github.com/MenkeTechnologies/stryke-arrow) places `stryke.toml` and `lib/{Arrow.stk, libstryke_arrow.{dylib,so}}` under `~/.stryke/store/arrow@<version>/`. That directory is the entire discovery mechanism — no env vars, no PATH probing.

stryke-parquet — the Parquet file inspector

`stryke-parquet` (v0.19.0) does not connect to a database or a network service. It connects to a file format: Apache Parquet. The external system here is the columnar storage layer itself — the footer metadata, row groups, column chunks, and per-column statistics that Parquet writers embed on disk. It is diagnostic tooling: “I have a parquet file and I want to understand it,” as the README puts it ([README.md](#)), not a general-purpose data pipeline.

It ships as a separate opt-in package rather than living in stryke core for the same reason every connector in this book does: the core binary stays slim, and Parquet decoding (`parquet` crate, footer parsing, compression codecs) is dead weight for a script that never touches a `.parquet` file. It has a sibling, `stryke-arrow`, which handles full Arrow-based read/write pipelines across parquet/IPC/CSV/JSON; the README’s own comparison table draws the line precisely: `stryke-arrow` is for “I want to read or write parquet data and do things with it,” `stryke-parquet` is for “I have a parquet file and I want to understand it” ([README.md](#), section 0x00). Both packages link the same `parquet` and `arrow` crates, but `stryke-parquet`’s binary comes in smaller (~6 MB vs ~8.5 MB per the README’s own figures) because its surface is narrower — inspection and transform, not a full pipeline API.

What it wraps

`Cargo.toml` pins `parquet` = “58” (Arrow’s `parquet-rs`, `default-features` = `false`, with `arrow`, `snap`, `brotli`, `flate2`, `flate2-rust_backened`, `lz4`, `zstd`, and `base64` features enabled), plus `arrow` = “58”, `arrow-json` = “58”, and `arrow-csv` = “58” for the CSV/JSON conversion paths. The crate builds as a `cdylib` (`[lib] crate-type = [“cdylib”]`, `src/lib.rs`) named `stryke_parquet`.

Per `lib/Parquet.stk`’s header comment, this `cdylib` is dlopened in-process the first time a stryke script does `use Parquet` — resolved through stryke’s `pkg::commands::try_load_ffi_for` hook and registered via `rust_ffi::load_cdylib`. The comment is explicit that this replaced “the v1 helper-binary model (fork(2) + exec(2) + NDJSON over a pipe)” — the binding used to shell out to a subprocess and now calls straight into the loaded library. It’s a stateless package: “parquet operations are pure file inspection and transforms; no process-level state.”

Exported verbs

The authoritative export list lives in `[ffi].exports` in `stryke.toml` — every `parquet__*` C-ABI symbol the `cdylib` exposes. `lib/Parquet.stk` wraps each one in a `Parquet::*` stryke function that JSON-encodes its arguments, calls the FFI symbol, and decodes the JSON response through a shared `Parquet::_decode` helper that turns an `{error}` payload into a `die`. The surface, as documented in [README.md](#) section 0x04, spans:

- **Footer-only inspection:** `inspect`, `schema`, `count`, `rowgroups`, `row_group_summary`, `stats`, `metadata`, `dtypes`, `schema_diff`
- **Row read:** `head`, `tail`, `slice`, `reverse`, `gather`, `top_k`, `value_counts`, `to_json`, `to_csv`, `to_ndjson`, `stream` (per-row callback, no full buffering)

- **Polars-style reshaping:** `filter`, `where_count`, `filter_to`, `distinct`, `sort`, `column`, `sum`, `min`, `max`, `mean`, `n_unique`, `quantile`, `median`, `describe`, `group_by`, `random_sample`, `with_row_index`
- **Conversion/write:** `from_csv`, `from_json`, `write`, `write_partitioned`, `merge`, `hstack`, `select`, `drop`, `rename`, `repartition`, `compress`
- **Diagnostics:** `validate`, `column_chunk_stats`, `size_report`, `null_summary`, `encoding_summary`, `bloom_filter_summary`, `sorting_columns_summary`, `sample`, `features`

`validate` reads every row group and reports corruption as data rather than dying; `column_chunk_stats`, `size_report`, `encoding_summary`, and `bloom_filter_summary` read only the footer.

Usage

From `examples/inspect.stk`, the actual example shipped in the repo:

```
use Parquet

val $path = $ARGV[0]
die "usage: inspect.stk PATH.parquet\n" unless defined $path

val $info = Parquet::inspect $path
p sprintf("rows=%d row_groups=%d cols=%d size=%d (0.1f%% of raw)",
    $info->{num_rows}, $info->{num_row_groups}, $info->{num_columns},
    $info->{file_size},
    100.0 * ($info->{compression_ratio} // 0))
p "created by: " . ($info->{created_by} // "?")
p "compressions:"
for val $k (sort keys %{ $info->{compressions} }) {
    p "  $k → $info->{compressions}{$k} column-chunks"
}
```

`examples/discover.stk` composes it with the sibling package — synthesizing rows via `Arrow::write`, then reading them back with `Parquet::inspect`, `Parquet::schema`, `Parquet::count`, `Parquet::rowgroups`, and `Parquet::head`, entirely against a `/tmp` file, no network or external service required.

Config surface

There is no connection string or credential surface — the only “config” is the file path passed as the first argument to each verb, plus a handful of per-verb option hashes (`n`, `columns`, `codec`, `offset`, `row_group`, `q`, `seed`, and similar, documented inline in the `README.md` API reference). `stryke.toml`’s `[ffi]` block sets `lib-name = "stryke_parquet"` and `namespace = "Parquet"`, which is what binds the `use Parquet` namespace to the compiled `libstryke_parquet`. `{dylib,so}` artifact produced by `cargo build --release`. Compression codec selection (`snappy`, `zstd`, `gzip`, `lz4`, `brotli`, `uncompressed`) is passed as the `codec` option to `compress`, `from_csv`, `from_json`, `write`, and related write verbs — backed by `snap`, `zstd`, `flate2`, `lz4_flex`, and `brotli` respectively per the `Cargo.toml` feature list.

stryke-polars — the pandas + numpy surface

`stryke-polars` is not a database or network connector — it wraps a computation library. It puts the `polars` DataFrame engine, `ndarray`, `nalgebra`, `rustfft/realfft`, and `rand`-family distributions behind a single stryke package, `use Polars`, giving stryke scripts a pandas-plus-numpy surface without linking any of that into the core stryke binary.

Why it's a separate package

README.md:47-54 states the reasoning directly: stryke core stays small on purpose — “most one-liner / awk-replacement work doesn’t need 200 transitive crates of pandas + numpy machinery.” The core binary (~40 MB) carries string/math/regex/parallel builtins; anything with heavy, narrow-use dependencies — parquet, arrow, big-ML, cloud SDKs, and here “full pandas + numpy” — goes in the package tier instead. Concretely, `stryke-polars` ships as a `cdylib` (`libstryke_polars.{dylib,so}`, Cargo.toml:12-15) that stryke `dlopens` on first `use Polars`; the stryke-side `.stk` files are thin wrappers, the heavy code lives in the shared library, and “core stryke is never linked against any of them” (README.md:54).

What it wraps

Cargo.toml:1-131 pins the crate to `polars 0.45` with a large feature list (lazy, dtype-full, csv/parquet/json/sql IO, rolling windows, joins, string ops, etc.), plus `ndarray 0.16`, `nalgebra 0.33` for linear algebra (README.md:151-165 notes this is deliberately not `ndarray-linalg`+OpenBLAS, “avoiding ndarray-linalg → openblas-src → openblas-build → broken ureq 3 TLS chain”), `rustfft/realfft` for FFT, `rand/rand_distr/rand_chacha/ndarray-rand` for random generation, `rust_decimal` for the Decimal dtype, and `chrono/chrono-tz` for datetime64/timedelta64. Parquet/Arrow IO is deliberately routed through the sibling `stryke-arrow` package rather than linking `arrow-rs` directly, “to avoid `dlsym` conflicts” (README.md:165).

Exported surface

The FFI boundary is flat and JSON-based: each `#[no_mangle] extern "C" fn polars__*` in `src/lib.rs` takes a `*const c_char` (JSON args) and returns a `*mut c_char` (JSON result); the caller must release the result via the `cdylib`’s own `stryke_free_cstring` export, which stryke’s `rust_ffi::load_cdylib` wires automatically (README.md:145-149). `stryke.toml`’s `[ffi] exports` array lists every `polars__*` symbol stryke registers at load — e.g. `polars__arr_argmax`, `polars__arr_bincount`, `polars__arr_dot`, `polars__arr_diagonal`.

On the stryke side, `lib/` holds 46 `.stk` wrapper modules totaling 1,505 functions (README.md:87-137, `grep -c '^fn ' lib/*.stk`), namespaced per family: `Polars::DataFrame` (122 fns), `Polars::Series` (206), `Polars::NdArray` (124), `Polars::Ufunc` (70), `Polars::Linalg` (19), `Polars::Random` (20), `Polars::FFT` (8), and 38 more covering categorical, masked arrays, datetime64, image, signal, stats, text, geo, sparse, and graph operations. Each wrapper fn calls exactly one `polars__*` `cdylib` export — e.g. `lib/DataFrame.stk:24 fn Polars::DataFrame::head ($cols, $n)` returns `{ frame }`, first `$n` rows.

DataFrame ops are stateless: every method takes the raw column hash directly rather than a handle to a previously constructed object ([lib/DataFrame.stk:5-6](#)).

Usage — from [examples/groupby.stk](#)

```
use Polars
use Polars::DataFrame

val %cols = (
  region => ["east", "east", "east", "west", "west", "west", "north", "north",
    "north"],
  product => ["a", "b", "c", "a", "b", "c", "a", "b", "c"],
  revenue => [100, 150, 80, 90, 130, 70, 60, 85, 55],
  units => [10, 12, 8, 9, 11, 7, 6, 8, 5],
)

val ($nrows, $ncols) = @{ Polars::DataFrame::shape(\%cols) }
p "input: ${nrows} rows × ${ncols} cols"

# Sum revenue + units per region.
val $sum_region = Polars::DataFrame::groupby_sum(\%cols, ["region"])
p to_json($sum_region)

# Mean revenue + units per product (across regions).
val $mean_product = Polars::DataFrame::groupby_mean(\%cols, ["product"])
p to_json($mean_product)
```

[examples/discover.stk](#) runs the equivalent tour of [new/shape/columns/dtypes/head/select](#) against an in-memory `%cols` hash, with no external file or service dependency — both examples are self-contained and runnable via `s examples/<name>.stk`.

Config surface

There's no connection string or `.env` here — the config surface is install/load-time, not runtime credentials. `stryke.toml`'s `[package]` block carries name/version/description/edition; its `[ffi]` block (`lib-name = "stryke_polars", namespace = "Polars", exports = [...]`) is what stryke reads to know which symbols to resolve out of the shared library and under what `Polars:*` namespace to register them. Installation is `make install` (`cargo build --release && s pkg install -g .`) from source, or `s pkg install -g github:MenkeTechnologies/stryke-polars` from a release ([README.md:56-70](#)).

stryke-duckdb — the embedded analytical SQL engine

`stryke-duckdb` wraps `DuckDB` — an in-process, columnar, analytical SQL engine, the same query surface PostgreSQL uses, running embedded like SQLite rather than as a server process. It gives a stryke one-liner the ability to run SQL directly against Parquet, CSV, or JSON files on disk or over HTTP/S3, with no CREATE TABLE and no schema declaration step, plus optional persistent `.duckdb` files when state needs to survive the process.

It is an opt-in package (`s pkg install -g github.com/MenkeTechnologies/stryke-duckdb`), kept out of the stryke core binary. DuckDB's `bundled` feature compiles `libduckdb` from source and links it statically into the `cdylib` — a first build costs 3–5 minutes (Cargo.toml: “Adds ~3-5 minutes to a cold build”). Nobody who doesn't touch analytical SQL should pay that compile tax or carry the resulting binary weight in the default install.

What it wraps

Cargo.toml:

```
duckdb = { version = "1", features = ["bundled", "json", "parquet"] }
```

`crate-type = ["cdylib"]` — the crate builds `libstryke_duckdb.{dylib,so}`, not a Rust library consumed via `Cargo.toml` dependency. stryke's package runtime dlopens it in-process on the first `use DuckDB`, via the FFI resolver hook `pkg::commands::try_load_ffi_for` (README §0x05). Each `DuckDB::*` verb in the stryke-side wrapper (`lib/DuckDB.stk`) calls a sibling `duckdb__*extern "C"` symbol exported from `src/lib.rs`. The `cdylib` caches one `duckdb::Connection` per `(db, session, read_only)` tuple in a `OnceCell<Mutex<HashMap>>` for the life of the process — a named `:memory:` session persists across calls within one stryke run, unlike a v1 helper-binary design that forked a fresh empty `:memory:` per invocation (README, connection-options section).

Exported verbs

`stryke.toml [ffi] exports` lists 31 `duckdb__*` symbols; the `DuckDB::` namespace wrapping them covers read paths (`query`, `query_stream`, `query_one`, `query_col`, `query_scalar`, `dump`), DDL/DML (`execute`, `exec_file`, `explain`, `insert_many`, `appender`, `appender_columns`, `import`, `export`, `copy_from`, `create_index`, `drop/drop_index`, `attach/detach`, `update`, `delete`, `truncate`, `upsert`), a literal/identifier-quoting family (`quote_ident`, `quote_literal`, `quote_like`, `format_list`, `format_struct`, `format_map`, and their inverses), transactions (`begin`, `commit`, `rollback`, `transaction`), metadata (`tables`, `databases`, `views`, `functions`, `settings`, `extensions`, `schema`, `inspect`, `version`, `server_version`, `ping`, `count`, `exists`, `table_info`, `indexes`, `constraints`, `database_size`, `all_columns`, `schemas`, `memory_usage`, `describe_query`), and a pure-SQL analytics layer built on top of `query/execute` with no new FFI (`describe`, `columns`, `column_types`, `summarize`, `head`, `sample`, `distinct`, `aggregate`, `sum/avg/min/max`, `group_count`, `create_table_as`, `read_parquet/read_csv/read_json`, `copy_to`, `install_extension/load_extension`, `pragma`, `set/get_setting`, `vacuum`, `checkpoint`).

`appender` is DuckDB's native bulk-ingest path (arrayref-of-arrayrefs, no per-row SQL parse) and is documented as the fastest bulk-load route; `appender_columns` is its column-subset variant.

Usage — examples/aggregate_csv.stk

```
use DuckDB

val ($path, $col) = @ARGV
die "usage: aggregate_csv.stk PATH.csv GROUP_COLUMN\n" unless defined $col

val $sql = sprintf("SELECT %s, COUNT(*) AS n FROM read_csv_auto('%s') " .
                  "GROUP BY %s ORDER BY n DESC LIMIT 20",
                  $col, $path, $col)

val @rows = DuckDB::query($sql)
for val $r (@rows) {
    p sprintf("%-40s %d", $r->{$col} // "(null)", $r->{n})
}
```

No table exists before this runs — `read_csv_auto()` infers the schema from the file and DuckDB's vectorized scanner does the grouping. Six other examples ship in the repo (`query_parquet.stk`, `crud.stk`, `analytics.stk`, `discover.stk`, `parquet_to_db.stk`, `window.stk`), covering the read/CRUD/analytics/introspection surfaces above.

Config surface

Every `DuckDB::*` op takes a trailing `%opts` hash. Fields the cdylib reads (README §0x03): `db` (path to a `.duckdb` file; omitted = `:memory:`), `session` (name to scope a distinct in-memory instance; default `_default`), `read_only`, `pragmas` (arrayref of `SET name=value`; statements run on connect), `extensions` (arrayref — `INSTALL <name>; LOAD <name>;` per entry, e.g. `httpfs` for S3/HTTP reads, `aws`, `iceberg`, `delta`, `spatial`, `excel`). There is no separate connection- string surface — connection identity is the `(db, session, read_only)` tuple keyed into the process-lifetime connection cache.

Output typing

Result rows are produced via the Arrow result iterator, so JSON encoding matches `stryke-arrow`'s conventions: `HUGEINT/DECIMAL` come back as strings (precision preservation), `BLOB` as `"base64:..."`, `DATE/TIMESTAMP` as ISO-8601 strings, `LIST/STRUCT/MAP` as JSON array/object (README §0x07).

stryke-spark — the Apache Spark client

`stryke-spark` (v0.18.0) is the stryke binding for Apache Spark. It is an opt-in package, not part of the stryke core binary — the same rationale as the other `stryke-*` data connectors (`stryke-arrow`, `stryke-parquet`, `stryke-kafka`): Spark integration drags in a JVM, `spark-submit`, and PySpark on the host, and most stryke one-liners never touch Spark. Keeping it out of core keeps the daily-driver install slim; you `s pkg install -g` it only when a script needs Spark.

What it wraps

The crate (`Cargo.toml`) builds a `cdylib` named `stryke_spark` — there is no Spark Rust client underneath. Dependencies are generic: `serde/serde_json` for the JSON request/response envelope, `anyhow`, `tempfile`, `once_cell`. The actual Spark integration is an embedded PySpark driver script, `src/driver.py`, compiled into the `cdylib` via `include_str!` and written to a temp file at call time so `spark-submit` can execute it. The `cdylib` shells out to `spark-submit` per call and the driver reads a JSON request off `stdin/argv` and writes JSON rows to `stdout`. This works anywhere `spark-submit` is on `PATH`, across Spark 3.x and 4.x, without pulling a JVM binding crate into the Rust build.

The `cdylib` is dlopened in-process on first `use Spark` — stryke's FFI bridge (`pkg/commands.rs::try_load_ffi_for`) loads `libstryke_spark.{dylib,so}` and registers each `spark__*` export as a callable stryke function. `stryke.toml`'s `[ffi]` table lists 55 exports under `namespace = "Spark"`; `lib/Spark.stk` is the thin `.stk` wrapper that calls them, decodes the JSON, and dies on an `{error}` payload (`Spark::_decode` in `lib/Spark.stk:20`).

Honest scope note, from the README: each `Spark::*` call pays a fresh `SparkSession` boot through `spark-submit` — roughly 5–10s of JVM warmup, since there is no persistent-session daemon. That's deferred; it would need a long-running JVM sidecar process, which the README flags as bigger than the v0.2.1 helper-binary → `cdylib` refactor that got Spark this far. What the `cdylib` model removed is the `fork+exec` overhead of the old helper-binary architecture sitting on top of `spark-submit`, not the JVM startup cost itself.

Exported verbs

Read paths: `Spark::query`, `query_stream`, `query_one`, `query_col`, `query_scalar`, `dump`, `count`. DDL/DML: `Spark::execute`, `explain`. External I/O: `Spark::read` / `Spark::write` (parquet/csv/json/orc, `read` accepts `view => "v"` to query the loaded data in the same call). Metadata: `ping`, `tables`, `databases`, `views`, `catalogs`, `current_database`, `create_temp_view/drop_temp_view`, `set_database`, `refresh_table`, `schema`, `columns`, `functions`. Caching/config: `cache`, `uncache`, `config`. Submit pass-through: `Spark::submit` for `.py/.jar` jobs outside the SQL surface. Plus a large block of pure helpers with no Spark process involved — `parse_master_url/build_master_url`, `parse_table_name/build_table_name`, `parse_memory/build_memory/convert_memory`, `ident/literal` quoting, JDBC URL parsing, `conf-line` parsing, `partition-path` parsing, `duration/storage-level` parsing, Maven coordinate parsing — each with a documented inverse (`parse_X ↔ build_X`).

Usage (from `examples/quick_query.stk`)

```
use Spark

var @rows = Spark::query "
  WITH t AS (
    SELECT 'alice' AS name, 1.5 AS score UNION
    SELECT 'bob',      2.0      UNION
    SELECT 'charlie',   3.25
  )
  SELECT name, score, score * 2 AS doubled
  FROM t
  ORDER BY score DESC
"
for val $r (@rows) {
  p "$r->{name}: $r->{score} (doubled = $r->{doubled})"
}
```

`examples/discover.stk` shows the eval-guarded pattern used for CI (the suite must run clean with no Spark cluster present): `eval { Spark::ping() }` gates whether live queries run at all, falling back to printing the “expected call shape” when no driver is reachable.

Config surface

No `stryke.toml` connection block — connection options are passed per-call as `%opts` merged by `Spark::_req` (`lib/Spark.stk`): `master`, `spark_home`, `spark_submit`, `app_name`, `deploy_mode`, `packages`, `jars`, `database`, `confs` (hashref of `spark.*` settings), `limit`. Default `master` is `local[*]`; the README’s remote example passes `master => "spark://cluster:7077"` with a `confs` hash for executor memory/cores. Environment: `$SPARK_HOME` for locating `spark-submit`, and `$JAVA_HOME` — Spark 4.x needs JDK 17 (JDK 21+ trips a `getSubject is not supported` error in the Hive catalog path; the driver defaults `spark.sql.catalogImplementation=in-memory` to dodge it, but JDK 17 is still the smoothest path). Local runs drop `./spark-warehouse/` and `metastore_db/` in the cwd (gitignored).

Bind parameters are not supported at the FFI layer for v1 — Spark SQL’s `args=` keyword is gated on Spark Connect for some deployments, so values go inline into the SQL string using Spark literal syntax (`'string'`, `DATE '2026-01-01'`, etc.).

Tests: `cargo test` runs Rust unit tests with no live JVM; `JAVA_HOME=/path/to/jdk-17 s test t/` runs the end-to-end suite against `local[*]`, which skips cleanly when `spark-submit` isn’t on `PATH`.

stryke-kafka – the Apache Kafka client

`stryke-kafka` connects stryke to an Apache Kafka cluster (0.10+, including KRaft-mode brokers with no ZooKeeper): producer, consumer, consumer-group offset/lag introspection, and topic/cluster/config admin. It is an opt-in package, not part of stryke core, for a concrete reason stated in the README (`/Users/tommy/RustRoverProjects/MenkeTechnologiesMeta/stryke-kafka/README.md`): Kafka access requires `librdkafka`, the C client every Kafka tool eventually wraps, and the Rust binding pulls in a real build artifact. Bundling that into the daily-driver stryke binary would bloat every install for people who never touch Kafka. Instead it ships as a separate cdylib (`libstryke_kafka.{dylib,so}`) that gets dlopened in-process the first time a script says `use Kafka`.

What it wraps

The crate is `rdkafka` 0.39 (Cargo.toml), pulled in with `default-features = false` and the `cmake-build`, `libz`, `tokio` features. `cmake-build` is the deliberate choice here — it compiles `librdkafka` from source at build time instead of linking a system `librdkafka.so`, so the resulting cdylib is portable across glibc, musl, and macOS without a system dependency. The tradeoff is stated plainly: the first build takes 3-5 minutes to compile `librdkafka` via cmake; subsequent builds reuse the cached static archive. The crate is `stryke-kafka` v0.18.0, dual-tracked in both `Cargo.toml` (Rust crate) and `stryke.toml` (stryke package manifest), both pinned to the same version.

The binding holds a shared tokio runtime plus a `FutureProducer` cache keyed per `(brokers)` tuple in a `OnceCell`, per the README and `Cargo.toml`'s crate description. This replaces a v1 design (documented in the README's collapsed "v1 wire shape" section) that forked a `stryke-kafka-helper` binary per call — a model that defeated producer batching and compression because every call got a fresh producer. The persistent-cdylib model exists specifically to fix that.

How a stryke call reaches Kafka

Each `Kafka::*` function in `lib/Kafka.stk` builds a JSON args dict and calls a sibling `kafka__*` symbol resolved out of the cdylib — e.g. `Kafka::produce` calls `kafka__produce`. The full export list is `[ffi].exports` in `stryke.toml` (56 symbols: `kafka__ping`, `kafka__produce`, `kafka__consume`, `kafka__create_topic`, `kafka__partition_for_key`, and so on). Errors come back as a `{error}` JSON payload; `lib/Kafka.stk::_decode` dies with `Kafka::<op>: <reason>` on that shape.

Exported surface

From `README.md`'s API reference (matches `stryke.toml`'s `[ffi].exports`):

- **Producer:** `Kafka::produce $topic, $value, %opts, Kafka::produce_many \@rows, %opts` — opts cover `key`, `partition`, `timestamp`, `headers`, `encoding` (`utf8/base64/hex`).
- **Consumer:** `Kafka::consume $topic, %opts, Kafka::consume_stream $topic, %opts` (snapshot poll, then per-message callback) — opts cover `group`, `limit`, `timeout_ms`, `commit`.
- **Consumer offsets:** `Kafka::lag`, `Kafka::watermarks`, `Kafka::offsets_for_times`, `Kafka::committed`.

- **Admin:** `Kafka::topics`, `Kafka::describe`, `Kafka::groups`, `Kafka::describe_group`, `Kafka::cluster`, `Kafka::create_topic`, `Kafka::delete_topic`, `Kafka::create_partitions`, `Kafka::describe_configs`, `Kafka::alter_configs`, `Kafka::delete_groups`, `Kafka::delete_records`, `Kafka::ping`.
- **Pure/offline helpers** (no broker round trip): `partitioner` ports (`partition_for_key` — a faithful port of Kafka's `Utils.murmur2` with seed `0x9747b28c`, plus `partition_for_key_crc32`, `partition_for_key_fnv1a`), assignment simulators (`range_assignment`, `roundrobin_assignment`, `sticky_assignment`, `replica_assignment`, `assignment_diff`), and a set of config validators/canonicalizers (`compression_codec`, `cleanup_policy`, `acks_value`, `security_protocol`, `auto_offset_reset`, `isolation_level`, `timestamp_type`).

A real example

`examples/produce_one.stk`:

```
#!/usr/bin/env stryke
# examples/produce_one.stk - send a single message.
# Run:  KAFKA_BROKERS=localhost:9092 s examples/produce_one.stk my-topic 'hello'

use Kafka

val ($topic, $value, $key) = @ARGV
die "usage: produce_one.stk TOPIC VALUE [KEY]\n" unless defined $value

var %opts = {}
$opts{key} = $key if defined $key
$opts{headers} = { source => "stryke-example" }

val $r = Kafka::produce $topic, $value, %opts
p "wrote to $r->{topic}/$r->{partition} @ offset $r->{offset}"
```

`examples/group_lag.stk` shows the offset-introspection side, calling `Kafka::lag $group, %opts` and printing per-partition committed/high-watermark/lag rows.

Config surface

There is no `stryke.toml`-level Kafka config block. Connection is per-call: pass `brokers => "kafka-1:9094,kafka-2:9094"` as a named opt on any call, or set `$ENV{KAFKA_BROKERS}` once and omit it everywhere (`localhost:9092` is the fallback default shown in the README quick start). The README states plainly that SASL/SSL connection options are deferred in the 0.2.x line — `brokers` is currently the only per-call connection option, though `Kafka::security_protocol` exists as an offline canonicalizer for the `security.protocol` values (`PLAINTEXT/SSL/SASL_PLAINTEXT/SASL_SSL`) ahead of that wiring landing.

Tests run two ways per the README: `cargo test` compiles without live calls, and `KAFKA_BROKERS=localhost:9092 s test t/` does a live round trip (create temp topic, produce, consume, delete) against a real broker — the README includes a `bitnami/kafka:3.9` KRaft-mode docker one-liner for spinning one up locally.

stryke-zmq — the ZeroMQ client

`stryke-zmq` (crate version 0.17.0) is stryke's binding to ZeroMQ, the brokerless messaging library. It covers the four canonical patterns — request/reply, publish/subscribe, push/pull, dealer/router — plus PAIR, over TCP, IPC, or in-process (`inproc://`) transport.

Why a separate package

The Rust crate it wraps is `zmq = "0.10"` (Cargo.toml), which pulls in `zmq-sys`. `zmq-sys`'s build script vendors and compiles libzmq and libsodium from source via `zeromq-src` using cmake — a build step that needs cmake and a C/C++ toolchain and takes 1-2 minutes on first build. That's too much weight for the core stryke binary, so it ships as an opt-in package:

```
s pkg install -g github.com/MenkeTechnologies/stryke-zmq
```

The payoff of vendoring: the resulting `libstryke_zmq.{dylib,so}` has libzmq statically linked in. There is no `libzmq.so/.dylib` requirement on the machine actually running stryke code — only the build machine needs cmake.

How a stryke call reaches libzmq

The package is a Rust cdylib (`src/lib.rs`) dlopened in-process on first `use Zmq`, plus a thin stryke wrapper (`lib/Zmq.stk`). `stryke.toml`'s `[ffi]` table lists every exported symbol — `zmq__socket`, `zmq__send`, `zmq__recv`, `zmq__poll`, `zmq__curve_keypair`, and roughly sixty others — which stryke's FFI bridge resolves on load. Each `zmq__*` function is JSON-string-in / JSON-string-out: the wrapper encodes call arguments to JSON, the cdylib decodes them, does the libzmq work, and returns a JSON result that gets copied into a stryke string (freed via `stryke_free_cstring`). A handler error becomes `{"error": "...}"`, which the wrapper turns into `die "Zmq::<op>: <reason>"`; a Rust panic is caught at the FFI boundary rather than crossing it and aborting the host shell; a receive timeout returns `{"timeout": true}`, mapped to `undef` (or an empty list for the multipart/drain variants).

Sockets are stateful, so the API is handle-based: `Zmq::socket($type, %opts)` returns an integer handle that every later call takes. A shared `zmq::Context` and a socket-handle registry are held in `OnceCell` (`README.md` §0x01), guarded by a mutex — the same lock libzmq requires for one-thread-at-a-time access per socket. That's what lets a SUB socket keep receiving, or a REQ/REP conversation keep its state, across separate `Zmq::send/Zmq::recv` calls instead of paying reconnect cost each time.

Exported surface

Beyond the socket lifecycle (`socket`, `close`, `bind/connect`, `unbind/disconnect`, `send/recv`, `send_multipart/recv_multipart`, `subscribe/unsubscribe`, `set/get`, `poll/poll_many`), the package exports a substantial set of pure helpers that touch no socket at all: endpoint parsing/building (`parse_endpoint`, `build_endpoint`, `valid_endpoint`), topic-matching utilities for SUB filters (`topic_match`, `topic_overlaps`, `prune_subscriptions`, `subscription_diff`), the z85 codec plus CURVE key helpers (`z85_encode/decode`, `curve_keypair`, `curve_public`, `valid_curve_key`), socket-event monitoring

(`monitor/monitor_recv/parse_monitor_event/socket_event_names`), and topology-introspection tables (`socket_types`, `socket_caps_all`, `socket_types_compatible`). The full list — around sixty `zmq_*` exports — is enumerated in `stryke.toml`'s `[ffi].exports` array and documented function-by-function in `README.md` §0x04.

A real example

`examples/pub_sub.stk` — PUB/SUB with a topic filter, over `inproc://`:

```
use Zmq

val $endpoint = "inproc://example-pubsub"

val $publisher = Zmq::socket("pub", bind => $endpoint)
val $subscriber = Zmq::socket("sub", connect => $endpoint, subscribe => "weather")

# Give the subscription time to propagate, then publish.
for val $i (1:5) {
  Zmq::send($publisher, "weather sunny $i")
  val $msg = Zmq::recv($subscriber, timeout_ms => 200)
  if (defined $msg) {
    p "received: $msg"
    last
  }
}

Zmq::close($subscriber)
Zmq::close($publisher)
```

The comment in the source explains the retry loop: ZMQ's slow-joiner behavior means a SUB that hasn't finished connecting can miss messages sent immediately after `bind`, so the example sends a few and takes the first one that lands rather than assuming the first send arrives.

Configuration surface

There's no `stryke.toml` connection-string section to configure — ZeroMQ endpoints are passed per-call as strings following ZMQ's own transport syntax (`tcp://host:port`, `ipc:///tmp/sock`, `inproc://name`, `pgm://`, `epgm://`), validated by `Zmq::valid_endpoint/parse_endpoint`. The one process-wide knob is `Zmq::io_threads($value?)`, which gets or sets the shared context's `ZMQ_IO_THREADS` pool size (default 1). Per-socket tuning (timeouts, buffers, high-water marks, TCP keepalive, heartbeat, CURVE keys, PLAIN auth, identity) goes through the generic `Zmq::set/Zmq::get` pair rather than dedicated config file fields — the full option table is introspectable via `Zmq::socket_option_names()`.

Payloads default to UTF-8 framing; arbitrary bytes need `encoding => "hex"` or `encoding => "base64"` on both the send and matching recv call.

stryke-grpc — the reflection-based gRPC client

stryke-grpc talks to any gRPC server that has server reflection turned on — no `.proto` files required at call time. It fills the role `grpcurl` fills for curl-and-gRPC workflows, except the surface is a stryke package: `use Grpc`, then `Grpc::list`, `Grpc::describe`, `Grpc::call`, and the streaming trio, all JSON in/out over the existing stryke value model. Package version, per `Cargo.toml:3` and `stryke.toml:3`, is 0.18.0.

Why it's a separate package

A gRPC client pulls in `tonic`, `prost`, `tokio`, `hyper`, `rustls`, and the reflection descriptor machinery — README.md puts it at “~30+ transitive crates.” That’s weight the stryke core binary doesn’t carry for users who never touch gRPC. So stryke-grpc ships as an opt-in package: a thin `.stk` wrapper (`lib/Grpc.stk`) plus a Rust cdylib (`libstryke_grpc.{dylib,so}`) that gets dlopened in-process the first time a script does `use Grpc`. Installed via `s pkg install -g github.com/MenkeTechnologies/stryke-grpc` or from a local checkout with `cargo build --release && s pkg install -g .` — the built artifact lands in `~/.stryke/store/grpc@<version>/.`

The crate it wraps

`Cargo.toml` names the actual dependency stack: `tonic = "0.14"` (with `transport`, `tls-ring`, `tls-native-roots`, `gzip`, `zstd`, `deflate` features), `tonic-prost = "0.14"`, `prost = "0.14"`, and `prost-reflect = "0.16"` for the `DescriptorPool` reflection machinery. The crate is built as a `cdylib` (`[lib] crate-type = ["cdylib"], src/lib.rs:1`) rather than a normal Rust binary — it exists to be dlopened by stryke’s runtime, not `cargo run` directly.

Reflection flow, as documented in README.md §0x06: the cdylib opens a bidi stream to `grpc.reflection.v1alpha.ServerReflection/ServerReflectionInfo`, sends `ListServices` or `FileContainingSymbol`, follows `FileByFilename` import chains to build the full `FileDescriptorProto` graph, constructs a `prost-reflect::DescriptorPool`, and deserializes your JSON into a `DynamicMessage` before firing the call through tonic’s low-level `Grpc::unary` with a passthrough `BytesCodec`. A shared tokio runtime plus a `tonic::Channel` cache per endpoint and a `DescriptorPool` cache per `(endpoint, symbol)` are held in `OnceCell`, so repeated calls in one process reuse the same HTTP/2 connection instead of paying reflection cost per call.

Exported verbs

The FFI boundary is explicit in `stryke.toml`’s `[ffi]` block — `namespace = "Grpc"`, `lib-name = "stryke_grpc"`, and a 44-entry `exports` list. On the stryke side the RPC-facing verbs are:

```
Grpc::list           %opts → @services
Grpc::describe       $symbol, %opts → \%info
Grpc::call           $method, $request, %opts → \%response | $scalar | \@array
Grpc::server_stream  $method, $request, %opts → \%{ messages, count }
```

```
Grpc::client_stream $method, \@requests, %opts → \%response
Grpc::bidi_stream  $method, \@requests, %opts → \%{ messages, count }
Grpc::ping         %opts → 1 | ""
Grpc::version()    → $version_string
```

Each wrapper resolves a sibling `grpc__*` symbol out of the `cdylib` (`grpc__list`, `grpc__call`, `grpc__server_stream`, ...) via `stryke's pkg::commands::try_load_ffi_for` hook. Beyond the RPC calls, the same package exports roughly three dozen connection-free wire helpers — `status_code`, `parse_target`, `build_method`, `frame_message/unframe_message`, `valid_metadata_key`, `parse_timeout`, `health_status`, and more — pure functions over gRPC's wire conventions (status codes, `grpc-timeout` encoding, metadata key rules, target URIs) that open no connection at all.

Real usage

From `examples/call_unary.stk`:

```
#!/usr/bin/env stryke
use Grpc

var ($target, $method, $data_json) = @ARGV
die "usage: call_unary.stk HOST:PORT pkg.Service/Method '<json>'\n" unless defined $method

$data_json //= "{}"
val $reply = Grpc::call($method, from_json($data_json),
                        target => $target, plaintext => 1)
p "response: " . to_json($reply)
```

Run as `s examples/call_unary.stk localhost:50051 helloworld.Greeter/SayHello '{"name":"stryke"}'`. `examples/discover.stk` shows the defensive pattern used in CLI: `Grpc::ping` inside `eval` so the script degrades gracefully when no server is reachable, then `Grpc::list/Grpc::describe` only run against a live endpoint.

Config surface

There's no `stryke.toml` connection config — everything is per-call via `%opts` passed to each `Grpc::*` function: `target` (`host:port`, required), `plaintext` (force HTTP/2 cleartext), `authority` (SNI override), `headers` (hashref or arrayref, sent as gRPC metadata — `authorization => "Bearer $ENV{JWT}"` is the documented pattern), and `timeout_s` (default 30). Additional per-call options cover `deadline_ms`, `send_compression/accept_compression` (`gzip/zstd/deflate`), `max_recv_mb/max_send_mb`, `ca_cert/client_cert+client_key` for mTLS, and `max_messages` to cap a stream drain. The CLI (`grpc`) mirrors these as `--plaintext`, `--insecure`, `--authority`, `-H/--header`, `--timeout-s`.

Open items per the README's scope table: no `--proto FILE` fallback for servers with reflection disabled, and no unbounded/callback-style streaming — server/client/bidi streams are drained into JSON arrays, which is fine for bounded exchanges but not for infinite streams.

stryke-aws — the AWS client

`stryke-aws` is the stryke package for AWS: S3, DynamoDB, SQS, Lambda, STS, SNS, SSM Parameter Store, Secrets Manager, SES, CloudWatch, CloudWatch Logs, EC2, KMS, IAM, Kinesis, ECR, and Step Functions — seventeen services behind one `use AWS::*` surface (`stryke-aws/README.md`).

Why a package and not a builtin

The official `aws-sdk-rust` crates pull in `tokio`, `hyper`, `rustls`, and the `smithy/signing/endpoint-resolution` machinery underneath them. Baking seventeen SDKs into stryke core would bloat the daily-driver binary for every user, most of whom never touch AWS. `stryke-aws` keeps that weight opt-in: install it only if you need it (`stryke-aws/README.md`, “[0x00] Why this is a package, not a builtin”).

What it wraps

`Cargo.toml` names the dependencies directly — one `aws-sdk-*` crate per service, all pinned to major version 1: `aws-sdk-s3`, `aws-sdk-dynamodb`, `aws-sdk-sqs`, `aws-sdk-lambda`, `aws-sdk-sts`, `aws-sdk-sns`, `aws-sdk-ssm`, `aws-sdk-secretsmanager`, `aws-sdk-cloudwatch`, `aws-sdk-cloudwatchlogs`, `aws-sdk-sesv2`, `aws-sdk-ec2`, `aws-sdk-kms`, `aws-sdk-iam`, `aws-sdk-kinesis`, `aws-sdk-ecr`, `aws-sdk-sfn`, plus `aws-config` with `behavior-version-latest`. The crate itself is `stryke-aws` v0.19.0, built as a `cdylib` (`Cargo.toml`, `[lib] crate-type = ["cdylib"]`).

The stryke-specific work is the FFI shim in `src/lib.rs`: every exported function is a `#[no_mangle] extern "C" fn aws__*` — JSON string in, JSON string out — wrapping the corresponding `aws-sdk-rust` call. Two pieces of persistent state back every call: a single shared `tokio::runtime::Runtime` (`RUNTIME: OnceCell<Runtime>`) and a `HashMap<String, SdkConfig>` cache keyed by region (`CONFIGS: OnceCell<Mutex<...>>`), so credential/region resolution runs once per region, not once per call (`src/lib.rs:1-16`). The README is explicit that this replaced a v1 helper-binary model that forked a process per call and re-ran the full IMDS/SSO/env credential chain every time — v0.2.0 moved to in-process dlopen instead (`README.md`, “[0x00]”).

`stryke.toml` is the package manifest stryke reads: `[package] name = "stryke-aws", namespace = "AWS"`, and an `[ffi] exports` list of over 100 `aws__*` symbol names — the exact contract between the `.stk` wrapper files and the `cdylib`.

The exported surface

`lib/` holds one `.stk` file per service namespace (`AWS.stk`, `S3.stk`, `Dynamo.stk`, `SQS.stk`, `Lambda.stk`, `STS.stk`, `SNS.stk`, `SSM.stk`, `Secrets.stk`, `SES.stk`, `CloudWatch.stk`, `Logs.stk`, `EC2.stk`, `KMS.stk`, `IAM.stk`, `Kinesis.stk`, `ECR.stk`, `StepFunctions.stk`). Each defines the stryke-facing verbs that call into the matching `aws__*` FFI symbol. Representative signatures from the README’s API reference:

```
AWS::S3::ls      $uri, %opts → @entries
AWS::S3::get     $uri, %opts → $body
AWS::S3::put     $uri, %opts → \%resp
AWS::Dynamo::get $table, $key, %opts → \%item | undef
```

```

AWS::Dynamo::put  $table, $item, %opts → { ok: 1 }
AWS::SQS::pump    $queue, %opts → $count  # receive → callback → delete-on-
success
AWS::Lambda::call $name, $payload, %opts → $result

```

A handful of ops are documented as *deferred in the cdylib* rather than silently missing: `S3::presign`, `Dynamo::batch_write/scan_stream`, and Lambda's `invocation_type => "event"` fire-and-forget mode all *die* with a clear message today (README, "[0x03]"). There's also a block of pure helpers — `parse_arn`, `build_s3_uri`, `valid_bucket_name`, `arn_matches`, `service_endpoint`, and similar — that do string parsing/validation locally and open no AWS client at all.

A real example

`examples/whoami.stk` — an STS identity check plus a visible-bucket summary, `eval`-guarded so missing credentials exit cleanly instead of crashing:

```

use AWS
use AWS::STS
use AWS::S3

val $id = eval { AWS::STS::caller_identity() }
if ($@ || !defined $id || !defined $id->{account}) {
  p "no usable AWS creds – set AWS_PROFILE or run in an instance role to exercise the
  ~ demo"
  exit 0
}

p "account: $id->{account}"
p "arn:      $id->{arn}"

val @buckets = eval { AWS::S3::buckets() }
if (!$@) {
  p "S3 buckets visible: " . scalar(@buckets)
}

```

`examples/s3_browse.stk` shows the S3 listing verb against a CLI arg:

```

use AWS::S3

val $uri = $ARGV[0]
val @entries = AWS::S3::ls $uri, delimiter => "/"
for val $e (@entries) {
  if ($e->{type} eq "prefix") {
    p "  [dir] $e->{key}"
  } else {
    p sprintf("  [obj] %10d %s", $e->{size}, $e->{key})
  }
}

```

Configuration

There's no `stryke-aws`-specific connection string. Credentials and region come from the standard AWS resolution chain — env vars, `~/.aws/config/credentials`, IMDS — the same chain the `aws` CLI uses (README, "[0x00]"). Every public function also accepts named per-call overrides:

```
AWS::S3::ls "s3://...",  
  region    => "us-west-2",  
  profile    => "prod",  
  endpoint  => "http://localstack:4566"
```

`$ENV{AWS_ENDPOINT_URL}` works as a global default endpoint override, which is how the README documents pointing the package at LocalStack or MinIO for local testing (README, “[0x06]”). Install is via `s pkg install -g github.com/MenkeTechnologies/stryke-aws` from a release, or `cargo build --release + s pkg install -g .` from a local checkout, landing the cdylib at `~/.stryke/store/aws@<version>/`.

stryke-gcp — the Google Cloud client

`stryke-gcp` (v0.19.0) is the GCP connector in the stryke package tier: Cloud Storage, Pub/Sub, Secret Manager, BigQuery, Firestore, Compute Engine, Cloud Run, Cloud Functions, Cloud Logging, Cloud Monitoring, and IAM, all reachable from a stryke one-liner via `use GCP`.

Why a separate package

Same rationale as every `stryke-*` cloud connector: talking to GCP needs an Application Default Credentials (ADC) chain, a TLS HTTP client, and JSON marshaling on every call. That's real weight, and it's not weight every stryke install needs — most one-liners never touch a cloud API. So it's opt-in, shipped as a Rust cdylib (`libstryke_gcp.{dylib,so}`) that stryke's FFI bridge dlopens in-process the first time a script does `use GCP`. The core binary stays slim; the cost only shows up when the package is actually loaded.

What it wraps

`Cargo.toml` shows this is deliberately *not* built on Google's official SDK crates. It talks to GCP's REST APIs directly:

```
google-cloud-auth = "1"
request = { version = "0.12", features = ["rustls-tls-native-roots", "json", "stream"] }
tokio = { version = "1", features = ["macros", "rt-multi-thread", "io-std", "io-util",
  ↪ "fs"] }
serde_json = { version = "1", features = ["preserve_order"] }
```

`google-cloud-auth` handles the ADC chain (service-account JSON via `$GOOGLE_APPLICATION_CREDENTIALS`, `gcloud auth application-default login` on dev boxes, or the GCE/Cloud Run/GKE metadata server in production). Everything else — GCS objects, Pub/Sub messages, BigQuery jobs, Firestore documents, Compute instances, Cloud Run/Functions/Logging/Monitoring/IAM calls — is a `request` call against the corresponding REST endpoint, serialized/deserialized with `serde_json`. The README states the reasoning plainly: this sidesteps “the version-conflict tax from chrono / arrow / smithy that the proper SDK crates currently impose.”

`src/lib.rs` (4,822 lines) is the cdylib: one `#[no_mangle] extern "C" fn gcp__*` per operation, each a JSON-string-in/JSON-string-out wrapper. State is held once, not per-call — a shared `tokio::Runtime`, a shared `request::Client`, and a cached `Credentials` object all live in `OnceCell`, so a script that makes 50 GCS calls doesn't re-run ADC discovery 50 times.

The stryke side

`stryke.toml` declares the FFI contract — `namespace = "GCP"`, `lib-name = "stryke_gcp"`, and an `[ffi] exports` list of ~75 `gcp__*` symbols the cdylib must provide (`gcp__gcs_list_objects`, `gcp__pubsub_pull`, `gcp__bigquery_query`, `gcp__firestore_set`, `gcp__compute_start_instance`, and so on). The `lib/*.stk` files are the stryke-facing wrappers over those symbols — `Storage.stk`, `PubSub.stk`, `BigQuery.stk`, `Firestore.stk`, `Compute.stk`, `Run.stk`, `Functions.stk`, `Logging.stk`, `Monitoring.stk`, `IAM.stk`, plus `GCP.stk` for identity/ping and a set of pure, credential-free helpers

(GCP::parse_gs_uri, GCP::valid_bucket_name, GCP::parse_resource_name, GCP::parse_table_ref, etc.).

Every public op is a verb under its service namespace — `GCP::Storage::put`, `GCP::PubSub::pull`, `GCP::BigQuery::query`, `GCP::Firestore::get` — each takes an options hash and every one accepts a `project => "..."` override.

Usage — `examples/gcs_put_get.stk` (verbatim)

```
#!/usr/bin/env stryke
# examples/gcs_put_get.stk - round-trip a string through GCS.
# Run:   s examples/gcs_put_get.stk gs://my-bucket/demo.txt

use GCP::Storage

val $uri = $ARGV[0]
die "usage: gcs_put_get.stk gs://bucket/key\n" unless defined $uri

val $body = "hello from stryke at " . time()
val $r = GCP::Storage::put($uri, data => $body)
p "put: " . to_json($r)

val $back = GCP::Storage::get $uri
p "get: $back"

p "match: " . ($back eq $body ? "yes" : "no")

GCP::Storage::rm($uri)
p "deleted"
```

`examples/pubsub_pump.stk` shows the pull-callback-ack loop collapsed into one verb:

```
use GCP::PubSub

val $n = GCP::PubSub::pump($sub,
  iterations => 5,
  max         => 10,
  callback    => sub ($m) {
    p "got $m->{message_id}: $m->{data}"
  })
p "handled $n messages"
```

`GCP::PubSub::pump` is the one composite verb in the surface: pull → run the callback per message → ack, repeated for `iterations`, so a subscriber loop is one call instead of a hand-rolled pull/ack cycle.

Config surface

No connection string. Auth is pure ADC: point `$GOOGLE_APPLICATION_CREDENTIALS` at a service-account JSON, run `gcloud auth application-default login` locally, or run on GCP infrastructure and let the metadata server answer. Project defaults from `$GOOGLE_CLOUD_PROJECT`, overridable per call with `project => "..."`. Install is `s pkg install -g github.com/MenkeTechnologies/stryke-gcp` from a release, or `cargo build --release && s pkg install -g .` from a checkout, landing the cdylib in `~/.stryke/store/gcp@<version>/`.

Testing

`cargo test` compiles without live calls. `s test t/` runs `t/test_gcp.stk` end-to-end, gated on ADC reachability and opt-in env vars (`STRYKE_GCP_TEST_BUCKET`, `STRYKE_GCP_TEST_TOPIC`, `STRYKE_GCP_TEST_SUB`) — it skips cleanly when any of those aren't present. `t/test_stryke_gcp_surface.stk` pins wrapper completeness against the `stryke.toml` export list.

stryke-azure – the Azure cloud client

`stryke-azure` is a `cdylib` dlopened in-process by the stryke runtime, exposed to stryke scripts as the `Azure` package (`Cargo.toml`, `stryke.toml`). It is not part of the stryke core binary — it is an opt-in package installed with `s pkg install -g .` (`README.md`), which places `libstryke_azure.{dylib,so}` in `~/.stryke/store/azure@<ver>/`. Keeping cloud SDKs out of the core keeps the daily stryke install slim: nobody who never touches Azure pays for `azure_core`, `azure_identity`, `azure_data_cosmos`, and the rest of the dependency tree unless they `use Azure`.

What it wraps

Version `0.18.0` (`Cargo.toml`, `stryke.toml`). It wraps the official `Azure SDK for Rust`:

Dependency	Version
<code>azure_core</code>	1
<code>azure_identity</code>	1
<code>azure_storage_blob</code>	1
<code>azure_storage_queue</code>	1
<code>azure_data_cosmos</code>	0.35.0 (preview — Cosmos DB has not reached GA in the Rust SDK; <code>Cargo.lock</code> pins the exact preview build)
<code>azure_security_keyvault_secrets</code>	1
<code>azure_security_keyvault_keys</code>	1

Everything else (`url`, `futures`, `tokio`, `serde/serde_json`, `anyhow`, `base64`, `once_cell`) is plumbing.

The binding holds one shared `tokio` runtime and one Entra credential (`DeveloperToolsCredential` — the `az login / azd auth login` chain) reused across every service client — no per-call runtime spin-up, no per-call credential rebuild. Services with no GA typed Rust SDK (Resource Groups, Virtual Machines, Storage account management, Service Bus, Monitor, Container Instances, AKS) go over Azure Resource Manager REST using that same shared credential and the same `azure_core` HTTP client the typed data-plane clients use internally. One auth path, one runtime, one HTTP stack for both typed-SDK and REST-only surfaces — that unification is the actual engineering content of this binding, not a re-skin of the SDK.

Exported surface

`stryke.toml`'s `[ffi] exports` lists 90 `azure__*` C-ABI symbols the `cdylib` exposes; the `Azure` namespace maps them onto stryke sub-packages (`README.md`, “Packages” table):

- `Azure::Blob` — `az://container/blob` URIs: `ls`, `get`, `put`, `head`, `rm`, `containers`, `create_container`, `delete_container`, `set_metadata`.

- `Azure::Queue` — `ls`, `send`, `receive`, `delete`, `clear`, `count`, `create`, `drop`, plus a `pump` receive → callback → delete loop.
- `Azure::Cosmos` — `databases`, `containers`, `put`, `get`, `delete`, `query`, `replace`, `create_database`, `create_container`, `delete_database`, `delete_container`.
- `Azure::Secrets` / `Azure::Keys` — Key Vault secrets and keys (`get`, `set`, `ls`, `rm`, `versions`, `backup`; `encrypt`, `decrypt`).
- `Azure::Compute`, `Azure::Containers`, `Azure::Storage`, `Azure::ResourceGroups`, `Azure::ServiceBus`, `Azure::Subscriptions`, `Azure::Monitor` — the ARM management-plane surface (VMs, AKS/Container Instances, storage-account admin, resource groups, Service Bus, subscriptions/regions, metrics).
- A flat `Azure` package re-exports everything (`Azure::blob_*`, `Azure::cosmos_*`, ...) plus `Azure::identity_token()` and a set of credential-free pure helpers — resource-ID parsing/building, connection-string parse/build/redact, blob-URI parse/build, storage-endpoint resolution, and name/GUID validators (`valid_storage_account_name`, `valid_blob_name`, `valid_cosmos_id`, `normalize_guid`, `format_guid`, etc.) that open no client at all.

The service map deliberately mirrors `stryke-aws`: S3 → Blob Storage, DynamoDB → Cosmos DB, Secrets Manager → Key Vault Secrets, EC2 → Virtual Machines, and so on — a consistent verb shape across cloud connectors rather than an Azure-idiosyncratic API.

Usage

`examples/blob_browse.stk` is the working end-to-end example, listing a container prefix:

```
#!/usr/bin/env stryke
# examples/blob_browse.stk - list a container prefix, print name + size.
# Run:  AZURE_STORAGE_ACCOUNT=myacct s examples/blob_browse.stk
# az://my-container/some/prefix

use Azure::Blob

val $uri = $ARGV[0]
die "usage: blob_browse.stk az://container/prefix\n" unless defined $uri

val @entries = Azure::Blob::ls($uri)
for val $e (@entries) {
  p sprintf(" %12s %s", ($e->{size} // "-"), $e->{name})
}
```

`examples/discover.stk` is the CI-safe credential probe: it calls `Azure::identity_token()` inside an `eval` block, so it degrades gracefully — printing the expected call shapes instead of erroring — when no Azure credential is reachable in the CI environment.

Configuration

There is no persistent connection state beyond the shared credential; each call takes a trailing options hash naming its target, with an environment-variable fallback (README.md, “Connection options”): `account => / AZURE_STORAGE_ACCOUNT` for Blob and Queue, `account => / AZURE_COSMOS_ACCOUNT` for Cosmos (plus `region =>`), `vault => / AZURE_KEYVAULT_NAME` for Secrets and Keys, `subscription => / AZURE_SUBSCRIPTION_ID` for the ARM management plane (scoped ops add `resource_group =>`, sovereign clouds add `arm_endpoint =>`), and `namespace =>` for Service Bus data-plane calls. Authentication is `az login` or `azd auth login` beforehand — the crate never handles credentials directly, it defers to `DeveloperToolsCredential`.

stryke-k8s – the Kubernetes client

`stryke-k8s` (v0.19.0) is the Kubernetes binding for stryke: get, apply, delete, scale, patch, rollout, cordon/evict, watch, logs, and exec against any kubeconfig-reachable cluster — kind, k3s, minikube, EKS, GKE, AKS, OpenShift, vanilla. It ships as a separate package, not part of the stryke core binary. That split is deliberate: the daily-driver stryke install stays a small text-processing/scripting core, and anything that pulls in a Kubernetes API client, TLS stack, and async runtime is opt-in. `Cargo.toml` names the cost directly — `kube = "0.95"` (client/rustls-tls/config/runtime/derive features), `k8s-openapi = "0.23"` (full “latest” type schema), plus `tokio`, `futures-util`, `serde/serde_json/serde_yaml`. That’s real dependency weight a user who’s just piping text through stryke has no reason to carry.

What it wraps

The Rust side (`src/lib.rs`, `Cargo.toml`) is a `cdylib` built on the `kube` crate — the standard Rust Kubernetes client, talking the same REST/discovery API `kubectl` does, authenticated the same way (`$KUBECONFIG`, `~/.kube/config`, or in-cluster service account, no extra setup). `kube` and `k8s-openapi` do the actual HTTP/TLS/typed-manifest work; stryke-k8s’s own contribution is the FFI bridge and the verb surface.

Two persistence details are the binding’s own design, not `kube`’s: a shared `tokio` multi-thread `RUNTIME` and a `kube::Client` cache (`CLIENTS`) keyed by kubeconfig context, both held in `OnceCell` for the process lifetime (`README.md` §0x05). The README calls out the alternative it replaced — a v1 helper that rebuilt the client (full TLS+auth handshake) on every call. The FFI layer here reuses one client and HTTP pool across calls instead of forking or reconnecting per invocation.

Exported verbs

The package registers into stryke under the `K8s` namespace (`stryke.toml [ffi] namespace = "K8s"`), with each `K8s::*` stryke function calling a sibling `k8s__*` symbol resolved out of `libstryke_k8s.{dylib,so}` — dlopened in-process on first `use K8s`. `stryke.toml`’s `[ffi].exports` list is the authoritative export set; grouped by function, per `README.md` §0x04:

- **Read:** `get`, `get_one`, `get_yaml`, `exists`, `namespaces`, `nodes`, `api_resources`
- **Write:** `apply` (server-side apply), `create`, `replace`, `patch`, `delete_resource`, `delete_collection`, `scale`, `get_scale`
- **Rollouts/workload:** `set_image`, `rollout_restart`, `rollout_status`, `rollout_history`, `autoscale`, `taint`, `untaint`, `label`, `annotate`
- **Nodes/eviction:** `cordon`, `uncordon`, `evict`, `drain_filter` (pure)
- **Events/metrics/wait:** `events`, `top_pods`, `top_nodes`, `wait`
- **Logs/exec:** `logs` (buffered), `logs_follow`, `exec`
- **Connection/plumbing:** `version`, `ping`, `contexts`, `current_context`, `healthz`, `raw`, `pkg_version`
- **Pure helpers** (no cluster contact): `quantity math` (`parse_quantity/format_quantity/normalize_quantity/compare_quantity/sum_quantities/sub_quantity/scale_quantity/resource_ratio`), `selector/GVK/image-ref parsing and building`, `pod_status`, `container_images`, `condition`, `age_seconds`, `owner_refs`, `diff_merge_patch`.

Three ops are explicitly not live: `logs_follow`, `watch`, and `exec` are listed in `README.md` as “deferred in the v0.2.0 cdylib — they die until the callback FFI ships,” because v1’s `FfiSig::StrToStr` wire shape can’t

model a streaming callback yet.

Usage

From `examples/apply.stk` — server-side apply of a Namespace, ConfigMap, Deployment, and Service, then a scale call:

```
use K8s

val $ns = "stryke-demo"

K8s::apply { apiVersion => "v1", kind => "Namespace", metadata => { name => $ns } }

K8s::apply {
  apiVersion => "apps/v1",
  kind      => "Deployment",
  metadata  => { name => "echo", namespace => $ns, labels => { app => "echo" } },
  spec     => {
    replicas => 2,
    selector => { matchLabels => { app => "echo" } },
    template => {
      metadata => { labels => { app => "echo" } },
      spec     => {
        containers => [{
          name      => "main",
          image     => "hashicorp/http-echo:latest",
          args      => ["-text=hello from stryke-k8s"],
          ports     => [{ containerPort => 5678 }],
        }],
      },
    },
  },
}

val $r = K8s::scale("deploy", "echo", 3, namespace => $ns)
p " spec.replicas = $r->{spec}{replicas}"
```

`examples/get.stk` shows the read side — `K8s::get("pods", all_namespaces => 1, limit => 20)`, `K8s::get("svc", namespace => "kube-system")` — and `examples/cluster_info.stk` wraps calls in `eval { ... }` to exit cleanly when RBAC or kubeconfig isn't set up, the pattern used throughout the example set.

Config surface

No stryke-level connection config file — auth follows kubeconfig conventions directly: `$KUBECONFIG`, `~/.kube/config`, or an in-cluster service account. Every public function accepts per-call overrides (`context`, `kubeconfig`, `namespace`) as a trailing options hash, e.g. `K8s::get "pods", namespace => "payments", %prod` where `%prod = (context => "prod-eks")`. The standalone `k8s` CLI mirrors this with `--context/--kubeconfig/--default-namespace` flags and matching `$KUBE_CONTEXT/$KUBECONFIG/$KUBE_NAMESPACE` env vars.

stryke-docker — the Docker daemon client

`stryke-docker` (v0.19.0) binds the Docker Engine API into stryke as an opt-in package: `s pkg install -g github.com/MenkeTechnologies/stryke-docker`, then `use Docker` in any `.stk` script. It is not bundled with the stryke core binary — containers, images, networks, and volumes are a workload most scripts never touch, and every core-bundled dependency is startup weight and attack surface every stryke process pays for whether it uses Docker or not. Keeping it a separate package means the daily-driver install stays slim; you pull this in only when a script actually needs to talk to a daemon.

What it wraps

The crate is a `cdylib` (`src/lib.rs`, package `stryke-docker`, `Cargo.toml`) built on `bollard` 0.17 (`ssl` feature, default-features off) — the standard async Rust Docker Engine API client — driven by a `tokio` 1.x multi-thread runtime. `serde/serde_json` (with `preserve_order`) handle the JSON marshaling to and from the daemon's REST API. The binding targets any Docker-API-compatible endpoint: Docker Desktop, a Linux `dockerd`, Podman exposing the Docker socket, or a remote `DOCKER_HOST`.

The original engineering here is not the Docker client — that's `bollard`. It's the FFI bridge: a shared `tokio` runtime and a `bollard::Docker` client cache (`CLIENTS`, keyed by `DOCKER_HOST`) held in `OnceCell/parking_lot` for the process lifetime. The daemon connection is dlopened once on first `use Docker` via stryke's `pkg::commands::try_load_ffi_for` hook, and every subsequent `Docker::*` call reuses the same HTTP pool — no fork-per-call, no fresh connection per invocation.

Exported surface

`stryke.toml`'s `[ffi].exports` lists 68 `docker__*` symbols; each has a `Docker::*` wrapper in `lib/Docker.stk` that JSON-encodes its args and calls the matching `cdylib` export. They split into two tiers:

Daemon-backed (containers, images, networks, volumes, exec, logs, prune) — e.g. `Docker::ps`, `Docker::run`, `Docker::inspect`, `Docker::pull`, `Docker::exec`, `Docker::network_create`, `Docker::volume_rm`, `Docker::prune`. `stats` is a one-shot snapshot (`--no-stream`); the continuous-stream ops — `events`, `logs_follow`, `build`, `push` — are declared but `die` with a clear message, because they need a callback FFI / tar-stream / auth design the current blocking `StrToStr` call shape doesn't model (README §0x03, §0x07).

Pure helpers, no daemon required — `Docker::parse_image_ref`, `Docker::normalize_image_ref`, `Docker::build_image_ref`, `Docker::valid_container_name`, `Docker::valid_image_tag`, `Docker::parse_port_spec/build_port_spec`, `Docker::parse_mount/build_mount`, `Docker::parse_memory/format_memory`, `Docker::parse_restart_policy`, `Docker::parse_duration/format_duration`, and more — round-trip parsers/builders for the string grammars Docker's CLI flags use (`-v src:dst:opts`, `--memory 512m`, `--restart on-failure:3`, `--platform linux/arm64/v8`).

Example (from `examples/health.stk`)

```
use Docker

p "=== Docker health snapshot ==="

val $info = eval { Docker::info() }
if ($@ || !defined $info) {
  p "docker daemon not reachable - start docker or set DOCKER_HOST to exercise the demo"
  exit 0
}

p "Containers:      " . ($info->{Containers}      // 0)
p "  Running:      " . ($info->{ContainersRunning} // 0)
p "Images:         " . ($info->{Images}           // 0)
p "Server version: " . ($info->{ServerVersion}    // "?")

val @running = Docker::ps()
val $shown = scalar(@running) < 5 ? scalar(@running) : 5

if ($shown > 0) {
  for (var $i = 0; $i < $shown; $i++) {
    val $c = $running[$i]
    val $insp = eval { Docker::inspect($c->{Id}) }
    next if $@
    val $name = ($c->{Names}[0] // "?") =~ s|^/||r
    p "  $name ($c->{Image}): $insp->{State}{Status}"
  }
}
```

This is a real script from the repo (`examples/health.stk`), unmodified apart from a couple of elided lines — an `eval`-guarded daemon probe (`Docker::info`), then a `Docker::ps` / `Docker::inspect` pass over the first five running containers. The other examples cover `build.stk`, `discover.stk`, `logs.stk`, and `run.stk`.

Configuration surface

There is no `stryke.toml` connection config for the package itself — connection resolution is Docker's own convention: `$DOCKER_HOST` (unix socket path or `tcp://host:port`) if set, otherwise the local socket. Per-call overrides go through `%opts`, e.g. `Docker::ps host => "tcp://docker.example.com:2376"`, `timeout => 30`. TLS client certs (`DOCKER_CERT_PATH` / `DOCKER_TLS_VERIFY`) are listed in the README roadmap as not yet shipped.

Tests: `cargo test` compiles without live calls; `DOCKER_HOST=unix:///var/run/docker.sock s test t/` runs the live round-trip suite (`t/test_docker.stk`, `t/test_stryke_docker_surface.stk`), which pulls `busybox:latest`, spins up a uniquely-named container, and cleans up on exit.

stryke-scrape — the web scraping / crawling client

`stryke-scrape` is an opt-in stryke package for fetching, crawling, and extracting from HTML. It is not part of the core `stryke` binary — web scraping pulls in an async HTTP crawler and an HTML5 parser, dependencies most stryke scripts never touch, so the package ships separately and is installed on demand:

```
s pkg install -g github.com/MenkeTechnologies/stryke-scrape
```

What it wraps

The crawl engine is `spider` (`spider = { version = "2", features = ["sitemap"] }` in `Cargo.toml`). `stryke-scrape` does not reimplement crawling — `spider` is the vendored, durable engine; the crate's job is to expose it to stryke and layer HTML extraction on top. The pure extraction side runs on `scraper` 0.20, an `html5ever`-backed CSS selector engine, plus `url` 2 for absolute-URL resolution. Everything async runs on one embedded `tokio` multi-thread runtime (`rt-multi-thread`), created lazily and reused for the process lifetime — the stryke side stays a synchronous JSON call while `spider`'s crawl/scrape futures are driven with `block_on` underneath.

The crate builds as a `cdylib` (`src/lib.rs`, package name `stryke_scrape`). `stryke.toml`'s `[ffi]` table pins the exported symbol list and the `Scrape` namespace it's mounted under:

```
[ffi]
lib-name = "stryke_scrape"
namespace = "Scrape"
```

`use Scrape` dlopens `libstryke_scrape.{dylib,so}` in-process via stryke's `pkg::commands::try_load_ffi_for` resolver hook, then each `Scrape::*` call in `lib/Scrape.stk` is a thin wrapper that builds a JSON args dict and calls a sibling `scrape__*` extern "C" symbol. Errors come back as `{"error": "<msg>"}` and the wrapper `dies` with it.

Exported verbs

Two halves, both listed in `stryke.toml`'s `exports` array and documented in the README's API reference:

Network (drives `spider` on the shared `tokio` runtime):

```
Scrape::fetch    $url, %opts → { url, status, html }
Scrape::crawl    $url, %opts → { count, pages => [ { url, status, html? } ] }
Scrape::links    $url, %opts → { count, links => [ url, ... ] }
Scrape::sitemap  $url, %opts → { count, pages => [ ... ] }
```

Pure (no network — `scraper`-backed CSS selection over an HTML string you already have): `extract`, `extract_table`, `extract_links`, `extract_attrs`, `extract_text`, `extract_meta`, `extract_images`, `extract_feeds`, `structured` (JSON-LD / OpenGraph / Twitter cards), `microdata`, `headings`, `forms`, `select`, `select_text`, `absolutize`, plus URL/query-string helpers (`url_encode`, `url_decode`, `parse_query`, `build_query`, `url_parse`, `same_origin`, `normalize_url`, `set_query_params`) and `version`.

Crawl-family options: `limit`, `depth`, `subdomains`, `respect_robots` (default 1), `delay`, `concurrency`, `timeout_ms`, `blacklist`, `proxies`, `user_agent`, `include_html`.

Usage

From `examples/crawl.stk`:

```
use Scrape

val $url = $ARGV[0] // "https://example.com"

# bounded, robots-respecting crawl with a per-request delay
val $res = Scrape::crawl($url,
  limit      => 10,
  depth      => 2,
  delay      => 200,      # ms between requests
  concurrency => 4,
)
p "crawled $res->{count} pages"

# pull the <title> out of each fetched page
for val $p (@{ $res->{pages} }) {
  val $rec = Scrape::extract($p->{html}, { title => "title" })
  p "  [$p->{status}] $p->{url} - $rec->{title}"
}
```

And pure extraction, from `examples/extract.stk`:

```
val $rec = Scrape::extract($html, {
  title => "h1.title",
  price => "span.price",
})

# table → records
val $rows = Scrape::extract_table($html)
for val $r (@$rows) {
  p "  $r->{Feature} = $r->{Value}"
}
```

A selector ending in `@attr` (e.g. `"a.nav @href"`) pulls that attribute instead of element text; `extract_links` resolves relative hrefs when given a `base`.

Config surface

There is no `stryke.toml` runtime config for connection strings — the package has no persistent connection to configure, each call takes its target URL as the first positional argument and crawl/politeness knobs as `%opts`. The only environment hook is `SCRAPE_LIVE_URL`, used to opt in to a live-network test run (`SCRAPE_LIVE_URL=https://example.com s test t/`); without it, `cargo test` and `s test t/` exercise only the pure extraction engines and an offline wrapper-surface test, so CI runs with no egress.

Politeness

`crawl`, `links`, and `sitemap` default `respect_robots` to true and send an identifying `stryke-scrape/<version>` User-Agent; `delay` and `concurrency` throttle request rate. Per the README, respecting

a site's terms of use and rate limits is the caller's responsibility — the package sets courteous defaults, it doesn't enforce them.

stryke-selenium — the browser automation client

`stryke-selenium` binds stryke to Selenium WebDriver: launch a browser (Chrome, Firefox, Safari, Edge — headless or visible), navigate, query elements by CSS/XPath/id/name/tag/class/link-text, click/send_keys/clear, read attributes/properties/CSS, run JavaScript, capture screenshots (full-page or per-element), control windows/tabs/frames, and manage cookies. It is packaged as a separate opt-in stryke package, not part of the core binary — browser automation pulls in a WebDriver-protocol client and an async runtime bridge that most stryke programs never touch, so it stays out of the daily install and is fetched only when a script does `use Selenium` (`s pkg install -g github.com/MenkeTechnologies/stryke-selenium`).

What it wraps

The `cdylib` wraps `thirtyfour 0.35`, the async Rust WebDriver client, with `default-features = false` and `rustls-tls` enabled — a deliberate choice to keep the build off system OpenSSL, per `Cargo.toml`:

“rustls-tls so the `cdylib` doesn’t drag the openssl system library into the build matrix (macOS arm64 + Linux musl users have notoriously fragile openssl link paths; rustls is pure Rust).”

Supporting dependencies: `tokio 1.x` (`rt-multi-thread`, `macros`, `time`), `serde/serde_json` (JSON args/return marshaling), `anyhow`, `once_cell`. Crate version is `0.17.0` (`Cargo.toml`, `stryke.toml`).

How a stryke call reaches Selenium

This is the binding’s actual engineering content, not the browser automation itself. `src/lib.rs` documents the reasoning: `thirtyfour` is async, stryke’s FFI is synchronous, and per-call `Runtime::new().block_on(...)` would cost roughly 1ms of `tokio` bring-up on every `Selenium::*` invocation. So the package ships as a precompiled `cdylib` that stryke `dlopen`s in-process on first `use Selenium`:

1. Every export is a `#[no_mangle] extern "C" fn selenium__*(args_json: *const c_char) -> *const c_char` — JSON-string-in, JSON-string-out, wrapped by a shared `ffi_call` helper that catches panics and converts errors into `{"error": "..."} (src/lib.rs:42-67)`.
2. One process-global `tokio::runtime::Runtime` (built lazily, `src/common.rs::runtime`) bridges every call — built once, not per invocation.
3. WebDriver sessions and `WebElement` handles persist in `OnceCell<Mutex<HashMap<u64, _>>>` registries, keyed by integer session id / element id, so a script opens a browser once and drives it across many calls without re-handshaking.
4. `lib/Selenium.stk` is the thin stryke-side wrapper: JSON-encode args, call the FFI symbol, parse the JSON return. `stryke_free_cstring` is exported so the FFI bridge can free the `cdylib`’s allocation after copying it.

Multiple browser sessions are first-class — every `Selenium::open()` returns a fresh session id; calls default to the “active” session unless an explicit `$sid` is passed.

Exported API surface

stryke.toml's [ffi] table lists 90 `selenium__*` exports under `namespace = "Selenium"`, spanning six source modules (`driver.rs`, `element.rs`, `script.rs`, `actions.rs`, `capture.rs`, `window.rs`). Grouped as documented in the README:

- **Session lifecycle** — `open(%opts)`, `quit`, `quit_all`, `sessions`, `active`, `set_active`, `supported_browsers`, `locator_strategies`.
- **Navigation** — `goto/get`, `current_url`, `title`, `source`, `back/forward/refresh`, timeout setters, `status`.
- **Element queries** — `find`, `find_all`, `wait_for` (polls every 200ms), `active_element`.
- **Element ops** — `click`, `send_keys`, `clear`, `text`, `attr`, `prop`, `css`, `tag`, `rect`, `is_displayed/is_enabled/is_selected/is_clickable/is_present`, `inner_html/outer_html`, `focus`, `drop`.
- **JavaScript** — `execute_script`, `execute_async_script` (element args passed as `{ __element__ => $eid }`, JSON-parsed on return).
- **Action chains** — `action_click`, `action_double_click`, `action_context_click`, `action_send_keys`.
- **Screenshots** — `screenshot`, `element_screenshot`, `print_page` (PDF), `scroll_to_element`.
- **Window/frame** — `rects`, `handles`, `switch_window`, `new_window/new_tab`, named windows, frame switching, `maximize/minimize/fullscreen`.
- **Alerts** — `accept/dismiss/text/send_text`.
- **Cookies** — `cookies`, `get_named_cookie`, `add_cookie`, `delete_cookie(s)`.
- **Pure helpers** (no browser session required) — locator/cookie/CSS/XPath parsing and building: `parse_locator`, `build_locator`, `locator_to_w3c/w3c_to_locator`, `key_code/key_name`, `parse_cookie/build_cookie`, `cookie_domain_matches/cookie_path_matches` (RFC 6265 §5.1.3/§5.1.4), `css_escape/css_unescape/css_escape_string`, `build_css_selector/parse_css_selector`, `build_xpath`, `xpath_literal`.

Usage example

From `examples/selenium_form_fill.stk` (verbatim):

```
use Selenium

val $QUERY = "stryke programming language"

Selenium::open(browser => "chrome", headless => 1)
Selenium::goto("https://duckduckgo.com/")

val $box = Selenium::wait_for('input[name="q"]', "css", 10)
Selenium::clear($box)
Selenium::send_keys($box, $QUERY)

val $form = Selenium::find('form[role="search"]', form#search_form_homepage, form')
Selenium::execute_script("arguments[0].submit()", [{ __element__ => $form }])

Selenium::wait_for('a[data-testid="result-title-a"]', h2 a, article a', "css", 15)
p "result page title: " . Selenium::title()

Selenium::quit()
```

Configuration surface

There is no connection-string config file — the WebDriver server address and browser choice are runtime arguments to `Selenium::open(%opts)`: `browser` (chrome/firefox/safari/edge, default `chrome`), `url` (WebDriver server, default `http://localhost:9515`), `headless` (1/0). The package itself requires a WebDriver server running out-of-process (`chromedriver`, `geckodriver`, `safaridriver`, or a Selenium Grid 4 standalone

server) — `stryke-selenium` is a client only, per the README's `[0x04]` section, which documents launching each driver locally. `stryke.toml`'s `[ffi]` table (`lib-name`, `namespace`, `exports`) is the only package-level manifest surface; there's also a `[bin]` entry installing a permission-free smoke test (`selenium-test`, exercising `supported_browsers/locator_strategies/sessions` without a live browser).

stryke-mcpd – the MCP server discipline layer

`stryke-mcpd` (0.22.0, MIT, [MenkeTechnologies/stryke-mcpd](#)) is a pure-stryke package — `stryke.toml` has no `[ffi]` table, there is no `Cargo.toml`, no `cdylib`, no helper binary. It is not a binding to an external Rust client crate the way a Postgres or Kafka connector would be. It wraps *stryke core's own* MCP (Model Context Protocol) builtins — `mcp_server_start`, `mcp_connect`, `mcp_tools`, `mcp_call`, `mcp_resource`, `mcp_prompt`, `mcp_close`, `ai_register_tool`, and the `tool fn / mcp_server "name" { ... }` source-level desugar — and adds the discipline layer those primitives don't impose on their own (README §0x00). Everything ships as `.stk` modules under `lib/`, resolved by stryke's normal package loader on `use Mcpd`.

It is opt-in rather than core for a concrete reason stated in the README: the protocol plumbing is already in stryke core, but the parts every hand-rolled MCP server ends up rewriting — spec validation, crash isolation, a jailed filesystem/shell tool pack, result-envelope parsing — are policy, not protocol. Keeping policy out of the core binary keeps the daily-driver stryke install slim; a server that doesn't need `Mcpd::Tools`'s 35-function filesystem pack doesn't pay for it.

What it enforces

Two failure modes are specific to stdio MCP servers, and `Mcpd::Server` exists to close both (`lib/Server.stk:1-11`):

1. **stdout is the wire.** A stray `print` in a tool body corrupts the JSON-RPC stream for every connected client. `Mcpd::Server::log_to($path)` redirects diagnostics to a file (`lib/Server.stk:19-21`); `tests/repo-contract.sh` greps `lib/` in CI and fails the build if anything in the package writes to stdout.
2. **A dying tool must not kill the server.** `Mcpd::Server::wrap` runs each tool's coderef inside `eval`; a `die` becomes an `"ERROR: ..."` text result to the client instead of an EOF, with the stryke/Perl `" at FILE line N."` suffix stripped so an LLM client sees the message, not internal file paths (`lib/Server.stk:31-36`).

The four sublibraries

Module	File	Fns	Purpose
<code>Mcpd::Schema</code>	<code>lib/Schema.stk</code>	21	<code>tool()</code> builds a validated spec; <code>check_args</code> , <code>to_json_schema</code> , <code>from_json_schema</code> , <code>to_tool_list</code> , <code>from_tool_list</code> round-trip MCP's <code>tools/list</code> payload

Module	File	Fns	Purpose
<code>Mcpd::Server</code>	<code>lib/Server.stk</code>	9	<code>serve()</code> validates specs, wraps every <code>run</code> coderef, and calls <code>mcp_server_start</code> ; <code>log_to/log</code> for file-only diagnostics
<code>Mcpd::Tools</code>	<code>lib/Tools.stk</code>	35	root-jailed stock pack — <code>fs_read</code> , <code>fs_list</code> , <code>fs_grep</code> , <code>fs_write</code> , <code>sh_exec</code> (allowlist), <code>sys_info</code> , etc.; <code>all()</code> and <code>jail()</code>
<code>Mcpd::Client</code>	<code>lib/Client.stk</code>	16	<code>call_text</code> , <code>is_error</code> , <code>tool_names</code> , <code>tool_schema</code> — envelope extraction for the calling side

`lib/Mcpd.stk` (36 lines) is the aggregate `use Mcpd` entry point pulling in all four.

A real server, from `examples/calc_server.stk`

```
use Mcpd

Mcpd::Server::serve("calc", [
  Mcpd::Schema::tool("add", "Add two numbers",
    +{ a => "number", b => "number" },
    fn { val $args = shift; $args->{a} + $args->{b} })),

  Mcpd::Schema::tool("reverse_text", "Reverse a string",
    +{ text => "string" },
    fn { val $args = shift; join("", reverse split "", $args->{text}) })),

  Mcpd::Schema::tool("always_fails", "Demonstrates the error envelope",
    +{},
    fn { die "intentional failure" })),
])
```

`stryke examples/calc_server.stk` serves this over stdio. `s build --release` AOT-compiles it to a single static native binary with no interpreter needed on the target machine — the README's stated point of difference from Node/Python MCP servers.

The client side, from `examples/round_trip.stk`, drives that exact server:

```

val $h = mcp_connect("stdio:stryke $_dir/calc_server.stk")

p "tools: " . join(", ", @{ Mcpd::Client::tool_names($h) })
p "add(2, 40)      = " . Mcpd::Client::call_text($h, "add", +{ a => 2, b => 40 })
p "reverse(stryke) = " . Mcpd::Client::call_text($h, "reverse_text", +{ text => "stryke"
  ~ })

val $err = mcp_call($h, "always_fails", +{})
p "error envelope? = " . (Mcpd::Client::is_error($err) ? "yes" : "no")
p "server survived? = " . (Mcpd::Client::call_text($h, "add", +{ a => 1, b => 1 }) eq "2"
  ~ ? "yes" : "no")

mcp_close($h)

```

`t/test_mcpd.stk` runs this same round trip as a CI assertion: connect, list tools, call one, kill one with `always_fails`, then call `add` again on the same connection to prove the server survived the crash.

CLI and config surface

`bin/mcpd.stk` is the package's own CLI, invoked as `s bin/mcpd.stk <subcommand>`:

```

s bin/mcpd.stk new myserver      # write ./myserver.stk skeleton
s bin/mcpd.stk serve-stock /srv/data # stock pack jailed to /srv/data, on stdio
s bin/mcpd.stk tools             # list the stock tool pack

```

There is no connection-string or credentials config — the only runtime configuration is the jail root passed to `Mcpd::Tools::all(+{ root => ... })` (confines `fs_*/sh_exec` to that directory tree) and the log path passed to `Mcpd::Server::log_to($path)`. To wire a server into an external MCP client, the README's documented shape is `{ "command": "stryke", "args": ["server.stk"] }`, or point at the AOT-built binary directly with no args.

`Mcpd::Schema`'s explicit non-goals matter as much as its goals: anything that's a one-call passthrough to a stryke builtin (`mcp_connect`, `mcp_call`, `slurp`, `spurt`, `getcwd`, `qx`) is deliberately not re-wrapped (README §0x04) — “if a function in this library can be replaced with one builtin call, it's a bug.”

stryke-email — the SMTP + campaign client

`stryke-email` is the opt-in stryke package for sending mail: single transactional sends, personalized bulk campaigns with `{{merge}}` templates, List-Unsubscribe headers, suppression-list filtering, and rate limiting — all over an authenticated SMTP server the caller provides. It does not talk to a hosted provider's HTTP API (SES, SendGrid, Postmark); it is a `lettre`-backed SMTP client, nothing more, nothing less.

Why it's a separate package

Like `stryke-mongo` and `stryke-redis`, it ships as its own cdylib rather than being folded into the stryke core binary. Core stays small; a language install that never sends mail carries none of `lettre`'s TLS/SMTP machinery. On first `use Email` in a script, stryke dlopens `stryke_email` in-process and registers every `email__*` export (`stryke.toml:[ffi], namespace = "Email"`) — no process boundary, no RPC, just a loaded shared library.

What it wraps

`Cargo.toml` pins the transport:

```
lettre = { version = "0.11", default-features = false, features = [
  "builder", "smtp-transport", "pool", "hostname", "rustls-tls",
] }
```

`default-features = false` drops tokio: this is the blocking `SmtptTransport`, TLS via `rustls-tls` (no OpenSSL), with `lettre`'s built-in connection `pool`. A `SmtptTransport` is cached per `(host, port, tls, user)` for the life of the process, so a `send_bulk` campaign of a thousand recipients reuses one pooled connection rather than reconnecting per message. `src/lib.rs` is 1,440 lines — the FFI dispatch, message building, template merge, address parsing, and suppression/rate logic all live in this one crate. Crate version: `0.3.0`.

The exported surface

`stryke.toml` lists 27 `email__*` FFI exports; `lib/Email.stk` is the thin stryke-side wrapper around them (JSON args in, JSON result decoded and errors raised via `die`). Grouped as documented in the wrapper file:

Group	Functions
Send	<code>send</code> , <code>send_raw</code> , <code>send_bulk</code> , <code>send_template</code> , <code>build_message</code> , <code>verify_connection</code>
Templates	<code>render</code> , <code>merge</code> , <code>merge_many</code>

Group	Functions
Addresses	<code>validate, validate_list, validate_message, parse, split_addresses, format_address, domain, normalize</code>
Lists	<code>unsubscribe_header, suppress_filter, dedupe, chunk, group_by_domain, mailto</code>
URL	<code>parse_url, build_url, redact_url</code>
Meta	<code>version</code>

The template, address, and compliance helpers (`merge`, `validate`, `dedupe`, `chunk`, `suppress_filter`, ...) take no connection and run with no SMTP server — that's what CI exercises. The send-path functions need a live server, so `t/test_email.stk` gates them behind `$ENV{SMTP_HOST}` and short-circuits cleanly when it's unset.

Usage, from the repo

Single send (README, [0x01]):

```
use Email

var %conn = ( host => "smtp.example.com", username => "me", password => $ENV{SMTP_PASS} )

Email::send(
  {
    from    => "Me <me@example.com>",
    to      => "customer@example.com",
    subject => "Your license key",
    text    => "Thanks for your purchase. Key: ABC-123",
    html    => "<p>Thanks for your purchase. Key: <code>ABC-123</code></p>",
  },
  %conn,
)
```

Bulk campaign with suppression, rate limiting, and per-recipient unsubscribe (README, [0x02]):

```

val $res = Email::send_bulk(
  { subject => "Hi {{name}}, {{product}} v2 is out",
    html    => "<p>Hi {{name}}, your {{product}} update is ready.</p>" },
  [
    { email => "ada@example.com", name => "Ada", vars => { product => "Audio-Haxor"
  }, },
    { email => "alan@example.com", name => "Alan", vars => { product => "traderview" }
  },
  ],
  from          => "Me <me@example.com>",
  suppression   => Email::suppress_filter([], $unsub_list)->{removed},
  rate          => { per_minute => 120 },
  list_unsubscribe => { url => "https://example.com/u?e={{email}}", mailto =>
  "unsub@example.com" },
  %conn,
)

p $res->{sent}      # 2
p $res->{failed}    # 0

```

`Email::send_bulk` merges each recipient's `vars` into the template, skips anyone in `suppression`, injects the `List-Unsubscribe` header, throttles per `rate`, and returns `{ sent, failed, skipped, results }`.

Config surface

Connection params come in through `%opts` on every send-path call, or fall back to `$SMTP_HOST` / `$SMTP_USER` / `$SMTP_PASS` env vars (`Email::_req` in `lib/Email.stk`):

Key	Default	Notes
<code>host</code>	— (required)	SMTP server hostname
<code>tls</code>	<code>starttls</code>	<code>starttls</code> (587), <code>tls</code> (465), or <code>none</code> (25)
<code>port</code>	by <code>tls</code>	override the TLS-implied default
<code>username</code>	—	SMTP auth user
<code>password</code>	—	SMTP auth password

Testing against a live server without emailing anyone real: point `$SMTP_HOST` at a local catch-all like MailHog with `tls => none`, port 1025.

Compliance mechanics, not policy

The README is explicit that this is a tool for mailing your own opted-in recipients through your own SMTP — it isn't a way around anything. What it provides mechanically: RFC 2369/8058 `List-Unsubscribe` headers auto-added when `list_unsubscribe` is passed to `send_bulk`, suppression-list honoring via `suppress_filter`, rate limiting via `rate => { per_minute }` or `{ delay_ms }`, and address validation before send. Consent and honoring opt-outs (CAN-SPAM, GDPR, provider ToS) remain the sender's responsibility.

stryke-office — Excel, Word, Power-Point, ODF, and PDF I/O

`stryke-office` (v0.18.0) is the office-document connector for `stryke`. It reads and writes Excel/Calc (`xlsx/ods`), Word/Writer (`docx/odt`), PowerPoint/Impress (`pptx/odp`), and PDF, plus a PIL-scale raster image surface, entirely from native Rust crates. There is no `sooffice`, no LibreOffice headless process, no `pandoc` subprocess — nothing to shell out to and nothing extra to install on the target machine (README.md, [0x00]).

That's also why it's a separate package instead of a `stryke` core builtin. Office format support means dragging in a spreadsheet writer, an OpenDocument serializer, a `docx` writer, a PDF engine, and a full image-codec stack — real weight that has no business in the daily-driver core binary. `stryke-office` ships as a thin `stryke` library plus a Rust `cdylib` (`libstryke_office.{dylib,so}`), dlopened in-process the first time a script does `use Office`. Everything is statically linked pure Rust, so the `cdylib` is self-contained (README.md, [0x00]).

What it wraps

Cargo.toml lists the actual format crates behind the `Office::*` namespace — no single “office” crate exists upstream, so this package is a synthesis over several narrow ones:

- `calamine 0.35` — reads `xlsx/ods/xls/csv`
- `rust_xlsxwriter 0.95` — writes `xlsx`
- `lo_odf 0.4` / `lo_core 0.4` — OpenDocument (`ods/odt/odp`) serializers and a self-contained PDF reader/writer (no external font files)
- `lopdf 0.40` (default-features off) — PDF parse/merge/split/rotate, pure-Rust core
- `docx-rs 0.4` — writes `docx`
- `zip 2` + `quick-xml 0.40` — read/write the OOXML/ODF zip containers and the XML inside them
- `image 0.25` (with `avif`) + `imageproc 0.27` + `ab_glyph 0.2` — the raster codec and drawing stack
- `qrcode 0.14`, `barcoders 2` — barcode/QR generation

The `stryke` binding is the original work here: the package model (opt-in `cdylib`, dlopened on first `use`), and — this is the real scope of the crate — a very large exported verb surface built on top of those format crates: not just read/write, but spreadsheet analytics (`sheet_corr`, `sheet_regress`, `sheet_ttest`, `sheet_anova`), data-wrangling verbs mirroring `pandas`/SQL (`sheet_pivot`, `sheet_join`, `sheet_groupby`-style `sheet_aggregate`), document structure extraction (`doc_blocks`, `doc_outline`, `doc_comments`), and full PDF manipulation (`pdf_merge`, `pdf_watermark`, `pdf_form_fields`, `pdf_outline`).

Exported API

`stryke.toml`'s `[ffi]` block registers the namespace `Office` and lists every exported symbol (`office__*` on the Rust side, `Office::*` in `stryke`). The export list runs to several hundred functions across seven families: `sheet_*` (spreadsheet — read/write/transform/stats/dates/ranking), `doc_*` (Word/ODT structure, text mining, diffing), `slides_*` (PowerPoint/Impress), `pdf_*` (merge/split/annotate/forms/outline), `text_*` (Unix-text-tool-style verbs: `text_grep`, `text_sed`, `text_uniq`, `text_tr`), `img_*` (the PIL-complete image surface — resize/crop/convert/draw/filter), and small coordinate helpers (`parse_al`, `col_to_letter`, `range_union`). `Office::version()` returns the package version string.

Usage

From `examples/spreadsheet.stk`:

```
use Office

val $xlsx = "/tmp/office-example.xlsx"
val $ods  = "/tmp/office-example.ods"

# Write a workbook with one sheet.
Office::sheet_write($xlsx, [
  {
    name => "Sales",
    rows => [
      ["product", "units", "price"],
      ["widget", 120, 9.99],
      ["gadget", 75, 14.50],
    ],
  },
])
p "wrote $xlsx"

# Read it back.
val $sheets = Office::sheet_read($xlsx)
p "sheet '$sheets->[0]{name}' has " . scalar(@{ $sheets->[0]{rows} }) . " rows"

# Convert xlsx -> ods by re-writing the same data.
Office::sheet_write($ods, $sheets)
p "converted to $ods"
```

Four other examples ship in `examples/`: `presentation.stk`, `document_and_pdf.stk`, `excel_to_charts.stk`, and `image.stk`, covering the other three format families and the chart/image surface.

Config surface

There's no connection string or network config here — the package operates on local files. The output format is inferred from the path extension (`sheet_write("report.ods", ...)` writes ODS from the same in-memory `$sheets` structure that was read from xlsx), overridable per-call with a `format => "..."` option (README.md, [0x03]). Install is via `s pkg install -g github.com/MenkeTechnologies/stryke-office` from a release tarball, or `cargo build --release && s pkg install -g .` from a local checkout, landing the cdylib in `~/.stryke/store/office@<version>/` (README.md, [0x01]).

stryke-utils — the pure-stryke utility belt

`stryke-utils` is not a connector to an external system. There is no `Cargo.toml` in the repo, no `[ffi]` table in `stryke.toml`, no `cdylib`, no helper binary. It is a `.stk`-only package: six sublibraries — `String`, `List`, `Hash`, `Num`, `Time`, `Path` — of composite functions loaded via `use Utils`. The manifest states this directly:

```
[package]
name = "stryke-utils"
version = "0.22.0"
description = "Pure-stryke utility belt: String / List / Hash / Num / Time / Path helpers. No c"
(stryke.toml:2-4)
```

Why it's opt-in, not core

The README frames the split as a tier boundary, not a runtime boundary. `stryke` core ships `%b`, a builtin table with 10k+ entries — `slugify`, `chunk`, `deep_merge`, `format_bytes`, `levenshtein`, `basename`, and roughly seventy other composites that started life in this package and graduated into core once they proved broadly useful. `stryke-utils` is what's left: 149 functions that are still useful glue but sit below the bar for baking into the daily-install binary. The README is explicit that this is a build-time-enforced boundary, not a convention: “Every function in this repo is cross-checked against `%b` at build time — zero name collisions with builtins” ([README.md](#)), and lists an entire section ([0x04]) of functions that *used* to live here and were deleted the moment the corresponding builtin landed in core. So the “opt-in” framing isn’t about avoiding a heavy dependency (there isn’t one) — it’s about keeping the core binary’s builtin surface from growing unbounded with long-tail composites most scripts never call.

What it wraps

Nothing external. No Rust crate, no client library, no network protocol. The entire package is `lib/*.stk` resolved by `stryke`’s normal package loader — install and load cost is parsing `stryke` source, not linking a dynamic library or negotiating a connection.

Exported API

Six modules, 149 functions total, enumerated in full in the README’s sublibrary table:

Module	<code>use</code>	fns	Examples
String	<code>use Utils::String</code>	33	<code>pad_center</code> , <code>squeeze</code> , <code>partition</code> , <code>mask_</code> , <code>middle</code> , <code>escape_shell</code> , <code>unwrap</code>

Module	<code>use</code>	fns	Examples
List	<code>use Utils::List</code>	20	<code>difference,</code> <code>intersection, union,</code> <code>windows, transpose,</code> <code>top_n</code>
Hash	<code>use Utils::Hash</code>	26	<code>deep_merge_all,</code> <code>deep_get, deep_set,</code> <code>deep_has, flatten_</code> <code>keys</code>
Num	<code>use Utils::Num</code>	26	<code>round_to_multiple,</code> <code>ordinal, percent_</code> <code>change, digit_sum,</code> <code>gcd</code>
Time	<code>use Utils::Time</code>	18	<code>parse_duration, ago,</code> <code>format_iso8601,</code> <code>parse_iso8601,</code> <code>quarter</code>
Path	<code>use Utils::Path</code>	26	<code>compound_ext, set_</code> <code>ext, normalize,</code> <code>relative, with_name</code>

`use Utils` pulls all six; `use Utils::Path` (etc.) pulls one module if that's all a script needs. A `bin/Utils.stk` CLI dispatcher exposes a subset of these as subcommands (`pad-center`, `squeeze`, `mask`, `ordinal`, `parse-duration`, `ago`, `iso`, `compound-ext`, `normalize`, `relative`, ...) — see `README.md` [0x05].

A real usage example

`examples/config_merge.stk` layers three config hashes (defaults → env overrides → runtime overrides) with `Utils::Hash::deep_merge_all`, then reads nested keys with `Utils::Hash::deep_get`:

```

use Utils

val $defaults = {
  db => {
    host => "localhost",
    port => 5432,
    pool => { min => 1, max => 10 },
  },
  cache => { ttl => 60 },
  log   => { level => "info" },
}

val $from_env = {
  db   => { host => "db.prod.internal", pool => { max => 50 } },
  log  => { level => "warn" },
}

val $runtime = {
  db   => { pool => { max => 100 } },
  cache => { ttl => 300, enabled => 1 },
}

val $cfg = Utils::Hash::deep_merge_all($defaults, $from_env, $runtime)

p "db.pool.max      : " . Utils::Hash::deep_get($cfg, "db.pool.max")
p "db.pool.min     : " . Utils::Hash::deep_get($cfg, "db.pool.min") # from defaults

```

([examples/config_merge.stk:1-38](#)). `deep_merge_all` folds later hashes over earlier ones at any nesting depth; `deep_get` reads by dot path and distinguishes a missing key from an explicit `undef` via the sibling function `deep_has`.

Config surface

There is no connection string, no credential, no environment variable this package reads. The only configuration is the package manifest itself (`stryke.toml`), which declares `test/install` scripts (`s test t/`, `s pkg install -g .`) and carries no `[ffi]` or dependency table. Installation is `s pkg install -g github.com/MenkeTechnologies/stryke-utils` or, from a checkout, `s pkg install -g ./make install`; the store directory ends up holding `stryke.toml` + `lib/*.stk` only — no compiled artifact.

stryke-gui — the desktop automation cdylib

`stryke-gui` connects stryke scripts to the local desktop input/display stack: mouse motion and buttons, keyboard press/type/hotkey, screen geometry, pixel reads, screenshots, and clipboard get/set. It is the stryke equivalent of PyAutoGUI — the README says as much: “*PyAutoGUI, one stryke pipe away.*”

It ships as a separate opt-in package rather than living in the stryke core binary for a structural reason, not a licensing one: the crates it wraps pull in the platform input/display stack directly — X11 XTest/XGetImage, Wayland virtual-pointer + portal, macOS CGEvent + ScreenCaptureKit, Win32 SendInput/GDI. `Cargo.toml` spells this out in a comment on the `[dependencies]` block: “*the cdylib model isolates them here so the stryke core binary needs zero X11/Wayland system libs to build.*” A stryke install that never touches a GUI doesn’t pay for `libxcb`, `libwayland-dev`, or a Screen Recording permission prompt.

What it wraps

`Cargo.toml` (`stryke-gui` v0.19.0) pins:

- `enigo` = “0.6” — mouse and keyboard event synthesis
- `xcap` = “0.9” — screen capture and pixel reads
- `arboard` = “3” — cross-platform clipboard
- `image` = “0.25” (png feature only) — screenshot encoding
- `serde` / `serde_json` (`preserve_order`) — the FFI wire format
- `once_cell`, anyhow

The crate builds as `crate-type = ["cdylib"]` (`Cargo.toml` `[lib]`), not an executable, not a `rlib`.

How a stryke call reaches Enigo

This is the second generation of the package. `src/lib.rs` docs the change directly: the v0.1.x predecessor, `stryke-gui-helper`, was a subprocess spawned per `GUI::*` call — one `fork(2)` + `exec(2)` + `Enigo::new()` every time. The current shape drops that to a single `dlopen` on first `use GUI`. Each `#[no_mangle] extern "C" fn gui__*` in `src/lib.rs` is a JSON-string-in / JSON-string-out wrapper (`ffi_call`) around the `mouse` / `keyboard` / `capture` / `clipboard` modules. `stryke.toml`’s `[ffi]` table names every exported symbol (`gui__mouse_move`, `gui__key_type`, `gui__pixel`, `gui__clipboard_get`, ~60 total including a large block of pure color-math helpers) and sets `namespace = "GUI"`. `lib/GUI.stk` is the thin stryke-side wrapper that JSON-encodes call arguments, invokes the FFI symbol, and parses the JSON return — that file, not `src/lib.rs`, is what a `use GUI` script actually links against at the language level.

The `Enigo` handle itself lives in a process-global `OnceCell<Mutex<Enigo>>` (`src/common.rs::enigo_lock`), constructed once and reused across every subsequent call for the life of the stryke process — no re-init per invocation.

A real example

`examples/gui_screen_info.stk`, a read-only introspection demo:

```
use GUI

val @s = GUI::screen_size()
p "screen_size:    #{$s[0]} x #{$s[1]} pixels"

val @p = GUI::mouse_pos()
p "cursor at:      (#{$p[0]}, #{$p[1]})"

for val $pt ([0, 0], [int($s[0] / 2), int($s[1] / 2)],
              [$s[0] - 1, $s[1] - 1], [$s[0] + 100, 0]) {
  val $x = $pt->[0]
  val $y = $pt->[1]
  p "GUI::on_screen(#{$x}, #{$y}) → #{GUI::on_screen($x, $y)}"
}

val @keys = GUI::keyboard_keys()
p "keyboard_keys:  #{len(@keys)} recognized names"
```

`examples/gui_mouse_circle.stk` shows the active-input side: `GUI::mouse_move($x, $y, $STEP_DURATION)` called in a loop over `sin/cos` to sweep the cursor in a circle, with `$SIG{INT}` restoring the starting position on Ctrl-C.

Config surface

There is no connection string or `stryke.toml [connection]` block — the target is always the local desktop, not a remote service, so the only configuration is install-time: `s pkg install -g github.com/MenkeTechnologies/stryke-gui[@version]`, with `STRYKE_TARGET` overriding the auto-detected host triple (e.g. for musl). Runtime “configuration” is OS permission grants, not a manifest key: macOS gates the first mouse/keyboard call behind Accessibility and the first pixel/screenshot call behind Screen Recording (one-time terminal-app grants); Wayland needs the `wlroots-virtual-pointer` protocol and the `org.freedesktop.portal.Screenshot` portal; X11 and Windows have no permission gate. `examples/stryke-gui-sample/stryke.toml` is a scaffolded consumer project (`main.stk`, a bench, a test) showing how a downstream stryke project depends on this package, not a config surface of the package itself.

Writing Your Own Battery — how a connector is built

Every chapter so far documented a battery from the outside. This one builds one from the inside. The fleet's shared shape — a stryke library, a Rust cdylib, a `stryke.toml` that wires them, a JSON boundary, a process-lifetime cache — is not a private convention; it is the public contract any connector can implement. If the system you need to reach is not in this book, the model here is how you add it.

The connector built in this chapter is a small one on purpose: a StatsD metrics client, `stryke-statsd`, that fires counters, gauges, and timers at a StatsD daemon over UDP. It is a *teaching example* — it does not ship in the fleet, and it is deliberately chosen because StatsD's wire protocol is a one-line text format over a connectionless socket, so the client code stays small enough to read in full while every piece of the package model still appears. Everything structural here — the manifest, the ABI, the cache, the wrappers, the install — is exactly what the shipping connectors do; only the wrapped system is trivial.

Step 1 — the manifest

A connector is defined by its `stryke.toml` before a line of Rust exists. The `[ffi]` table is the contract between the two halves: the library name the runtime `dlopens`, the namespace the package mounts under, and the full list of `extern "C"` symbols the cdylib promises to export.

```
[package]
name       = "stryke-statsd"
version    = "0.1.0"
description = "StatsD metrics over UDP: counters, gauges, timers, sets."
edition    = "2024"

[ffi]
lib-name = "stryke_statsd"
namespace = "Statsd"
exports = [
    "statsd__count", "statsd__gauge", "statsd__timing", "statsd__set",
    "statsd__incr",  "statsd__decr", "statsd__flush", "statsd__ping",
    # pure helpers – no socket
    "statsd__format_metric", "statsd__parse_metric",
    "statsd__valid_metric_name", "statsd__build_url", "statsd__parse_url",
]

[scripts]
build = "cargo build --release"
test  = "s test t/"
install = "s pkg install -g ."
```

Three things to notice, because they are load-bearing across the whole fleet. The `exports` array is authoritative — it is the list the runtime registers on load, and a wrapper that calls a symbol not in this list fails to resolve. The `namespace` is what turns `statsd__count` on the Rust side into `Statsd::count` in stryke. And the `[scripts]` block gives the standard `build` / `test` / `install` verbs every package answers to, all driven through `s`, the stryke CLI.

Step 2 – the ABI floor

Now the Rust. The crate is a `cdylib` — nothing else — and every export has the one C signature from the FFI-boundary chapter: a JSON string in, a JSON string out. A single dispatch helper decodes the arguments, runs a closure inside `catch_unwind`, and encodes either the result or an `{ error }` envelope. Writing that helper once keeps every export a two-liner.

```
// Cargo.toml: [lib] crate-type = ["cdylib"], name = "stryke_statsd"
use std::ffi::{CStr, CString};
use std::os::raw::c_char;
use std::panic::catch_unwind;
use serde_json::{json, Value};

/// Decode args JSON, run `f`, encode the reply. A panic or an `Err`
/// becomes `{ "error": "<msg>" }` — never an unwind across the boundary.
fn dispatch(args_json: *const c_char, f: impl FnOnce(Value) -> anyhow::Result<Value>) ->
    *mut c_char {
    let out = catch_unwind(|| {
        let s = unsafe { CStr::from_ptr(args_json) }.to_str().unwrap_or("{}");
        let args: Value = serde_json::from_str(s).unwrap_or(json!({}));
        match f(args) {
            Ok(v) => v,
            Err(e) => json!({ "error": e.to_string() }),
        }
    })
    .unwrap_or_else(|_| json!({ "error": "panic in statsd handler" }));
    CString::new(out.to_string()).unwrap().into_raw()
}

/// Free a reply the cdylib allocated — the runtime calls this after copying.
#[no_mangle]
pub extern "C" fn stryke_free_cstring(p: *mut c_char) {
    if !p.is_null() { unsafe { drop(CString::from_raw(p)); } }
}
```

That is the entire boundary contract: `dispatch` on the way in, `stryke_free_cstring` on the way out. Both are copied, structurally, from what the shipping packages do — the `stryke-polars`, `stryke-zmq`, and `stryke-gui` chapters all name the same `stryke_free_cstring` export and the same catch-at-the-seam rule.

Step 3 – the connection cache

StatsD is connectionless, but even a UDP socket is worth holding across calls rather than reopening per metric — and the moment a connector has *any* per-endpoint state, it uses the fleet's cache pattern. A `OnceCell<Mutex<HashMap>>` keyed by the destination address, exactly as every data connector in this book keys its client:

```

use once_cell::sync::OnceCell;
use parking_lot::Mutex;
use std::collections::HashMap;
use std::net::UdpSocket;

static SOCKETS: OnceCell<Mutex<HashMap<String, UdpSocket>>> = OnceCell::new();

fn socket_for(addr: &str) -> anyhow::Result<UdpSocket> {
    let map = SOCKETS.get_or_init(|| Mutex::new(HashMap::new()));
    let mut guard = map.lock();
    if let Some(s) = guard.get(addr) {
        return Ok(s.try_clone()?);
    }
    let s = UdpSocket::bind("0.0.0.0:0")?;
    s.connect(addr)?;
    guard.insert(addr.to_string(), s.try_clone()?);
    Ok(s)
}

```

The key here is the `host:port` string — the StatsD analogue of `stryke-postgres` keying by URL or `stryke-docker` keying by `$DOCKER_HOST`. Two scripts talking to two daemons get two cached sockets; a hundred metrics to one daemon reuse one.

Step 4 — an export and its pure helper

With `dispatch` and the cache in place, an export is short. `statsd__count` formats one metric line and sends it; `statsd__format_metric` is a *pure* helper that formats the line and touches no socket, so it is unit-testable with no daemon running — the same “pure helper” tier every connector chapter documents.

```

/// Pure: format one StatsD line. No I/O — testable in `cargo test`.
fn format_metric(name: &str, value: &str, kind: &str, rate: Option<f64>) -> String {
    match rate {
        Some(r) if r < 1.0 => format!("{name}:{value}|{kind}|@{r}"),
        _ => format!("{name}:{value}|{kind}"),
    }
}

#[no_mangle]
pub extern "C" fn statsd__format_metric(a: *const c_char) -> *mut c_char {
    dispatch(a, |v| {
        let line = format_metric(
            v["name"].as_str().unwrap_or_default(),
            &v["value"].to_string(),
            v["kind"].as_str().unwrap_or("c"),
            v["rate"].as_f64(),
        );
        Ok(json!({ "line": line }))
    })
}

#[no_mangle]
pub extern "C" fn statsd__count(a: *const c_char) -> *mut c_char {
    dispatch(a, |v| {
        let addr = v["addr"].as_str().unwrap_or("127.0.0.1:8125");
        let line = format_metric(
            v["name"].as_str().unwrap_or_default(),
            &v["n"].as_i64().unwrap_or(1).to_string(),
            "c",
            v["rate"].as_f64(),
        );
        socket_for(addr)?.send(line.as_bytes())?;
        Ok(json!({ "sent": 1, "line": line }))
    })
}

```

Every other export — `gauge`, `timing`, `set`, `incr`, `decr` — is a variant of `count` with a different `kind` byte (`g`, `ms`, `s`) and a different value convention, and each has the same shape: pull fields out of the JSON args, do the work, return a JSON reply.

Step 5 — the stryke side

`lib/Statsd.stk` is the package's stryke face — one thin wrapper per export, plus the two helpers (`_req` for option/endpoint merging, `_decode` for the error envelope) the whole fleet centralizes. Nothing here is bespoke; it is the shape `lib/Neo4j.stk` and `lib/Kafka.stk` use, applied to StatsD's verbs.

```
# lib/Statsd.stk - the Statsd:: namespace.

fn Statsd::_decode ($reply) {
  val $o = from_json($reply)
  die "Statsd::" . $o->{op} . ": " . $o->{error} if defined $o->{error}
  $o
}

fn Statsd::_addr (%opts) {
  $opts{addr} // $ENV{STATSD_ADDR} // "127.0.0.1:8125"
}

fn Statsd::count ($name, %opts) {
  Statsd::_decode statsd__count(to_json(+{
    name => $name,
    n    => ($opts{n} // 1),
    rate => $opts{rate},
    addr => Statsd::_addr(%opts),
  })))
}

fn Statsd::timing ($name, $ms, %opts) {
  Statsd::_decode statsd__timing(to_json(+{
    name => $name, value => $ms, addr => Statsd::_addr(%opts),
  })))
}

# pure - no socket, no daemon
fn Statsd::format_metric ($name, $value, $kind) {
  Statsd::_decode(statsd__format_metric(to_json(+{
    name => $name, value => $value, kind => $kind,
  })))>{line}
}

```

Step 6 – an example and a test

A connector ships with a runnable example under `examples/` and a gated test under `t/`. The example follows the fleet's `ping`-guarded pattern so it exits cleanly when no daemon is listening — the same `eval { ... }` guard the `stryke-redis`, `stryke-mongo`, and `stryke-docker` examples use:

```
#!/usr/bin/env stryke
# examples/metrics.stk - fire a few metrics at a StatsD daemon.
# Run:  STATSD_ADDR=127.0.0.1:8125 s examples/metrics.stk

use Statsd

for val $i (1:5) {
  Statsd::count("stryke.example.hits")
  Statsd::timing("stryke.example.latency_ms", 40 + $i)
}
p "sent 10 metrics"

```

The test asserts the pure helpers unconditionally — they need no daemon — and gates the send path behind an environment variable, so CI stays green on a machine with no StatsD:

```
# t/test_statsd.stk
use Statsd

assert_eq Statsd::format_metric("api.calls", 3, "c"), "api.calls:3|c",
  "counter line format"
assert_eq Statsd::format_metric("mem.used", 512, "g"), "mem.used:512|g",
  "gauge line format"

if (defined $ENV{STATSD_ADDR}) {
  val $r = Statsd::count("stryke.test.ci")
  assert_eq $r->{sent}, 1, "one datagram sent"
}
```

Step 7 – build and install

The manifest’s `[scripts]` verbs close the loop. From a checkout, build the cdylib and install it into the per-user store — the same two commands every chapter’s install section names:

```
cargo build --release
s pkg install -g .
```

That drops `stryke.toml` and `lib/{Statsd.stk, libstryke_statsd.{dylib,so}}` under `~/.stryke/store/statsd@0.1.0/`, and from that moment `use Statsd` resolves in any script. Published to a release, a consumer installs with `s pkg install -g github.com/you/stryke-statsd` and needs no Rust toolchain at all — the store receives a prebuilt cdylib for the host triple.

The standard repository layout

The files touched above are not arbitrary — they are the fixed layout every connector in this book uses, and the reason the chapters could cite paths like `lib/Postgres.stk`, `src/lib.rs`, `stryke.toml`, `examples/crud.stk`, and `t/test_*.stk` without ever explaining the structure. A connector repository is:

```
stryke-<name>/
Cargo.toml           # the cdylib crate: [lib] crate-type = ["cdylib"]
stryke.toml          # the package manifest: [package] [ffi] [scripts]
src/lib.rs            # the extern "C" exports + dispatch + cache
lib/<Namespace>.stk   # the stryke wrappers: one fn per export, _decode/_req
examples/*.stk        # runnable demos, ping-guarded for the no-server case
t/test_<name>.stk     # the gated live suite + a surface test
README.md            # the primary source every chapter quotes
```

Larger connectors fan `lib/` out into several `.stk` files rather than one — the `stryke-aws` chapter documents “one `.stk` file per service namespace” (`S3.stk`, `Dynamo.stk`, `SQS.stk`, ...); `stryke-arrow` has one per format (`Arrow.stk`, `Parquet.stk`, `IPC.stk`, `CSV.stk`, ...); `stryke-polars` has 46 wrapper modules. The pure-stryke packages (`stryke-mcpd`, `stryke-utils`) drop `Cargo.toml` and `src/lib.rs` entirely and ship only `stryke.toml` + `lib/*.stk`. But the skeleton is the same, and a tool — or a reader — that knows this layout knows where to look in any connector in the fleet without being told.

Two files deserve emphasis as the *source of truth* the closing chapter returns to. `stryke.toml`’s `[ffi].exports` is authoritative for what the package exposes; if a wrapper and the manifest disagree, the surface test catches it. And the `README.md` is what every chapter in this book was written against — the per-package chapters quote it by section (`[0x00]`, `[0x04]`, “Architecture”, “API reference”), because it is the document the author maintains alongside the code.

What the model gives you for free

The reason a connector is this short is that the boundary supplies the hard parts. You did not design a calling convention, a threading model, an error protocol, or a memory-ownership rule — those are the fleet's, fixed once. What you wrote was the part that is actually specific to StatsD: the metric-line grammar, the UDP send, and the cache key. Swap those three for a REST call and a bearer token and you have a cloud connector; swap them for a driver handle and a URL and you have a database connector. The next two chapters cover what surrounds the code you just wrote — how to test it against the boundary's guarantees, and how to version and ship it so it survives in the store next to everything else in this book.

Testing and Packaging a Connector

A connector is not done when it compiles. It is done when it is tested the way the fleet tests, and packaged so the store, CI, and a consumer with no Rust toolchain all agree on what “installed” means. This chapter pulls the testing and packaging discipline out of the per-package chapters — where each states its own version — into the shared method they all follow.

The two-tier test split

Every connector in this book that has a `t/` directory splits its tests the same way, and the split falls directly out of the pure-helper tier described in the FFI-boundary chapter. There are two kinds of code in a connector: functions that touch the external system, and functions that do not. They get tested differently.

- `cargo test` covers the connection-free logic in the Rust crate — the pure helpers. DSN parsing and redaction, identifier and literal quoting, `LIKE` escaping, result-extraction, connection-key derivation, wire-format encoders. None of it opens a socket, so it runs anywhere, offline, in milliseconds. The `stryke-postgres` chapter names this “unit/contract, no live calls”; the `stryke-mssql` chapter lists exactly what it covers — “ADO string parsing/redaction, defaults, param extraction”; `stryke-clickhouse` runs “string/identifier escaping, URL parse/redact, connection-key, result-extraction — no server needed.”
- `s test t/` runs the stryke-level suite. This has two halves of its own. A *surface* test pins the wrapper API — `t/test_stryke_scylla_surface.stk`, `t/test_stryke_clickhouse_surface.stk`, `t/test_stryke_docker_surface.stk` each assert that every function named in `stryke.toml`’s export list actually resolves at runtime, so a manifest that drifts from the wrappers fails loudly. And a *live* test exercises the real system end to end, gated behind an environment variable so it short-circuits when the system is absent.

The gate is the whole trick

The gating is what lets one test file serve both a developer with a live database and a CI runner with none. The pattern is uniform across the fleet: probe once, branch on the probe, run the connection-free asserts unconditionally and the live asserts only when the probe succeeds.

```
use Redis

val $alive = eval { Redis::ping() }
unless ($alive) {
  p "redis not reachable - set REDIS_URL or start redis to exercise the demo"
  exit 0
}
# ... live asserts below only run when a server answered ...
```

The `stryke-redis`, `stryke-mongo`, `stryke-spark`, `stryke-docker`, `stryke-selenium`, and `stryke-aws` chapters all show this exact `eval { ping }` guard; `stryke-search` gates on `Search::ping(%conn)`; `stryke-kafka` gates the live round trip on `$KAFKA_BROKERS`; `stryke-neo4j` gates on `$NEO4J_URL`; `stryke-scylla` gates on `$SCYLLA_NODES`; `stryke-gcp` gates on ADC reachability plus opt-in `STRYKE_GCP_TEST_*` bucket/topic/sub vars. The rule is the same everywhere: **CI must pass with no external system**

present, and the same file must exercise the full surface when one is present. A test that fails because Postgres is not installed is a broken test, not a caught bug.

The per-system gate variables, collected:

Connector	Live-test gate
postgres	<code>\$DATABASE_URL / \$POSTGRES_URL / \$POSTGRES_DSN</code>
mysql	<code>\$MYSQL_URL</code>
mssql	<code>\$MSSQL_HOST</code>
mongo	<code>\$MONGODB_URI</code> (probe <code>Mongo::ping</code>)
redis	<code>\$REDIS_URL</code> (probe <code>Redis::ping</code>)
neo4j	<code>\$NEO4J_URL</code> (+ <code>\$NEO4J_USER / \$NEO4J_PASS</code>)
scylla	<code>\$SCYLLA_NODES</code>
clickhouse	<code>\$CLICKHOUSE_URL</code> (or <code>127.0.0.1:8123</code>)
search	<code>\$SEARCH_URL</code> (probe <code>Search::ping</code>)
kafka	<code>\$KAFKA_BROKERS</code>
docker	<code>\$DOCKER_HOST</code> (probe <code>Docker::info</code>)
spark	<code>spark-submit</code> on <code>PATH</code> (probe <code>Spark::ping</code>)
gcp	<code>ADC + \$STRYKE_GCP_TEST_BUCKET / _TOPIC / _SUB</code>
scrape	<code>\$SCRAPE_LIVE_URL</code>

Bringing a system up for the live test

The live tier is only useful if the system is easy to stand up, so the connectors that can be exercised against a throwaway container document exactly that. The `stryke-kafka` chapter ships a `bitnami/kafka:3.9` KRaft-mode docker one-liner; `stryke-docker` pulls `busybox:latest` and spins a uniquely-named container, cleaning up on exit; `stryke-email` points `$SMTP_HOST` at a local MailHog catch-all with `tls => none` on port 1025 so a bulk campaign can be tested without mailing anyone real; `stryke-mssql` runs its live asserts “against a throwaway container when one is available.” The throwaway-container idiom is the fleet’s answer to “how do I test a real round trip without a permanent dependency” — a uniquely-named, self-cleaning instance that exists for the length of one test run.

Testing what the boundary itself guarantees

Beyond the system-specific asserts, a good connector test exercises the boundary’s own contracts — the ones the FFI chapter enumerated. Three are worth asserting explicitly:

1. **The error envelope reaches stryke as a die.** Force a failure and confirm it surfaces as `Namespace::<op>: <reason>`, not a silent `undef`. The `stryke-mcpd` chapter’s test is the clearest model: `t/test_mcpd.stk` connects, calls a deliberately-failing tool (`always_fails`), asserts the result is an error envelope, and then calls a working tool *on the same connection* to prove the server survived the crash. That single test covers the envelope, the panic seam, and the cache all at once.
2. **The cache actually caches.** A second call against the same endpoint should reuse the first connection. This is hard to assert directly from stryke, but the *behavior* it enables — a batch on one checked-out connection — is testable: the `stryke-mysql` chapter documents `MySQL::transaction(\@statements, %opts)` precisely because “the pool cache means back-to-back calls may otherwise land on different pooled connections,” so a transaction test that expects atomicity is also a cache test.
3. **The pure helpers are inverses where they claim to be.** Many helpers ship in `parse_X / build_X` pairs — the `stryke-spark` chapter states “each with a documented inverse.” A round-trip assert (`build(parse(x)) == x`) is a cheap, high-value test that needs no external system and catches a whole class of grammar bugs.

Packaging: what “installed” means

Packaging is the second half of done. A connector produces two artifacts — the `stryke` library and the `cdylib` — and `s pkg install` places them, as a unit, into the per-user store at `~/.stryke/store/<name>@<version>/`. That directory is the discovery mechanism; the `stryke-arrow` chapter is explicit that “that directory is the entire discovery mechanism — no env vars, no PATH probing.” A `use Arrow` looks there, finds the `stryke.toml`, reads the `[ffi]` table, and `dlopens` the `cdylib` beside it.

There are two install paths, and every chapter names both:

```
# from a release - no Rust toolchain needed on the consumer machine
s pkg install -g github.com/MenkeTechnologies/stryke-<name>

# from a local checkout - build the cdylib first, then install
cargo build --release
s pkg install -g .
```

The release path is the important one for consumers. Because the store receives a *prebuilt* `cdylib` for the host triple, a user installing a connector never needs `cargo`, never compiles `librdkafka` or `libduckdb` or `libzmq` themselves, and never waits on the 3–5 minute cold builds that the `stryke-duckdb`, `stryke-kafka`, and `stryke-zmq` chapters warn about — those costs are paid once, on the build machine, and shipped as a binary.

Integrity: the SHA-256-verified release

The release path only holds up if the downloaded `cdylib` is the one the author built. The `stryke-search` chapter documents the mechanism: `s add github.com/MenkeTechnologies/stryke-search` “records the dependency in `stryke.toml` and pulls a prebuilt `cdylib` (SHA-256 verified) for the host triple from the GitHub release.” The hash check is what makes the no-toolchain install trustworthy — the consumer verifies the artifact against a recorded digest rather than re-deriving it from source. A connector you publish should ship per-triple `cdylibs` with their hashes recorded, so a consumer’s `s add` can verify what it fetched.

The `[scripts]` block as the interface

All of this — build, test, install — is reachable through the manifest’s `[scripts]` table, which is why every chapter’s build/test section reduces to the same three verbs:

```
[scripts]
build    = "cargo build --release"
test     = "s test t/"
install  = "s pkg install -g ."
```

Some connectors add a `make` front end over these (`stryke-scylla` documents `make debug` / `make test` / `make install`; `stryke-clickhouse` and `stryke-utils` mention `make install`), but the `make` targets just call the `[scripts]` verbs. The manifest is the interface; the Makefile, where present, is a convenience over it. A tool that knows how to read `[scripts]` knows how to build, test, and install any connector in the fleet without special-casing any of them — which is exactly what `s` is.

Versioning, the Store, and Compatibility

The fleet is thirty-odd independently versioned packages that must nonetheless behave as one system. This chapter is about how that holds together: what a version number means here, where installed packages live, how two versions coexist, and why the stryke core’s endgame stance makes backward compatibility a constraint rather than an aspiration.

Independent versions, one book

The first thing to internalize is that there is no single “ecosystem version.” Each connector carries its own version in its own `Cargo.toml` and `stryke.toml`, and those numbers are all over the map at the time of writing — `stryke-postgres` at 0.20.0, `stryke-mongo` and `stryke-mysql` at 0.21.0, `stryke-mcpd` and `stryke-utils` at 0.22.0, `stryke-scylla` at 0.3.0, `stryke-neo4j` at 0.5.0, `stryke-email` at 0.3.0. That spread is not disorder; it reflects how much each connector has moved. A package that has been through several wire-shape refactors (Postgres, MySQL) is numerically ahead of one that arrived recently (Scylla, Neo4j). The preface’s own warning applies directly here: “Versions move and export tables grow; a book that rounds those into permanent claims goes stale the day it prints.” Every version number in this book is the one the source carried at the time of writing, and is marked as provisional for exactly that reason.

A connector’s `Cargo.toml` version and its `stryke.toml` version are pinned to the same value — the `stryke-mysql` chapter calls this “dual-tracked in both `Cargo.toml` (Rust crate) and `stryke.toml` (stryke package manifest), both pinned to the same version,” and `stryke-kafka` says the same. The two files describe two halves of one artifact, so they move together.

The store is the runtime

Installed packages live in the per-user store at `~/.stryke/store/<name>@<version>/`. The `@<version>` in that path is not decoration — it is what lets the store hold more than one version of a connector at once, side by side, each in its own directory. `stryke-postgres@0.19.0` and `stryke-postgres@0.20.0` are two directories, and a project that pins the older one resolves to the older directory. The store is content-addressed by name and version; there is no single “current” symlink that a second install clobbers.

Each version directory holds the same layout: the `stryke.toml` manifest and the `lib/` directory with the `.stk` wrappers and the compiled `cdylib`. For a pure-stryke package — `stryke-mcpd`, `stryke-utils` — there is no `cdylib` and the directory holds only `stryke.toml` plus `lib/*.stk`; the `stryke-utils` chapter notes “the store directory ends up holding `stryke.toml` + `lib/*.stk` only — no compiled artifact.” The store layout is uniform whether or not there is native code behind the package.

What a version has to protect

Because these packages are dependencies of real scripts, a version bump has an obligation: it must not break a `.stk` file that worked against the previous version. Three surfaces are the compatibility contract a connector maintainer is signing up to protect.

- **The verb surface.** The `Namespace::verb` names and their argument conventions are the API. Renaming `Postgres::query_scalar`, or changing whether a `%opts` key is required, breaks callers. The

surface tests described in the previous chapter exist to catch accidental drift here — they assert the manifest and the wrappers agree — but intentional removals are the maintainer's responsibility to avoid or to stage across a deprecation.

- **The JSON type encoding.** The mapping from a system's types into JSON is part of the contract, because scripts read those shapes. If `stryke-mongo` changed `ObjectId` from `{"$oid": "..."} to a bare string`, every script that pattern- matches the extended-JSON form would break. The encoding decisions each chapter documents in its *type encoding* section are, in effect, versioned interface — changing one is a breaking change even though no function signature moved.
- **The connection resolution order.** Callers depend on the documented priority — an explicit `url =>`, then discrete parts, then the environment fallback. Reordering that, or renaming the fallback env var, silently redirects a script that relied on the old order. The `stryke-postgres`, `stryke-mysql`, and `stryke-redis` chapters each spell out their resolution order precisely because it is a stable interface, not an implementation detail.

The wire shape can change underneath

What a connector *is* free to change is everything below those three surfaces — and several have, dramatically. The recurring story in this book is the *v1 helper-binary* → *cdylib* refactor: `stryke-mysql`, `stryke-arrow`, `stryke-parquet`, `stryke-aws`, `stryke-kafka`, `stryke-gui`, and others each replaced an early design that forked a subprocess per call with the in-process `dlopen` model. That was a total rewrite of how a call crosses the boundary — and it was invisible to callers, because the verb surface, the type encoding, and the connection resolution stayed put. The `stryke-mysql` chapter even keeps the old “v1 wire shape (historical)” documented under a collapsed section while noting it “is no longer the live call path.” That is the compatibility model working as intended: the mechanism moved, the interface did not, and no script noticed.

Roadmap items are honest gaps, not silent ones

A connector at version 0.x has parts it has not built yet, and the fleet's convention is to name them rather than hide them. The pattern appears as a *what's not there yet* section in chapter after chapter: `stryke-mongo` lists change streams, multi-document transactions, GridFS, and `bulk_write` as open roadmap items; `stryke-redis` defers streaming `SUBSCRIBE` to v2 pending a stryke Unix-socket builtin; `stryke-k8s` and `stryke-docker` document `logs_follow / watch / exec / events / build as declared but dying` — the wrappers exist and fail with a clear message, because the current `StrToStr` wire shape cannot model a streaming callback yet. The `stryke-aws` chapter puts it best: a handful of ops “are documented as *deferred in the cdylib* rather than silently missing” — `S3::presign`, `Dynamo::batch_write`, Lambda's fire-and-forget mode all `die` with a clear message today.

This is a deliberate compatibility stance. A verb that will exist later is better declared and failing loudly now than absent, because a caller gets a precise error (`this op is deferred`) instead of an undefined-function surprise, and because the surface tests can pin the *intended* shape before the implementation lands. The gap is real, but it is documented, tested, and named — never faked.

Why compatibility is a hard constraint here

The stryke core's design horizon makes all of this non-negotiable rather than merely polite. stryke embeds inside `zshrs`, the compiled shell that is meant to be a permanent, 30-year-horizon replacement for the user's daily driver — the next chapter covers that embedding in detail. A shell that a user cannot trust to run yesterday's script is not a replacement; it is a migration. The same logic flows down to the batteries: a connector that breaks a working `.stk` on a version bump is a compat-floor regression, and the endgame stance treats those as catastrophic. The versioning discipline in this chapter — independent versions, a version-addressed store

that holds multiple versions at once, a three-surface interface contract, free rein below it, and honestly-named gaps — is what lets the fleet evolve fast without ever making a script owner afraid to upgrade.

Embedding the Fleet in zshrs

The batteries are stryke packages, and stryke is not only a standalone interpreter — it embeds inside `zshrs`, the compiled Rust shell, reached from the command line through an `@`-prefix dispatch. That embedding is what turns the fleet from “a scripting library” into “reach the shell itself has.” This chapter traces how a connector call gets from an interactive shell prompt down through stryke and across the FFI boundary, and why the shared bytecode VM underneath makes that path cheap.

The `@`-prefix seam

`zshrs` dispatches a stryke expression through an `@`-prefix. Where a bare command line is parsed and run as a shell command, an `@`-prefixed line is handed to the embedded stryke engine and evaluated as stryke. That single seam is the entire interface between the two languages at the interactive layer: shell syntax by default, stryke on demand behind `@`. It is the same synthesis stryke makes elsewhere — absorbing `awk`’s line-processing role into the language — extended to the shell, so that reaching a Postgres database or an S3 bucket from a prompt is a stryke one-liner, not a subshell into a separate tool.

Because the connector packages are ordinary stryke packages, they are reachable from that seam with no extra integration. A `use Postgres` inside an `@`-expression loads the connector exactly as it would in a `.stk` file — the `dlopen`, the namespace registration, the cache — all of it runs inside the shell’s own process. The fleet does not need a shell-specific binding; it inherits shell reach for free the moment stryke is the embedded engine.

Why in-process embedding matters for the fleet

The connection cache described throughout this book pays off differently, and better, when stryke is embedded rather than launched fresh per command. A standalone `s script.stk` invocation gets a process-lifetime cache that lasts exactly as long as that one script. But an embedded stryke engine living inside a long-running shell session has a process lifetime measured in *the whole login session* — which means a cached `postgres::Client`, a `FutureProducer`, a `bollard::Docker` connection, or a reflection `DescriptorPool` can survive across many separate `@`-expressions typed at the prompt over minutes or hours.

This is the anti-fork discipline the stack is built on, taken to its conclusion. The `stryke-kafka` chapter’s whole argument for the `cdylib` model — hold one producer alive so batching and compression work — becomes even stronger under embedding: the producer can be alive for the entire session, and successive `@Kafka::produce(...)` lines at the prompt all ride the same warm producer. A shell that re-forked a client per command would throw that away; an embedded engine with a session-lifetime cache keeps it. The batteries were designed for a host that does not fork per call, and `zshrs` is exactly that host.

fusevm: the shared bytecode floor

stryke and `zshrs` are not two engines that happen to cooperate — they share a bytecode VM, `fusevm`. The preface names this directly: the stryke core ships “the bytecode VM it shares with `zshrs`.” Both languages compile down to the same bytecode and execute on the same VM, with a Cranelift JIT underneath. That shared floor is why the `@`-seam is cheap: crossing from shell context into stryke context is not a process boundary or a serialization step, it is a dispatch within one VM that already speaks both languages’ compiled form.

For the fleet, the significance is that a connector call originates as bytecode on the same VM the shell runs on. The path from an interactive `@Postgres::query(...)` to the `pg__query` cdylib export is: shell reads the `@`-line, hands it to the embedded stryke front end, stryke compiles it to fusevm bytecode, the VM executes the wrapper, and the wrapper makes the FFI call across the boundary the earlier chapter described. There is no extra interpreter layer inserted by the embedding — the same VM that would run the wrapper standalone runs it embedded. The JIT and the bytecode caches that make stryke fast make the embedded connector call fast too.

Portability rides along

The shared VM also carries the portability story the connectors depend on. fusevm bytecode is portable across architectures — the stack targets macOS aarch64, Linux x86-64, and Linux aarch64 as first-class, with the JIT covering x86-64 and aarch64 today. A connector's stryke wrapper is bytecode on that portable VM; only the cdylib underneath is per-triple native code, which is exactly why the store ships *per-host-triple* prebuilt cdylibs (the packaging chapter's SHA-256-verified release). The split is clean: the wrapper layer is portable bytecode shared with the shell, and the native layer is the one part that must be compiled per platform and is therefore the one part the release process builds per triple.

The batteries as the shell's reach

Put together, the embedding turns the connector fleet into something more than a library you call from scripts. When stryke is the engine inside zshrs, and the connectors are stryke packages, then every system in this book — every database, broker, cloud API, and browser — is reachable directly from the shell prompt through `@`, with the connection held warm across the session by the same cache model the standalone interpreter uses, executing on the same bytecode VM the shell itself runs on. The core stays slim because the batteries are opt-in; the shell gains reach into any of them the moment a session does `use`. That is the endgame argument applied to the fleet: not a shell with a scripting language bolted on, but one VM, one language surface, and an opt-in fleet of connectors that the shell and the scripts reach through the identical path.

Patterns Across the Fleet

The individual chapters each documented one connector in isolation. Real work rarely stays in one. A script pulls rows out of Postgres and writes them as Parquet; it drains a Kafka topic into ClickHouse; it scrapes a set of pages and lays the result into a spreadsheet. This chapter is about those compositions — the patterns that recur when two or more batteries are wired together in one stryke program. Every pattern here uses only verbs documented in the earlier chapters; the value is in the seams between packages, not in any new capability.

The pure-helper-first idiom

The single most common cross-cutting pattern is not a pair of connectors — it is the way every connector separates its pure helpers from its I/O verbs, and how a script leans on that. Each package's connection-free tier (DSN parsers, identifier quoters, `escape`, wire-format builders) exists so that the parts of a script that *shape* a request can run and be tested without touching the system, and only the final verb crosses the wire.

The clearest case is the connectors with no prepared-statement binding. The `stryke-scylla` and `stryke-clickhouse` chapters both show the same defensive shape: run untrusted input through the pure `escape` helper before it ever reaches an interpolated query.

```
val $name = Scylla::escape($user_input)
val @hits = Scylla::query("SELECT * FROM app.users WHERE name = '$name'", %conn)
```

That `escape` call is pure — it opens no socket, it is unit-tested in the crate — and it is the boundary between trusted query text and untrusted value. The pattern generalizes: build with the pure tier, fire with the I/O tier, and the surface that touches the network is as small as possible.

Extract, transform, land: Postgres to Parquet

A recurring shape is moving rows out of an operational store and into a columnar file for analysis. `stryke-postgres` reads; `stryke-arrow` writes Parquet. The `Arrow::convert` verb is a server-side reader-to-writer pipeline, but when the source is a live query rather than a file, the composition is a `query` into a `write`:

```
use Postgres
use Arrow

val @rows = Postgres::query(
  "SELECT region, product, revenue, units FROM sales WHERE ts >= \"${1}\",
  [\"2026-01-01\"], url => $ENV{DATABASE_URL})

Arrow::write("sales_2026.parquet", @rows, compression => "zstd")

val $stats = Arrow::stats("sales_2026.parquet")
p sprintf("wrote %d rows × %d cols", $stats->{num_rows}, $stats->{num_columns})
```

The seam is the JSON value model. Postgres rows come back as stryke hashes because the connector decoded them from its JSON boundary; `Arrow::write` accepts exactly that shape because its boundary speaks the same values. Neither package knows about the other — they compose because both sit on the one value model the FFI boundary establishes.

Land and query in place: Parquet to DuckDB

Once data is Parquet on disk, `stryke-duckdb` queries it with no load step — `read_parquet / read_csv_auto` infer the schema and DuckDB’s vectorized scanner does the work. The `stryke-duckdb` chapter’s own example groups a CSV with no `CREATE TABLE`; the same idea points at the Parquet the previous pattern wrote:

```
use DuckDB

val @top = DuckDB::query(
  "SELECT region, sum(revenue) AS rev
    FROM read_parquet('sales_2026.parquet')
    GROUP BY region ORDER BY rev DESC")
for val $r (@top) { p sprintf("%-8s %d", $r->{region}, $r->{rev}) }
```

The `stryke-parquet` chapter’s `examples/discover.stk` already composes with its sibling this way — synthesizing rows via `Arrow::write`, then inspecting them with `Parquet::inspect` — so the three data packages (`arrow`, `parquet`, `duckdb`) form a small pipeline family: Arrow writes, Parquet inspects, DuckDB queries, all against the same file, all offline.

Stream to store: Kafka to ClickHouse

An operational pattern rather than an analytical one: drain messages off a topic and bulk-insert them into a columnar store. `stryke-kafka` consumes; `stryke-clickhouse` inserts via `FORMAT JSONEachRow`. The `consume` verb returns a snapshot of messages, and `insert` takes an array of row hashes:

```
use Kafka
use Clickhouse

val @msgs = Kafka::consume("events", group => "sink", limit => 1000, brokers =>
  ~ $ENV{KAFKA_BROKERS})
val @rows = map { from_json($_->{value}) } @msgs

Clickhouse::insert("events", \@rows, url => "http://127.0.0.1:8123")
p "sank " . scalar(@rows) . " events"
```

The `stryke-clickhouse` chapter notes `insert` “sends the array of row hashes as `FORMAT JSONEachRow` and returns the row count actually sent” — which is precisely the shape a batch of decoded Kafka values already has. The connectors’ shared JSON value model is again what makes the two ends fit.

Scrape to spreadsheet: web to Office

A different corner of the fleet composes just as cleanly. `stryke-scrape` extracts records from HTML; `stryke-office` writes them into an `xlsx`. The scrape chapter’s `extract_table` returns table rows as records, and the office chapter’s `sheet_write` takes rows-of-arrays:

```

use Scrape
use Office

val $page = Scrape::fetch("https://example.com/pricing")
val $rows = Scrape::extract_table($page->{html})

val @grid = ([ "Feature", "Value" ])
push @grid, [ $_->{Feature}, $_->{Value} ] for @$rows

Office::sheet_write("pricing.xlsx", [{ name => "Pricing", rows => \@grid }])
p "wrote pricing.xlsx (" . scalar(@$rows) . " rows)"

```

Fetch on the network tier, extract on the pure tier, write to a local file — the same extract/land shape as the Postgres-to-Parquet pattern, with a web page as the source and a workbook as the sink.

The composite-verb pattern

Several connectors fold a common multi-step loop into a single verb, and recognizing the pattern saves hand-rolling it. The `stryke-aws` chapter's `SQS:::pump` is “receive → callback → delete-on-success”; the `stryke-gcp` chapter's `PubSub:::pump` is “pull → run the callback per message → ack, repeated for `iterations`”; the `stryke-azure` chapter's `Queue:::pump` is the same “receive → callback → delete” loop. These are the same idiom — a poll/handle/acknowledge cycle — expressed once as a verb so a subscriber is one call:

```

use GCP::PubSub

val $n = GCP::PubSub:::pump($sub, iterations => 5, max => 10,
  callback => sub ($m) { p "got $m->{message_id}: $m->{data}" })
p "handled $n messages"

```

The lesson for composition is to look for the composite verb before writing the loop. When a connector offers a `pump`, a `transaction`, or a `convert`, it has already packaged the multi-call pattern that a naive script would spell out — and usually with the correctness the naive version misses (delete-only-on-success, one-connection atomicity, server-side streaming).

Secret to connection: a vault feeding a database

A production script rarely has its database password in hand — it has the right to *fetch* one. Three cloud connectors expose a secret store with the same verb shape: `stryke-aws`'s Secrets Manager, `stryke-gcp`'s Secret Manager, and `stryke-azure`'s Key Vault Secrets. Any of them composes with a database connector by resolving the secret first and threading it into the connection at the call site — never storing it in the manifest, exactly as the configuration appendix requires:

```

use AWS::Secrets
use Postgres

val $dsn = AWS::Secrets:::get("prod/postgres/dsn")

val @rows = Postgres:::query("SELECT count(*) AS n FROM orders", url => $dsn)
p "orders: $rows[0][n]"

```

The seam is the same one every pattern in this chapter uses: the secret comes back as an ordinary stryke scalar because the AWS connector decoded it from its JSON boundary, and `Postgres:::query` takes it as `url =>` because connection resolution is per-call. The vault connector and the database connector share no code and know nothing of each other; they compose because the value model and the call-site configuration rule are fleet-wide. Swap `AWS::Secrets:::get` for `GCP::Secrets` or `Azure::Secrets:::get` and the shape is identical — the same consistency the `stryke-azure` chapter notes when it says its service map “deliberately

mirrors `stryke-aws`.”

Guarding a multi-connector script

The last cross-cutting pattern is defensive, and it comes straight from the test discipline: when a script touches several systems, guard each with the same `eval { ping }` probe the examples use, so a missing dependency degrades to a clean skip rather than a crash midway. The `stryke-aws whoami.stk` example guards its STS and S3 calls independently; a multi-connector job should do the same per system:

```
use Postgres
use Redis

val $pg_ok    = eval { Postgres::ping(url => $ENV{DATABASE_URL}) }
val $redis_ok = eval { Redis::ping() }

unless ($pg_ok && $redis_ok) {
  p "need both postgres and redis reachable; skipping"
  exit 0
}
# ... the real work, now that both backends answered ...
```

This is the same principle the testing chapter made for CI, applied to production scripts: probe before you commit to the work, and let an absent system be a graceful exit rather than a stack trace three calls in. Across the fleet, the `ping/info/ping-in-eval` verb is the universal liveness check, and every connector exposes one for exactly this.

Appendix A – The Catalogue of the Fleet

This appendix is the fleet at a glance: one row per connector, drawn entirely from the per-package chapters. Every figure here is repeated from the chapter that documents it, and carries the same caveat the preface states — versions, export counts, and wrapped-crate versions are taken from each package’s source at the time of writing and will move. Where a chapter did not state an export count, the cell reads “—”; nothing here is inferred beyond what its chapter asserts.

Databases

Package	Ver	Namespace	Wraps	Exports
stryke-postgres	0.20.0	Postgres	postgres 0.19 (sync)	68
stryke-mysql	0.21.0	MySQL	mysql 28	70
stryke-mssql	—	Mssql	tiberius 0.12 (TDS, pure-Rust)	34
stryke-mongo	0.21.0	Mongo	mongodb 3 + bson 2	59
stryke-redis	0.19.0	Redis	redis 1.x	150+
stryke-neo4j	0.5.0	Neo4j	neo4rs 0.8 (Bolt)	40
stryke-scylla	0.3.0	Scylla	scylla 1 (CQL)	33
stryke-clickhouse	0.4.0	Clickhouse	ureq 2 (HTTP)	—
stryke-search	0.3.0	Search	ureq 2 (ES/OpenSearch REST)	137

Data & analytics

Package	Ver	Namespace	Wraps	Exports
stryke-arrow	0.19.0	Arrow	arrow/parquet 58	48
stryke-parquet	0.19.0	Parquet	parquet 58	—
stryke-polars	—	Polars	polars 0.45 + ndarray/nalgebra	— (1,505 wrapper fns)
stryke-duckdb	—	DuckDB	duckdb 1 (bundled)	31
stryke-spark	0.18.0	Spark	embedded PySpark + spark-submit	55

Messaging & RPC

Package	Ver	Namespace	Wraps	Exports
stryke-kafka	0.18.0	Kafka	rdkafka 0.39 (cmake-build)	56
stryke-zmq	0.17.0	Zmq	zmq 0.10 (vendored libzmq)	~60
stryke-grpc	0.18.0	Grpc	tonic 0.14 + prost-reflect 0.16	44

Cloud & infrastructure

Package	Ver	Namespace	Wraps	Exports
stryke-aws	0.19.0	AWS	aws-sdk-* 1 (17 services)	100+
stryke-gcp	0.19.0	GCP	reqwest REST + google-cloud-auth 1	~75
stryke-azure	0.18.0	Azure	Azure SDK for Rust 1 (+ Cosmos preview)	90
stryke-k8s	0.19.0	K8s	kube 0.95 + k8s-openapi 0.23	—
stryke-docker	0.19.0	Docker	bollard 0.17	68

Web & automation

Package	Ver	Namespace	Wraps	Exports
stryke-scrape	—	Scrape	spider 2 + scraper 0.20	—
stryke-selenium	0.17.0	Selenium	thirtyfour 0.35 (WebDriver)	90
stryke-mcpd	0.22.0	Mcpd	stryke-core MCP builtins (pure-stryke)	— (81 fns / 4 libs)
stryke-email	0.3.0	Email	lettre 0.11 (SMTP)	27

Documents & foundation

Package	Ver	Namespace	Wraps	Exports
stryke-office	0.18.0	Office	<code>calamine/rust_xlsxwriter/lopdf/docx-rs/image</code>	— (several hundred fns)
stryke-utils	0.22.0	Utils	nothing external (pure-stryke)	— (149 fns / 6 libs)
stryke-gui	0.19.0	GUI	<code>enigo 0.6 + xcap 0.9 + arboard 3</code>	~60

Two packages that are not cdylibs

Two entries above have no `Cargo.toml`, no `[ffi]` table, and no compiled artifact, and they are worth calling out because they show the package model is not synonymous with “wraps a Rust crate.” `stryke-mcpd` is pure stryke: it wraps stryke *core’s own* MCP builtins and adds the discipline layer (spec validation, crash isolation, a jailed tool pack) those primitives do not impose, shipping as `.stk` modules only. `stryke-utils` is pure stryke too: six sublibraries of composite helpers cross-checked against core’s `%b` builtin table at build time so they never collide with a builtin. Both install into the same store, load through the same `use`, and answer to the same `[scripts]` verbs as the cdylib packages — the package model is the unit of distribution, and native code is optional inside it.

Appendix B – Connection & Configuration Reference

The recurring answer to “how does this connector find the system” is: per-call options first, an environment-variable fallback second, and no credentials in the manifest ever. This appendix collects the connection surface of every connector into one place. Each row is the primary connection input and the environment- variable fallback its chapter documents; the full option set lives in the per-package chapter’s *config surface* section.

The universal rule

Three properties hold across the entire fleet, stated in chapter after chapter:

1. **Configuration lives at the call site, not the manifest.** The `stryke.toml` carries the `[ffi]` export table and the `[scripts]` verbs — never a connection string or a credential. The `stryke-postgres` chapter states it plainly: “connection state lives at the call site, not the manifest.”
2. **Resolution is ordered: explicit, then discrete, then environment.** A call takes an explicit `url =>` (or `uri =>`, `target =>`, `addr =>`); failing that, discrete parts (`host/port/user/...`); failing that, a package-specific environment variable. The order is a stable interface, not an implementation detail.
3. **Auth defers to the system’s own model.** Cloud connectors defer to the platform credential chain — AWS’s `env/~/.aws/IMDS` chain, GCP’s ADC, Azure’s `DeveloperToolsCredential` (`az login`) — rather than inventing a credential store. Kubernetes defers to kubeconfig; Docker to `$DOCKER_HOST`. The connector never handles secrets it can make the platform handle.

Connection surface by connector

Connector	Primary input	Env fallback
postgres	<code>url =></code> or <code>host/port/user/password/database</code>	<code>\$DATABASE_URL</code> / <code>\$POSTGRES_URL</code>
mysql	<code>url =></code> or discrete parts	<code>\$MYSQL_URL</code>
mssql	<code>url =></code> (ADO string) or <code>host/port/database/username/password</code>	<code>\$MSSQL_URL</code>
mongo	<code>uri =></code> (per-call, cached separately)	<code>\$MONGODB_URI</code>
redis	<code>url =></code> or <code>host/port/db/username/password/127.0.0.1:6379</code>	<code>\$REDIS_URL</code> (then <code>127.0.0.1:6379</code>)
neo4j	<code>%conn</code> (<code>uri/user/password/database</code>)	<code>\$NEO4J_URL</code> (+ <code>\$NEO4J_USER/\$NEO4J_PASS</code>)
scylla	<code>node/host/port/nodes/keyspace/username/password</code>	<code>\$SCYLLA_HOST</code> / <code>\$SCYLLA_PORT</code> / <code>\$SCYLLA_USERNAME</code> / <code>\$SCYLLA_PASSWORD</code>

Connector	Primary input	Env fallback
clickhouse	<code>url => or host/port/tls/username/password/database</code>	<code>\$CLICKHOUSE_URL</code>
search	<code>url => or host/port/tls/username/password/api_ key</code>	<code>\$SEARCH_URL</code>
arrow / parquet	local files — no connection	—
polars	in-memory — no connection	—
duckdb	<code>db (path or :memory:), session, read_only</code>	—
spark	<code>master => (default local[*]), spark_home, confs</code>	<code>\$SPARK_HOME / \$JAVA_HOME</code>
kafka	<code>brokers =></code>	<code>\$KAFKA_BROKERS</code> (then localhost:9092)
zmq	per-call endpoint string (<code>tcp://</code> , <code>ipc://</code> , <code>inproc://</code>)	—
grpc	<code>target => (host:port), plaintext, headers, timeout_ s</code>	—
aws	per-call <code>region/profile/endpoint</code> overrides	AWS chain; <code>\$AWS_ENDPOINT_URL</code>
gcp	per-call <code>project =></code>	ADC; <code>\$GOOGLE_APPLICATION_ CREDENTIALS / \$GOOGLE_CLOUD_ PROJECT</code>
azure	per-call <code>account/vault/subscription/name</code>	<code>\$AZURE_STORAGE_ACCOUNT / \$AZURE_COSMOS_ACCOUNT / \$AZURE_KEYVAULT_NAME / \$AZURE_SUBSCRIPTION_ID</code>
k8s	per-call <code>context/kubeconfig/namespace</code>	<code>\$KUBECONFIG / ~/.kube/config</code> / in-cluster SA
docker	per-call <code>host =></code>	<code>\$DOCKER_HOST</code> (else local socket)
scrape	per-call URL argument + crawl <code>%opts</code>	<code>\$SCRAPE_LIVE_URL</code> (test only)
selenium	<code>Selenium::open(browser =>, url =>, headless =>)</code>	— (WebDriver at <code>http://localhost:9515</code>)
mcpd	jail root + log path (no network config)	—
email	<code>host/tls/port/username/password</code>	<code>\$SMTP_HOST / \$SMTP_USER / \$SMTP_PASS</code>
office	local files — no connection	—
utils	none (pure-stryke)	—
gui	local desktop — OS permission grants, not config	<code>\$STRYKE_TARGET</code> (build triple)

The env-fallback convention

The environment-variable fallback is the fleet's concession to the shell. A script that always talks to one database should not have to thread `url => $dsn` through every call; setting `$DATABASE_URL` once and omitting it everywhere is the intended ergonomics, and the resolution order guarantees an explicit argument still wins when one is given. This is why the fallback names are stable interface: renaming `$DATABASE_URL` would silently break every script that relied on the omit-and-inherit pattern. The names above are the ones each chapter documents, and they follow the platform's own conventions where one exists (`$DATABASE_URL`, `$KUBECONFIG`, `$DOCKER_HOST`, `$AWS_*`, `$GOOGLE_APPLICATION_CREDENTIALS`) rather than inventing a `STRYKE_`-prefixed name for a variable a system already defines.

Appendix C – The Pure-Helper Layer

Every `cdylib` connector in this book exports two kinds of function: verbs that touch the external system, and *pure helpers* that do not. The pure layer is easy to overlook — it is documented as a footnote in most chapters — but taken across the fleet it is a substantial, connection-free library in its own right, and this appendix pulls it together as one surface.

What “pure” means and why it exists

A pure helper opens no socket, needs no credential, and touches no external system. It is a string or data transform over the *conventions* of a system: its connection-string grammar, its identifier-quoting rules, its wire encodings, its name-validity rules. Because it does no I/O, three things follow, and the chapters state all three repeatedly. It is unit-tested directly in the Rust crate, so CI validates it with no live server. It runs anywhere, offline, with no setup. And it is the safe building block for constructing requests — the `escape-before-interpolate` idiom from the patterns chapter lives entirely in this layer.

The `stryke-clickhouse` chapter is representative: “the ‘pure helpers’ group — `escape`, `quote_ident`, `parse_url`, etc. — takes no connection at all and is unit-tested directly in the Rust crate, so it validates in CI without a live server.” Nearly every chapter has a sentence like it.

The families that recur

Across connectors the pure helpers fall into a handful of families that appear again and again, because the same problems recur at every system boundary:

- **Connection-string / URL grammar** — `parse_url` / `build_url` / `redact_url`, `parse_dsn` / `build_dsn`, `parse_endpoint` / `build_endpoint`. Present in postgres, mysql, mssql, mongo, neo4j, clickhouse, search, scylla, spark, grpc, zmq, aws, azure, docker, selenium, email. The `redact_url` variant — strip the password before logging — is a fleet-wide courtesy.
- **Identifier & literal quoting** — `quote_ident`, `quote_literal`, `valid_identifier`, `escape`, `escape_like`. Every SQL and CQL connector ships these (postgres, mysql, mssql, scylla, clickhouse, duckdb), each in its own dialect: Postgres double-quotes, MySQL backticks, ClickHouse and Scylla their own rules. This is the layer that makes safe query construction possible without a prepared-statement API.
- **`parse_X` / `build_X` inverse pairs** — the round-trip parsers the testing chapter singled out. `stryke-spark` has the densest set — master URLs, table names, memory strings, JDBC URLs, conf lines, partition paths, duration and storage-level strings, Maven coordinates, “each with a documented inverse.” `stryke-docker` parses the string grammars its CLI flags use (`-v src:dst:opts`, `--memory 512m`, `--restart on-failure:3`, `--platform linux/arm64/v8`). `stryke-k8s` does quantity math (`parse_quantity`/`format_quantity`/`compare_quantity`/`sum_quantities`).
- **Wire-format & protocol helpers** — pure functions over a protocol’s own encoding rules, computed with no round trip. `stryke-kafka` ports Kafka’s partitioner exactly — `partition_for_key` is “a faithful port of Kafka’s `Utils.murmur2` with seed `0x9747b28c`” — plus assignment simulators (`range_assignment`, `roundrobin_assignment`, `sticky_assignment`). `stryke-grpc` encodes status codes, `grpc-timeout` values, and metadata-key rules; `stryke-zmq` has the z85 codec and CURVE key helpers; `stryke-selenium` does cookie domain/path matching per RFC 6265 §5.1.3/§5.1.4 and CSS/XPath escaping.

- **Name & value validators** — `valid_bucket_name`, `valid_index_name`, `valid_metric_name`, `valid_storage_account_name`, `valid_blob_name`, `valid_cosmos_id`, `normalize_guid`. Cloud connectors especially, where a malformed name is a 400 from the API — validating locally catches it before the request leaves the machine.

Why this layer is worth naming

Three reasons the pure layer deserves to be seen as a whole rather than a per-chapter footnote.

First, it is *testable coverage that needs no infrastructure*. A connector with a rich pure layer has a large fraction of its behavior verified in `cargo test` alone — the escaping, the URL grammar, the partitioner math — which is why the fleet's CI can be meaningful without standing up thirty external systems.

The `stryke-kafka` partitioner is the sharpest example: a faithful `murmur2` port means a script can compute *which partition a key will land on* without a broker, which is both a testable pure function and a genuinely useful offline capability.

Second, it is the *safe-construction layer*. Several connectors — scylla, clickhouse, spark — have no prepared-statement binding, so the only safe way to put a value into a query is to run it through the pure `escape` helper first. The pure layer is not a convenience there; it is the security boundary.

Third, it embodies the fleet's honesty about scope. A connector that has not yet built a live streaming verb can still ship the pure helpers for that protocol — the validators, the encoders, the parsers — and those are fully real even while the I/O verb is a documented roadmap gap. The `stryke-kafka` chapter does exactly this: `Kafka::security_protocol` exists as an offline canonicalizer for `PLAINTEXT/SSL/SASL_PLAINTEXT/SASL_SSL` values “ahead of that wiring landing.” The pure layer lets a package be genuinely useful at the edges of a system before the full connection to that system is finished — real capability shipped now, not a placeholder for capability promised later.

Reading the Source Alongside This Book

This book was written from source, and it is meant to be read against it. Every chapter cites the same four primary artifacts, and a reader who wants to go past what these pages document should open them in this order.

Start with `stryke.toml`. Its `[ffi]` table is the authoritative export list — the exact set of symbols the `cdylib` promises and the runtime registers — and its `[scripts]` table is the build/test/install interface. When a chapter says a package exports *N* symbols, `stryke.toml` is where that number comes from, and it is the file that will have grown by the time you read this. The preface's caveat is worth repeating one last time: export tables grow and versions move, so treat every count and version in these pages as the source's value at the time of writing, and let `stryke.toml` be the current truth.

Then read `lib/*.stk`. These are the wrappers — the real stryke-facing API, the `Namespace::verb` functions a script actually calls, and the centralized `_decode` and `_req` helpers that own error handling and connection merging. If you want to know precisely what arguments a verb takes, the wrapper is more reliable than any prose, because the wrapper is what runs.

Read `Cargo.toml` for the dependency truth — which crate is wrapped, at which version, with which features — and `examples/` (or `t/`) for a runnable call in context. Every usage block in this book was lifted or adapted from one of those, under the rule the preface states: no stryke syntax appears in these pages that was not copied or adapted from code that ships in the repo.

The map this book draws is stable — the package model, the FFI boundary, the cache, the two-tier test split, the versioned store — because those are the fleet's architecture, and architecture is what a book can fix in print. The verb tables and version numbers are the parts that move. Read the prose here for the shape; read the source for the current surface. Between the two, the fleet is fully documented, and it stays documented as it grows.